

信道编码、交织与译码方案技术报告

待补

2026 年 8 月 11 日

目录

插图

表格

第一章 编制目的与任务要求

1.1 编制目的

本任务面向类星导航信号体制中的低速导航电文和高速导航电文，研究电文从有效信息输入到接收端恢复之间的信道编码、译码与交织方案。教师附件给出的工作起点不是单一编译码器，而是两类业务、三类编码以及七组仿真任务：低速电文只研究 BCH，高速电文在卷积码和 5G NR LDPC 两类候选之间比较；交织作为突发错误条件下的独立测试项处理（教师附件第 1、6-7 页）。

不同业务对冗余、可靠性和处理时延的容忍度不同，因而不能用一种所谓“最强编码”覆盖全部电文。低速电文更强调接收稳健性、实现简单和译码时延可控；高速电文除可靠性外，还要考虑连续处理、有效吞吐、软判决收益以及迭代译码造成的时间波动。报告需要保留这些条件，比较的是具体业务约束下的适用性，而不是脱离输入长度、码率和信道的抽象强弱。

多信道、译码算法和交织分别回答不同问题。多信道试验检查既定码型遇到 AWGN、多径、频偏、遮挡或突发错误时的性能变化；译码算法对比关注同一码字条件下的纠错能力、运算量、存储量和时间；交织试验只考察错误离散化对 BCH 或卷积码的改善，并记录额外缓冲和时延。三类问题若混在同一曲线中，无法判断差异究竟来自编码结构、译码规则、信道样本还是交织器。

最终成果用于后续全数字仿真、半实物或样机实现以及测试验证。为此，曲线和结论必须能够追溯到输入长度、实际码率、信道条件、随机样本

身份、停止原因和实现版本，而不能只给出孤立的 BER 或 FER 数值。

1.2 业务对象与设计范围

教师附件把电文分为低速和高速两类业务。这里的“低速”和“高速”沿用任务分类，不在缺少信号体制依据时推定具体符号率、比特率或播发周期。

低速电文包含 200 bit 和 300 bit 两种原始有效长度，只研究 BCH。设计同时保留分组和整块两种组织方式：分组方案把一帧电文拆成若干短 BCH 信息块，整块方案把完整 payload 适配为较长的缩短 BCH 码字。比较内容包括可靠性、不同信道下的稳健性、实现复杂度和译码时间。

高速电文只保留 300 bit payload。卷积码承担连续处理和 Viterbi 译码候选，5G NR LDPC 承担短块迭代译码候选；两者需要在接近码率、相同 payload 和明确的信道条件下比较实时性、弱信号表现、冗余和译码代价。LDPC 只做整块短码适配，不配置交织。

上述 200 bit 和 300 bit 均指 K_{payload} ，即统计 BER、FER 时使用的原始有效信息长度。它们不是加入 filler、CRC 或尾比特后的编码器输入长度，不是编码模块输出长度，也不是打孔、缩短或速率匹配后的实际发送长度。各长度对象及其关系在第??章和第??章中分别从系统组织和仿真实现角度定义。

1.3 电文输入与编码约束

表 ??把教师要求和本报告的执行口径分开列出。教师附件规定了输入规模、候选码型和结果形式；实际长度、码率及字段命名仍以各任务实现和正式结果为准。

码率覆盖中的“附近”意味着短块适配后允许与名义码率存在差异。BCH 缩短、卷积码尾比特和打孔、LDPC filler 或速率匹配都会改变最终长度；报告使用实际长度计算码率，并在不能确认发送长度语义时保留限制

表 1.1: 主要设计约束及报告处理方式

约束项	教师要求	报告中的处理方式
输入长度	低速电文约 200 bit、300 bit; 高速电文只保留约 300 bit (第 1 页)	统一按 payload 记录, 附加位另列
总码长	单个编码块编码后总码长不超过 1000 bit (第 1 页)	逐任务核对编码块输出和实际发送长度, 不用母码参数代替
码率	覆盖 1/2、2/3、3/4、4/5 附近码率 (第 1 页)	报告 $K_{\text{payload}}/N_{\text{encoded}}$ 的实际值
BCH	分组短码与整块缩短 BCH 对比 (第 2-3 页)	用于 S1、S2、S6、S7 的低速候选
卷积码	硬/软 Viterbi, 整块及按时隙处理 (第 3-4 页)	用于 S3、S5、S6、S7 的高速候选
LDPC	BG2 短块, BP/NMS, 迭代及提前停止 (第 4-5 页)	用于 S4、S5、S6; Direct 实现边界单列
交织	独立测试; LDPC 不配置交织 (第 6 页)	基础曲线默认无交织, 仅在 S7 突发错误场景启用
输出	BER、FER、成功率、平均/最大时延、复杂度等曲线与表格 (第 7 页)	保存原始统计、参数表、图源字段及适用场景

说明。总码长约束按单个编码块检查，分组方案还要报告一帧由多少个子块组成。

1.4 编码、译码和交织任务要求

编码设计需要交代信息如何进入码字。BCH 需给出分组、整块、缩短和补零方式；卷积码需说明约束长度、母码、终止状态、打孔图样以及整块或时隙组织；LDPC 需说明 BG2 子矩阵、提升因子、filler 和 Direct 短块适配。所有方案都必须记录编码后长度，涉及未发送位时还要单列实际发送长度。

译码设计既评价恢复结果，也评价实现代价。短 BCH 分组可使用综合校验查表，较长 BCH 使用代数译码；卷积码比较硬判决与软判决 Viterbi，并记录量化、回溯深度、滑窗和状态连续方式；LDPC 比较 BP 与 NMS，记录最大迭代次数、提前停止、平均迭代次数和最坏观测时间。译码器声明成功、payload 实际正确、误纠和显式失败是不同状态，不能用一个“成功率”概括。

交织位于编码之后、BPSK 之前，去交织位于解调之后、译码之前。基础 AWGN 编码曲线不把交织器作为固定环节；S7 在突发错误或具有集中误差特征的信道下单独比较块交织、行列交织或伪随机交织。LDPC 始终保留无交织基线，不因 S7 增加交织支路。

1.5 S1–S7 仿真任务及其目的

七项任务构成由码内筛选到专题验证的顺序。表 ??列的是各任务的控制变量和输出用途，不预写尚未在对应章节审计的性能结论。

S1、S3、S4 先在各自码型内部减少候选数量，避免把未经筛选的参数组合全部带入跨方案比较。S2 和 S5 随后施加多信道条件，分别检验低速 BCH 和高速候选。S6 固定编码与信道条件后只比较译码器，S7 固定编码方案后只改变交织配置。这样可以把码型选择、信道适应性、译码实现和

表 1.2: S1–S7 仿真任务、控制变量与用途

任务	输入和候选	主要控制变量	输出	任务目的
S1	200/300 bit BCH, 分组与整块	码长、码率、分组数、纠错能力	BER、FER、成功状态、时延	在 BCH 内部筛选低速候选, 供 S2、S6、S7 使用
S2	S1 保留的 BCH	AWGN、多径、频偏、遮挡、突发错误	信道损失、FER、突发敏感性	检查低速候选的信道适用性
S3	300 bit 卷积码	码率、硬/软判决、量化、回溯、滑窗、时隙组织	BER、FER、有效吞吐、首输出及译码时间	筛选高速卷积码候选
S4	300 bit BG2 Direct LDPC	目标码长、BP/NMS、迭代上限、提前停止	BER、FER、迭代、复杂度、时延	筛选高速 LDPC 候选
S5	S3/S4 高速候选	接近码率、相同 payload、不同信道	BER、FER、时延和鲁棒性	比较卷积码与 LDPC 的场景适用性
S6	固定码长与信道的三类码	只替换译码规则	性能、操作计数、存储、平均/最大时间	分离译码算法本身的收益和代价
S7	固定 BCH 或卷积码方案	交织器、交织深度、突发长度	突发容限、FER 改善、附加时延	评价交织; LDPC 仅作无交织基线

交织收益拆成可解释的四类差异。

1.6 评价指标与成果形式

成果包括正式 BER/FER 主图、参数表、时延和复杂度表、场景说明以及可复现性记录。教师附件要求输出信噪比条件下的 BER、FER 曲线和表格，但没有指定横轴必须使用 E_b/N_0 或 E_s/N_0 。本项目规定正式 BER、FER 主图采用 E_s/N_0 ；仿真原始 E_b/N_0 、实际码率和转换后的 E_s/N_0 同时保留，转换不改变原始噪声或性能统计。

编码增益和交织改善属于曲线级结论，必须说明参考曲线、目标 BER/FER 和插值范围。方案推荐要先检查可靠性门限，再比较冗余、复杂度、存储和时延，最后结合信道与突发错误场景给出，不给出脱离业务条件的统一绝对排名。

表 1.3: 评价指标的工程含义与使用边界

类别	指标	使用边界
可靠性	BER、FER、payload 成功 率、译码器声明成功率、误 纠、译码失败	BER/FER 只统计恢复 payload；声 明状态与真实正确性分开
传输效率	编码后长度、发送长度、实 际码率、有效吞吐	附加位不进入 payload 分子；打孔后 长度必须按实现解释
实现代价	操作计数、平均迭代次数、 存储量、量化位宽	不用单一 CPU 时间替代算法复杂 度
时延	平均、P95/P99、最大观测、 首输出、结构时延、交织附 加时延	仅在任务实际提供相应字段时报告； 软件最大值不是硬件上界
场景表现	多信道损失、编码增益、交 织改善、适用场景	比较点需说明目标误码率、插值范围 和共同控制条件

1.7 本章小结

上述要求对应四个相互关联但不能混为一谈的问题：码型和码长筛选、信道适应性、译码实现代价，以及交织对突发错误的改善。后续章节先在 BCH、卷积码和 LDPC 内部确定候选，再开展多信道、译码器和交织专题比较；所有正式结果均需带有实际长度、SNR 口径和数据身份。

第二章 设计输入与总体方案

2.1 总体设计思路

本项目采用“公共仿真基础、可替换编码模块、任务级信道与统计配置”的组织方式。Common 位于三类编码模块之外，提供公共数据结构、BPSK、基础 AWGN、硬判决与 LLR、帧池和噪声池、payload 误码统计、停止控制、checkpoint、分片合并及公共结果字段。它不生成 BCH、卷积码或 LDPC 校验关系，也不执行三类码的核心译码，因此不是第四种编码算法。

BCH、CC 和 LDPC 模块接收相同类型的 payload 和公共控制信息，但各自负责码长适配、编码、译码、payload 恢复、算法状态以及任务专用指标。多径、频偏、遮挡和突发错误等扩展信道也由具体任务配置，不因 Common 提供 AWGN 基础接口就自动具有统一实现。

这种分工允许对比时固定公共环节，只替换目标变量。例如，比较卷积码硬、软 Viterbi 时可共享 payload、码字、打孔结果和接收符号；比较 LDPC BP 与 NMS 时可共享 payload、校验矩阵、噪声和 LLR。若具体任务没有做到这些共享条件，必须在对应章节记录偏离，不能借用 Common 的设计原则替代实际审计。

2.2 数据对象与长度关系

一帧电文经过适配、编码、速率调整和恢复时会出现多个长度。表 ??给出报告中的统一名称；各任务即使使用不同字段名，也应能够映射到这些对象。

表 2.1: 数据对象、长度符号及附加位关系

符号	数据对象	含义	附加位关系
K_{payload}	原始有效电文	用户需要恢复并用于 BER/FER 的比特	不含附加位
$K_{\text{codec input}}$	编码器输入	编码器实际接收的信息侧序列	可含 filler、CRC 或适配位
N_{encoded}	编码输出	Common 码率分母对应的最终编码长度	含校验、尾比特等
$N_{\text{transmitted}}$	实际发送序列	映射到 BPSK 并进入信道的比特数	不含未发送缩短位和打孔位
N_{received}	接收观测	进入硬判决或软信息计算的有效观测数	受接收处理影响
$K_{\text{recovered}}$	恢复 payload	去除附加位后提交统计的比特序列	应 等 于 K_{payload}

Filler 用于把短 payload 补到编码结构要求的输入规模，不增加码率分子；CRC 用于检错或停止判断，也不属于 payload；卷积码 tail 把编码器状态终止到约定状态，会增加编码输出长度；puncturing 从母码输出中删除部分比特，使实际码率提高；shortening 则把已知的信息位从发送序列中省略，接收端译码前按约定恢复这些位置。相同的 K_{payload} 因此可能对应不同的 $K_{\text{codec input}}$ 、 N_{encoded} 和 $N_{\text{transmitted}}$ 。

Common 的 CodeLengths 已定义上述主要长度和附加位字段。Common-04 基础汇总行只统一输出 payload length 与 encoded length；

transmitted length、CRC、tail、filler、puncturing 和 shortening 需要由任务结果 Schema 补充。公共汇总表缺少字段时，不能据此假定这些长度相等。

2.3 Common 与编码模块的职责分工

表 ??按数据流列出 Common 和编码任务的边界。公共接口的存在表示三类码可以接入同一处理框架，不表示具体任务已经使用完全相同的信道、统计字段或停止参数。

表 2.2: Common 与三类编码任务的职责分工

环节	Common	编码任务
payload 输入	确定性生成或从 Frame Pool 按帧索引读取	选择 200/300 bit 业务长度
码长适配	提供长度结构和公共字段	实现 filler、CRC、shortening、tail、puncturing
编码	提供编码器接口	实现 BCH、卷积码或 BG2 Direct LDPC
调制	固定实值 BPSK, 0 映射 +1, 1 映射 -1	通常直接复用
信道	AWGN 实现和信道接口	增加多径、频偏、遮挡、突发错误等
解调	硬判决和未量化 LLR	设置量化、裁剪、饱和或补偿
译码	提供输入/输出接口	实现查表、代数、Viterbi、BP 或 NMS
payload 恢复	提供统计入口和长度校验	去除 filler、CRC、tail 等附加位
统计	payload BER/FER、基础成功率、平均和最大时间	增加误纠、声明成功、迭代、复杂度、分位数等
审计	pool ID、hash、checkpoint、configHash、shard/merge	保存任务配置、manifest、命令和实现版本

2.4 低速 BCH 处理路径

低速路径从 200 bit 或 300 bit payload 开始。分组方案先按 BCH 信息位长度切分，不足一组时补入 filler；整块方案把完整 payload 映射到较长 BCH 母码并执行 shortening。编码产生一个或多个系统 BCH 码字，S7 可在编码后加入交织，随后进行 BPSK 和信道传输。接收端先硬判决，再在

启用交织时去交织，之后执行查表或代数 BCH 译码，删除补入或缩短对应的非 payload 位置，最后提交固定长度的恢复结果。

分组方案会产生多个短码字，一帧 FER 的判定取决于所有子块恢复后的 payload：只要任一 payload 位错误，整帧计为错误帧。整块方案的纠错边界、缩短位置和误纠状态由 BCH 章节结合实际实现说明。本章只规定数据如何流转，不据此预测哪种组织方式性能更好。

2.5 高速卷积码处理路径

高速卷积码输入固定为 300 bit payload。整块组织可在电文末尾加入零尾，使移位寄存器回到约定状态；按时隙处理可保留连续状态，并通过滑窗 Viterbi 降低等待完整电文的时间。项目以 1/2 母码为基础，通过打孔形成接近 2/3、3/4 的发送码率。打孔后的实际编码长度、尾比特数量和时隙边界都需要随结果保存。

接收端可将符号硬判为 0/1，也可把实值观测转换为 LLR 或量化软值。硬、软 Viterbi 共用相同码字和接收符号时，差异主要来自分支度量；回溯深度、滑窗长度、软值位宽和饱和策略会同时影响存储、首输出和译码时间。整块零尾、连续状态和时隙分块还会改变结构时延，不能只比较函数调用的 CPU 平均时间。

2.6 高速 LDPC 处理路径

LDPC 路径只处理 300 bit 整块 payload。编码模块依据 BG2 短块结构选择提升因子和使用的子矩阵，通过 filler 或 shortening 适配编码输入，并由 Direct QC-LDPC 编码器生成校验位。接收端计算软信息后，分别使用 Layered BP 和 Layered NMS 迭代译码；两者设置最大迭代次数，并在校验方程全部满足时允许提前停止。恢复阶段删除 filler 等非 payload 位置，只把原始 300 bit 范围交给公共统计。

本项目使用的是基于 5G NR BG2 结构的短块 Direct 实现，而不是默

认完整复现 5G NR 物理层传输链。是否加入标准 CRC、完整速率匹配、比特选择和接收恢复，必须以 LDPC 任务的实际源码、配置和结果字段为准。教师附件示例中的 576 bit 与后续项目某些 560 bit 实现偏离在冲突台账登记，前三章不提前裁定其性能影响。LDPC 不配置交织。

2.7 统一通信与仿真链路

三类码共享的上层数据流为：payload、码长适配、编码、可选交织、BPSK、信道、硬判决或 LLR、去交织、译码、payload 恢复和统计。下层控制链不改变码字内容，而是给每一帧绑定 Frame Pool、Noise Pool、seed/noise key、停止条件、checkpoint、分片范围、合并条件、CSV Schema 和 hash。

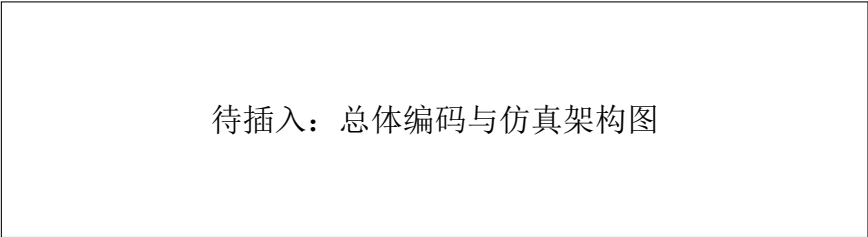


图 2.1: 总体编码与仿真架构图

结果整理阶段保留内部 ebN0Db 和实际码率，按每个点的 R 计算正式横轴 E_s/N_0 。该步骤只增加展示坐标，不改动接收符号、错误计数或停止结果。图中下层控制与审计链应明确连接到每个仿真点，而不是画成与数据流无关的附加说明。

2.8 S1–S7 的方案筛选关系

第 1 章按任务要求解释 S1–S7 “做什么”，本节只说明候选如何向后传递。S1、S3、S4 分别在 BCH、卷积码和 LDPC 内部筛选结构与参数；S2 接收 S1 的低速候选，S5 接收 S3、S4 的高速候选；S6 在固定方案和信道

下替换译码器；S7 在固定 BCH 或卷积码方案下替换交织器。最终推荐只使用完成相应 Gate 的候选。

表 2.3: 任务输出候选及后续使用关系

前序任务	输出候选或边界	后续使用
S1	低速 BCH 码长与分组/整块候选	S2 信道检查、S6 译码、S7 交织
S3	高速卷积码码率、组织和 Viterbi 参数	S5 多信道、S6 译码、S7 交织
S4	高速 LDPC 码长、BP/NMS 和迭代配置	S5 多信道、S6 译码；不进入交织支路
S2、S5	候选在指定信道下的可靠性和时延记录	场景适用性判断
S6	同条件下译码性能、复杂度、实现代价判断存储和时间	
S7	交织改善和附加时延	突发错误场景配置

2.9 综合评价和结果组织

工程决策首先检查候选在目标信道和可靠性指标下是否满足要求；未满足可靠性门限的方案不因码率高或平均时间短而优先。通过可靠性检查后，比较实际码率和附加冗余，再比较操作计数、存储、迭代次数、平均/尾部时间和结构时延。最后检查多信道损失、突发错误敏感性及交织代价，并按低速、高速及具体信道场景给出推荐。

结果文件按“身份、配置、原始计数、派生比例、时间、停止原因、审计字段”的顺序组织。报告不把所有指标压缩为无权重的综合分数，也不给出跨业务的单一绝对排名。可靠性门限、复杂度预算和时延上限若尚未由任务明确，则保留为适用条件，而不是替作者虚构阈值。

表 2.4: 三类编码处理路径的主要组织差异

项目	BCH	卷积码	BG2 Direct LDPC
业务输入组织	200/300 bit 低速 分组或整块缩短	300 bit 高速 整块零尾或时隙连续	300 bit 高速 整块短块适配
码率调整	码型选择、缩短、填充	母码、尾比特、打孔	提升因子、filler、Direct 结构
译码输入	硬判决	硬比特或量化软信息	LLR 或量化软信息
译码状态	无错、纠错、失败、误纠	路径输出及任务状态	校验满足、迭代上限等
交织	S7 可选	S7 可选	禁止配置
关键时延	子块处理与整帧完成	首输出、回溯、滑窗	迭代次数和尾部时间

2.10 本章小结

总体方案把公共数据和控制链与三类编译码实现分开：Common 固定可复用的基础调制、AWGN、随机样本、统计和审计机制，编码模块决定码长适配、编译码和专用状态。低速 BCH、高速卷积码和高速 BG2 Direct LDPC 因组织方式不同保留独立长度和时延字段；S1-S7 按内部筛选、场景检验、实现代价和交织专题逐级使用候选。

第三章 统一仿真方法

3.1 Common 公共仿真基础

Common 是三类编码共同使用的仿真与审计基础，不参与 BCH 生成多项式计算、卷积码状态转移或 LDPC 校验矩阵迭代。它提供统一类型和接口，固定实值 BPSK、基础 AWGN、硬判决和未量化 LLR 的定义，并通过 Frame Pool、Noise Pool、停止控制、checkpoint、分片合并和结果身份字段约束实验输入与输出。

公共实现解决的是“相同环节如何用同一种方式执行和记录”，不能替代任务级说明。BCH、CC 和 LDPC 的 filler、CRC、tail、puncturing、shortening、量化、译码状态、复杂度字段及扩展信道仍由各任务负责；历史实验是否调用当前 Common 接口，也需要在对应章节逐项复核。

待插入：Common 统一仿真与审计链路图

图 3.1: Common 统一仿真与审计链路图

图??的上层应展示 payload、编码、BPSK、信道、判决或 LLR、译码、恢复和统计；下层应展示 frame/noise pool、seed、stop、checkpoint、shard/merge、CSV 和 hash。两层以帧索引、pool ID、配置 hash 和结果行

连接，使一个性能点能够追溯到确定的输入与执行范围。

3.2 帧结构与长度定义

K_{payload} 是原始有效信息长度，也是 BER 分母中每帧贡献的比特数。 $K_{\text{codec input}}$ 是编码器实际输入，可在 payload 之外包含 filler、CRC 或其他适配位。 N_{encoded} 是 Common `computeCodeRate` 使用的编码输出长度；具体任务需要说明该字段是否已经完成打孔或速率匹配。 $N_{\text{transmitted}}$ 是实际映射到 BPSK 并进入信道的比特数， $K_{\text{recovered}}$ 是去除附加位后的恢复长度。

Shortening 对应的已知位不进入发送序列；punctured 位从母码输出中删除；tail 和 CRC 即使参与译码，也不属于 payload。统计前必须把恢复结果裁剪或映射回 K_{payload} 。若恢复长度不同，Common 的 `countBitErrors` 会拒绝比较，不能用补零或截断掩盖配置错误。

表 3.1: 长度字段及附加位对各长度的影响

项目	是否属于 payload	影响编码器输入	影响编码输出/发送	统计处理
filler	否	增加	可能增加	恢复后删除
CRC	否	通常增加	增加	可用于状态判断，不计 BER
tail	否	改变卷积码输入过程	增加输出	不计 payload BER
puncturing	否	不改变	删除部分母码比特	记录打孔后长度
shortening	否	逻辑上补已知位	已知位不发送	译码前恢复约定位置
parity	否	不属于信息侧	构成编码冗余	不直接计 payload BER

3.3 实际码率定义

现行源码在 `types.hpp` 的 `computeCodeRate` 中给出唯一公共码率：

$$R = \frac{K_{\text{payload}}}{N_{\text{encoded}}}. \quad (3.1)$$

分子不包含 filler、CRC 或 tail；分块方案的分母使用所有子块最终编码长度之和；打孔后若任务把打孔结果记为 `encodedLength`，分母使用打孔后长度。母码理论码率不能代替式 (??)。

$N_{\text{transmitted}}$ 仍需独立保存，用于解释物理发送效率和未发送位。Common-04 基础 `SummaryRow` 尚未输出该字段，因此正文中的 Common 码率必须按源码写为 `payload/encoded`，而不能把旧稿中的 `payload/transmitted` 定义强加给公共函数。若任务结果以 $N_{\text{transmitted}}$ 生成噪声或计算横轴，则在该任务章节登记偏离，并保证 AWGN 和横轴换算使用同一个 R 。

3.4 BPSK 离散符号模型

Common 的 BPSK 调制按逐比特实值映射：

$$x_i = 1 - 2c_i, \text{quad}c_i \in \{0, 1\}, \quad x_i \in \{+1, -1\}. \quad (3.2)$$

式 (??)把比特 0 映射为 +1，把比特 1 映射为 -1，每个实际编码比特对应一个实值符号。当前 Common 是离散符号级模型，不显式设置采样率、过采样倍数、成形滤波器或载波相位；涉及这些物理层环节的实验不能直接用本节公式覆盖。

3.5 E_b/N_0 驱动的 AWGN 实现

`computeAwgnSigma` 的输入字段为 `ebN0_dB`。函数先完成 dB 到线性量的转换：

$$\gamma_b = 10^{(E_b/N_0)_{\text{dB}}/10}. \quad (3.3)$$

式 (??)完成 dB 到线性量的转换；随后调用式 (??)计算实际码率，并返回标准差

$$\sigma = \sqrt{\frac{1}{2R\gamma_b}}, \quad \sigma^2 = \frac{1}{2R\gamma_b}. \quad (3.4)$$

式 (??) 中的 σ 进入下列接收模型:

$$y_i = x_i + \sigma z_i, \text{quad} z_i \sim \mathcal{N}(0, 1). \quad (3.5)$$

式 (??) 由 `applyAwgn` 实现: 该函数要求符号向量和标准高斯向量等长, 并拒绝非有限或非正的 σ 。这里的 σ 是标准差而不是方差; Noise Pool 保存的是标准高斯 z_i , 不是已经按某个 SNR 缩放的噪声。

上述闭式只直接适用于确实调用 `Common computeAwgnSigma` 和 `applyAwgn` 的实值 BPSK-AWGN 任务。多径、频偏、遮挡、突发错误或历史 runner 可能采用不同的输入 SNR 和归一化方式, 对应章节必须从源码和正式配置重新确认。

3.6 E_s/N_0 正式绘图口径

教师附件要求给出信噪比条件下的 BER、FER 曲线, 没有强制横轴为 E_b/N_0 或 E_s/N_0 。本报告把正式主图横轴规定为符号信噪比 E_s/N_0 , 同时保留仿真输入 `ebN0Db`。在一编码比特对应一个 BPSK 符号且使用式 (??) 时, 线性关系为

$$\gamma_s = R\gamma_b. \quad (3.6)$$

式 (??) 对应的 dB 换算为

$$(E_s/N_0)_{\text{dB}} = (E_b/N_0)_{\text{dB}} + 10 \log_{10} R. \quad (3.7)$$

式 (??) 的换算只增加 `esN0Db` 展示字段, 原始 E_b/N_0 、噪声样本、 σ 、BER、FER、处理帧数和错误计数均保持不变。不得把 `ebN0Db` 直接改名为 `esN0Db`, 不得用母码率替代实际 R , 也不得对所有曲线统一减 3 dB。不同码率曲线换算后横坐标不必重合; 若需要在同一目标 FER 比较, 应限定在各自数据覆盖范围内插值, 不能为了对齐网格改变原始点。

3.7 硬判决与软信息

Common 的硬判决在零点明确归入 bit 0:

$$\hat{c}_i = \begin{cases} 0, & y_i \geq 0, \\ 1, & y_i < 0. \end{cases} \quad (3.8)$$

式 (??) 给出硬判决；实值 BPSK-AWGN 的未量化对数似然比为

$$L_i = \ln \frac{\Pr(c_i = 0 | y_i)}{\Pr(c_i = 1 | y_i)} = \frac{2y_i}{\sigma^2}. \quad (3.9)$$

由式 (??) 可知，正 LLR 倾向 bit 0，负 LLR 倾向 bit 1，绝对值表示可靠性。Common 基础接口不执行裁剪、定点量化或饱和；CC 和 LDPC 对软值的位宽、缩放、裁剪和溢出处理必须在任务章节说明。BCH 使用硬判决输入。

3.8 BER、FER 与译码状态

公共错误统计只比较原始 payload 和恢复 payload:

$$\text{BER} = \frac{N_{b,\text{err}}}{N_{b,\text{payload}}}, \quad (3.10)$$

$$\text{FER} = \frac{N_{f,\text{err}}}{N_{f,\text{processed}}}, \quad (3.11)$$

$$S_{\text{payload}} = \frac{N_{f,\text{correct}}}{N_{f,\text{processed}}} = 1 - \text{FER}. \quad (3.12)$$

式 (??) 和式 (??) 分别按 payload 比特与帧计数。Common-04 **SummaryRow** 中的 **successRate** 对应式 (??)，即 payload 无误帧比例，不是译码器声明成功率。

Common checkpoint 结构已预留 reported success、miscorrected 和 decoder failure 等扩展计数，但基础 **SummaryRow** 没有统一输出这些比例。只有任务结果实际包含相应计数且定义与本表一致时，才能报告译码器声明成功率或误纠率。

表 3.2: 译码器声明与 payload 结果的联合状态

译码器声明	payload 结果	统计解释
成功	正确	true success,同时可计入 declared success
成功	错误	miscorrection 或 undetected error, 不能计入 payload success
失败	错误	decoder failure, 同时是 frame error
失败	正确	需由任务状态字段解释, 基础 Summary-Row 不单列

3.9 确定性随机噪声与公平比较

Common 的每个标准高斯样本由完整 NoiseKey 决定:

$$\mathcal{K}_{\text{noise}} = (s_{\text{master}}, g_{\text{noise}}, f, i, v_{\text{policy}}), \tag{3.13}$$

式 (??) 的五个分量依次表示主噪声种子、噪声组、帧索引、符号索引和策略版本, 对应源码中的 NoiseKey 字段。固定完整键产生相同样本, 不依赖随机函数调用顺序。源码使用 SplitMix64 派生均匀字, 并用 Box-Muller 变换生成标准高斯值。

snrIndex 和 Eb/N0 不直接进入 NoiseKey。不同 SNR 点通过式 (??) 改变 σ ; 是否跨 SNR 复用同一标准高斯底噪, 取决于 noiseGroupId 及任务配置。不同 frameIndex 仍生成不同帧噪声, 跨 SNR 复用不等于所有帧复用一条噪声。

CC 硬、软 Viterbi 的公平比较应共享 payload、母码输出、打孔结果和 receivedSymbols, 只替换译码输入和分支度量。LDPC BP/NMS 应共享 payload、码字、校验矩阵、噪声和 LLR, 只替换校验节点更新规则。共享随机样本使结果差异不再同时混入另一组 Monte Carlo 噪声; 具体任务是否完全遵守这些条件, 仍由对应章节的配置和调用链审计。

3.10 帧池和噪声池

Frame Pool 和 Noise Pool 都支持确定性分片与完整性检查，但文件内容和格式不同，不能互相类推。Frame Pool 保存 payload bits, manifest schema 为 `common03.frame_pool_manifest.v2`; payload 使用 `splitmix64_payload_v2` 生成，按 packed bits、LSB-first 写入。分片必须从 frame 0 开始连续覆盖，每片记录文件名、帧范围、字节数和 SHA-256, overall hash 对稳定 manifest 身份字段和分片 hash 计算。读取器默认复核分片长度与 SHA-256。

Noise Pool 保存未缩放的标准高斯样本，格式为 float64 little-endian, 生成算法标识为 `splitmix64_box_muller_v1`。每个二进制分片含版本化头字段，包括首帧、帧数、每帧符号数、主种子、噪声组和策略版本。manifest 要求连续帧范围、分片 SHA-256 和 overall hash, `noisePoolId` 取 overall hash 的前 16 个十六进制字符；读取时同时复核文件 hash 和头字段。

两个 pool 的大型正式文件不进入 Git。报告和 checkpoint 保存 pool ID 或 hash、配置 hash、帧范围和策略版本，使外部存储中的池文件仍能与结果绑定。

3.11 仿真停止控制

Common 正式停止配置为 `minFrames=5000`、`targetFrameErrors=200`、`maxFrames=50000`。evaluateStop 的实际判断顺序如下：

```
if processedFrames >= maxFrames:
    MAX_FRAMES
elif processedFrames < minFrames:
    CONTINUE
elif frameErrors >= targetFrameErrors:
    TARGET_FRAME_ERRORS
else:
```

CONTINUE

最小帧数防止在错误较多的点过早停止；达到最小帧数后，目标错误帧数提供基本的错误事件数量；最大帧数限制高 SNR 点或低错误率点的计算量。由于每个 SNR 点独立判断，processedFrames 可以不同，不能笼统写成“所有正式点均仿真 50000 帧”。历史实验或特殊任务若使用其他停止参数，需要在结果表中保存停止配置和 stopReason。

3.12 时延统计口径

表 3.3: Common 基础时延字段、实际计时边界和限制

指标	Common-04 identity pipeline 实际边界	提供	限制
avgEncodeTimeUs	payload 生成/Frame Pool 读取、恒等编码、BPSK	是	不是纯编码时间
avgChannelTimeUs	噪声生成/Noise Pool 读取及 AWGN 叠加	是	包含池读取或生成
avgDecodeTimeUs	硬判决，或 LLR 计算后按符号判决	是	identity 基线不是码译码器
maxDecodeTimeUs	单帧上述 decode 阶段最大观测值	是	软件观测最大值
avgRecoveryTimeUs	从 decoded 序列截取 payload	是	具体码型恢复更复杂
avgTotalTimeUs	单帧主链路起点至恢复完成	是	受平台与调度影响
maxTotalTimeUs	单帧 total 最大观测值	是	不等于硬件确定上界

Common-01 文档把理想算法计时定义为不包含帧池和噪声文件读取；Common-04 identity pipeline 的函数边界实际上包含这些操作。正文按已执行源码边界解释现有基础字段，并把“排除 I/O 的纯算法计时”作为任务 runner 需要单独实现的扩展口径。P50/P95/P99、first-output、decision delay、structural delay、interleave time 也不是基础 SummaryRow 字段，只有任务实际输出时才报告。

3.13 Checkpoint 与恢复

Common-04 checkpoint JSON 保存 schema、实验和配置身份、frame/noise pool ID、payload/encoded 长度、SNR 索引和 Eb/N0、下一帧索引、已处理帧数、payload 总比特、错误与成功计数、各阶段累计时间、最大时间、扩展译码状态、噪声策略、全局种子、分片信息、checkpoint 次数、时间戳和停止原因。

表 3.4: Checkpoint 恢复前的身份与一致性检查

检查类别	字段	不一致时的处理
Schema 与实验 配置与输入	schemaVersion、experimentId	拒绝恢复
	configHash、framePoolId、noisePoolId	拒绝混合配置或池身份
长度	payloadLength、encodedLength	拒绝长度不同的累计状态
SNR 随机策略	snrIndex、ebN0_dB	拒绝跨点恢复
	noisePolicyVersion、globalSeed	拒绝随机身份变化
分片 进度	shardIndex、shardCount	拒绝分片身份变化
	nextFrameIndex、processedFrames、totalPayloadBits	必须与请求帧范围及长度一致
计数	successfulFrames、frameErrors	两者之和必须等于已处理帧数

记录结构和序列化格式已经包含扩展字段,但 Common-04 identity runner 创建 checkpoint 时未把所有噪声策略和分片字段显式赋为任务配置;这些字段可能保持默认值。报告因此只能说公共 Schema 支持并校验相应字段,不能据此断言所有历史 checkpoint 都已填充真实任务值。具体任务 runner 应在恢复测试中验证实际写入内容。

写文件时先生成同目录临时文件,刷新成功后再替换目标。Windows 实现使用带替换和写穿语义的 MoveFileExW;这一平台细节放在附录??,正文关注恢复身份和计数一致性。

3.14 分片与结果合并

`mergeSimulationShards` 先按 `frameStart` 排序，再检查 `shardIndex` 不重复、每片帧数非零、帧范围连续且没有重叠或空洞。各分片的 `experimentId`、`configHash`、`frame/noise pool ID`、`payload/encoded` 长度、SNR 索引和 `Eb/N0` 必须一致；分片统计还要满足 `processedFrames` 等于 `frameCount`、`totalPayloadBits` 等于 `frameCount` 与 `payloadLength` 的乘积，并且成功帧与错误帧之和等于分片帧数。

合并时原始计数和时间累计值求和，最大译码时间与最大总时间取各片最大值。BER、FER、成功率和平均时间应在合并后用总计数重新计算，不能直接平均各分片比例。当前基础合并结构未统一覆盖误纠、声明成功、迭代次数或复杂度扩展，具体任务若分片运行，需要为这些字段实现同样的计数一致性和合并规则。

3.15 公共结果 Schema

Common-04 基础汇总使用 `common04.result_summary.v1`。表 ??按用途归类，而不是按 CSV 列顺序逐字复制。

表 3.5: Common 基础结果 Schema 的字段类别

类别	基础字段	作用
身份	<code>schemaVersion</code> 、 <code>experimentId</code> 、 <code>stage</code> 、 <code>codeType</code> 、 <code>case-Name</code>	识别结果版本和实验用例
长度/SNR	<code>payloadLength</code> 、 <code>encodedLength</code> 、 <code>codeRate</code> 、 <code>ebN0_dB</code> 、 <code>snrIndex</code>	绑定长度、内部 SNR 和点序号
可靠性	处理帧、payload 总比特、 <code>bit/frame errors</code> 、成功帧、BER、FER、 <code>successRate</code>	保存原始计数及 payload 派生比例
时延	<code>encode/channel/decode/recovery/total</code> 平均时间及 <code>decode/total</code> 最大时间	描述基础阶段平均与最大观测时间
审计	<code>stopReason</code> 、 <code>framePoolId</code> 、 <code>noisePoolId</code> 、 <code>configHash</code>	绑定停止、输入池和配置

基础 Schema 尚未统一包含 `transmittedLength`、`mis correctionRate`、`decoderDeclaredSuccessRate`、置信区间、零错误标志、平均迭代次数、复

杂度或交织参数。它们由 BCH、CC、LDPC 或专题任务扩展；扩展结果必须保留基础身份和原始计数，避免只输出无法复核的比例。

3.16 零错误点与统计不确定性

Common-01 定义和 JSON 配置已经冻结 Wilson 95% 区间、绘图专用 BER/FER 及零错误标志。Common-04 的 C++ 基础汇总结构和指标实现尚未输出这些字段。因此，现阶段把它们作为报告绘图规则和任务扩展要求，不写成公共 runner 已经生成的字段。

原始 CSV 在未观测到错误时保留真实 0。对数坐标图可以使用空点、向下箭头或单独上界标记，不得把 0 静默替换为固定 10^{-5} 、 10^{-6} ，也不得绘制人为 error floor。若采用零观测事件的三倍法则近似 95% 上界，则

$$p_{b,\text{upper}} \approx \frac{3}{N_{b,\text{payload}}}, \quad (3.14)$$

$$p_{f,\text{upper}} \approx \frac{3}{N_{f,\text{processed}}}. \quad (3.15)$$

式 (??) 和式 (??) 给出的是统计上界，而不是实测 BER/FER。零观测错误只表示有限样本内未出现事件，不能解释为真实误码概率等于零。

3.17 任务级差异和适用边界

后续章节引用本章定义时，应同时说明所用 runner、实际码率分母、SNR 输入类型、停止配置和扩展字段。尤其是历史任务不能因为字段名含 `ebn0` 或 `snr` 就自动认定与 Common 当前接口相同。

在目标错误率 p_0 上比较候选与参考曲线时，正式 E_s/N_0 编码增益定义为

$$G_{\text{coding}}(p_0) = \left. \frac{E_s}{N_0} \right|_{\text{reference}, p_0} - \left. \frac{E_s}{N_0} \right|_{\text{candidate}, p_0}. \quad (3.16)$$

式 (??) 要求两条曲线都覆盖 p_0 ；只能在局部数据范围内插值，不进行远距离外推。若任一曲线未覆盖目标点，编码增益记为 N/A。

表 3.6: Common 公共定义与任务级差异

项目	Common 公共定义	任务级差异
payload	支持 200/300 bit 帧池	按业务选择长度
BPSK	0 到 +1, 1 到 -1	通常复用；其他调制另行定义
内部 SNR	AWGN 接口输入 E_b/N_0	历史或特殊信道可能采用其他归一化
正式横轴	按实际码率换算 E_s/N_0	非 SNR 图不适用；未确认码率不转换
停止	正式模板 5000/200/50000	历史实验和专项试验可能不同
译码输入	硬比特、接收符号或 LLR 接口	量化、裁剪、饱和和依码型设置
信道	基础 AWGN 和接口	多径、频偏、遮挡、突发错误扩展
时延	基础平均和最大观测字段	P95、首输出、结构和交织时延扩展
结果字段	基础身份、计数、比例、时间和 hash	迭代、误纠、复杂度、交织参数扩展

3.18 本章小结

Common 固定了 payload/encoded 码率、实值 BPSK、 E_b/N_0 驱动的 AWGN、硬判决、LLR、payload BER/FER、确定性随机样本、正式停止控制以及基础 checkpoint、分片和结果身份。正式图按每个点的实际码率把内部 E_b/N_0 转换为 E_s/N_0 ，原始统计保持不变。任务仍需补充发送长度、附加位、扩展译码状态、量化、专用时延、复杂度和特殊信道；历史结果是否满足公共定义将在第 4 章以后逐章审计。

第四章 BCH 编码方案与仿真分析

4.1 BCH 低速电文编码设计目标

本章针对长度为 200 bit 和 300 bit 的低速二进制电文，研究 Bose–Chaudhuri–Hocquenghem (BCH) 码的编码组织、软件实现和信道适应性。此类电文的数据量较小，但通常要求整帧正确交付；只要任一信息比特恢复错误，该帧就不能作为有效业务电文使用。因此，方案评价以帧错误率 (Frame Error Rate, FER) 为主，以比特错误率 (Bit Error Rate, BER)、发送长度、实际码率和软件译码时间为辅助指标。

设计工作围绕两条路线展开。第一条路线把原始电文划分为若干 11 bit 信息组，各组独立采用 BCH(15,11) 编码，主要特点是结构简单、单组译码开销小。第二条路线把一帧信息装入较长的 BCH 母码，通过缩短获得与业务长度相适应的码字，利用较大的整块纠错能力提高可靠性。两条路线采用相同的二进制相移键控 (Binary Phase Shift Keying, BPSK) 调制和硬判决条件，使比较重点落在编码组织与纠错能力本身。

除加性高斯白噪声 (Additive White Gaussian Noise, AWGN) 基准外，本章还考察固定多径、频偏造成的帧内相位累积、连续遮挡以及连续突发翻转。研究目标不是给出脱离使用条件的单一“最优码”，而是回答三个工程问题：在相同信息长度下，冗余和纠错能力如何影响可靠性；简单查表译码与长码代数译码需要付出多大软件代价；当错误从分散形态转为局部集中形态时，单纯增强 BCH 码是否仍然有效。

4.2 BCH 编码组织方案

4.2.1 分组 BCH 方案

分组方案以 11 bit 为一个信息组，逐组采用可纠正单个错误的 BCH(15,11) 系统码。若最后一组不足 11 bit，则在信息尾部加入已知零作为填充位；接收端译码并拼接各组信息后，再删除填充位，得到原始电文。200 bit 电文被划分为 19 组，加入 9 个填充位，发送长度为 285 bit；300 bit 电文被划分为 28 组，加入 8 个填充位，发送长度为 420 bit。

对每个信息组，系统编码可写为

$$c(x) = x^{n-k}m(x) + r(x), \quad (4.1)$$

其中， $m(x)$ 为信息多项式， $r(x)$ 为 $x^{n-k}m(x)$ 除以生成多项式所得的余式。信息位在码字中保持可见，便于检查编码与译码接口。由于每个子码字只能保证纠正 1 bit 错误，整帧包含的子块越多，任一子块出现多错并导致整帧错误的机会也越大；另一方面，错误被限制在局部子块内，译码器只需处理 15 bit 短码字，软件实现直观且耗时较低。

4.2.2 整块缩短 BCH 方案

整块方案首先选择信息容量不小于原始电文长度的 BCH 母码，再在母码信息区前端补入已知零。完成母码系统编码后，发送端删除这些已知零对应的位置；接收端在相同位置补回零，再按完整母码进行译码。该过程称为缩短。缩短不改变母码的设计纠错能力 t ，但减少了实际发送位数，实际码率应按

$$R = \frac{K_{\text{info}}}{N_{\text{tx}}} \quad (4.2)$$

计算，其中 K_{info} 为原始信息比特数， N_{tx} 为实际发送码长。

对于 200 bit 电文，候选母码包括 BCH(255,207)、BCH(511,421) 和 BCH(511,385)，相应保证纠错能力分别为 6、10 和 14 bit。对于 300 bit 电文，BCH(511,421) 和 BCH(511,385) 均可采用单个缩短码字；BCH(255,207)

的信息容量不足以容纳整帧，因而把电文平均分为两个 150 bit 子块并分别编码。双块方式仍属于长码代数译码，但每个子块独立承担错误，帧错误率由两个子块的联合恢复结果决定。

图??用于后续补充两类编码组织的 Visio 结构图。图中应明确标出信息分组、填充位、缩短零前缀、校验位以及最终发送序列，便于从接口层面区分“末组补齐”和“母码缩短”这两种码长适配方法。



图 4.1: BCH 分组编码与整块缩短编码结构

4.2.3 两类方案结构与参数比较

表??给出八种编码方案的业务参数。表中不使用程序内部索引，编码长度即进入 BPSK 调制器的比特数。“填充”发生在分组方案的信息尾部，“缩短”则是在长母码信息区加入并删除已知零，两者不能混为同一操作。

200 bit BCH(255,207) 整块方案的发送长度最短、码率最高,但纠错能力低于两个 511 bit 母码方案; BCH(511,385) 整块方案以较多校验位换取 $t = 14$ 。300 bit BCH(511,421) 整块方案只发送 390 bit，比双块 BCH(255,207) 方案还少 6 bit，同时把纠错范围扩展到整个码字; BCH(511,385) 整块方案进一步把 t 提高到 14，但发送长度增至 426 bit。由此可见，码长、码率和纠错能力共同变化，不能只根据母码长度判断性能。

表 4.1: 两类 BCH 编码方案的结构与参数

信息长度	编码组织	母码	块数	填充/缩短	发送长度	码率	纠错能力	译码方法
200	200 bit 分组方案	(15,11)	19	填充 9	285	0.7018	每块 1	综合查表
200	200 bit 整块方案	(255,207)	1	缩短 7	248	0.8065	6	代数译码
200	200 bit 整块方案	(511,421)	1	缩短 221	290	0.6897	10	代数译码
200	200 bit 整块方案	(511,385)	1	缩短 185	326	0.6135	14	代数译码
300	300 bit 分组方案	(15,11)	28	填充 8	420	0.7143	每块 1	综合查表
300	300 bit 双块方案	(255,207)	2	每块缩短 57	396	0.7576	每块 6	代数译码
300	300 bit 整块方案	(511,421)	1	缩短 121	390	0.7692	10	代数译码
300	300 bit 整块方案	(511,385)	1	缩短 85	426	0.7042	14	代数译码

4.3 BCH 编译码实现

4.3.1 系统编码与码长适配

编码器首先检查输入长度及二进制取值。分组方案补齐末组后逐块计算校验位；整块方案构造“已知零前缀 + 原始信息比特”的母码信息区，完成系统编码后删除零前缀。接收端执行相反的长度适配，只把去除填充位或缩短前缀后的原始信息区交给统计模块。BCH(255,207) 在 $GF(2^8)$ 上运算，两个 511 bit 母码在 $GF(2^9)$ 上运算，有限域表示、生成多项式和码字位序在编码器与译码器之间保持一致。

4.3.2 分组 BCH 查表译码

对 BCH(15,11)，接收子码字先计算综合。综合为零时直接输出信息位；综合非零时，以综合值查询预先建立的单错位置表，翻转对应比特并再次校验。由于 $t = 1$ 且码长很短，查表法避免了迭代求错误定位多项式，核心操作只是有限域计算、数组访问和一次候选翻转，适合低复杂度软件或逻辑电路实现。

若一个 15 bit 子块内出现两个及以上错误，译码结果可能有两种表现：译码器找不到满足条件的错误位置并声明失败，或者接收向量落入另一合法码字的判决区域，译码器虽然声明成功，却恢复出错误的信息。这也是

为什么整帧统计不能只读取译码器状态。

4.3.3 整块 BCH 代数译码

整块译码先补回缩短位置，再计算 $2t$ 个综合；随后采用 Berlekamp–Massey (BM) 算法求取错误定位多项式，并用 Chien 搜索遍历母码位置寻找根。仅当根数与定位多项式次数一致、根不落在已删除的缩短前缀中、翻转候选错误后综合重新归零时，译码器才声明成功。最后去除校验区与缩短前缀，恢复原始电文。

代数译码的主要开销来自综合计算、BM 迭代和对 255 或 511 个母码位置进行 Chien 搜索。随着 t 和母码长度增加，有限域乘法次数明显高于 15 bit 子块的查表路径；其收益是能够在整个长码字范围内联合处理更多分散错误。图??预留给两类译码流程图，后续 Visio 绘制应呈现长度适配、综合计算、错误定位、后验校验和信息提取的差异。

待插入：Visio 绘制的分组 BCH 与整块
BCH 译码流程图

图 4.2: 分组 BCH 与整块 BCH 译码流程

4.3.4 译码结果判定

BCH 码的保证纠错能力满足

$$t = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor, \quad (4.3)$$

但该式只给出不超过 t 个错误时的保证，并不承诺错误数超过 t 后一定能被检测出来。为避免把误纠计为成功，仿真采用两级判定：第一级记录译码器是否声明完成纠错，第二级将恢复后的原始信息比特与发送电文逐位比较。只有两者一致，才记为实际正确恢复；声明完成但信息仍错误的帧记为误纠，不能形成有效输出的帧记为译码失败。FER 最终按完整电文是否正确计算，因此分组方案中任一子块造成的信息错误都会使整帧计错。

4.4 AWGN 条件下的 BCH 方案选择

4.4.1 仿真条件

AWGN 比较采用统一 BPSK 调制、实值高斯噪声和硬判决译码。为了在瀑布区获得足够错误样本，同时控制高信噪比点的运行时间，各信噪比点设置最少帧数、目标错误帧数和最大帧数三重停止条件。表??列出正文需要的仿真条件，随机种子和原始数据列名等复现信息集中列于附录??。

表 4.2: AWGN 条件下的 BCH 方案比较设置

项目	设置
输入电文	200 bit、300 bit 独立等概率二进制序列
候选编码	表??所列 8 种分组、单块缩短和双块缩短方案
调制与信道	BPSK；实值 AWGN；硬判决
横轴范围与步长	$E_s/N_0 = -3.0103 \sim 14.9897$ dB，步长 0.5 dB
停止条件	每点至少 1000 帧，累计 200 个错误帧后可停止，每点最多 50000 帧
结果判定	恢复的原始信息比特与发送电文逐位比较
主要指标	FER；同时统计 BER 和软件译码时间

对幅度为 ± 1 的实值 BPSK，噪声方差定义对应的横轴统一换算为符号能量与噪声功率谱密度之比 E_s/N_0 。高信噪比点若在 50000 帧内未观察到错误，图中保留其“零观测”事实，但不把它解释为理论错误率为零，也

不人为绘制错误平台。

4.4.2 200 bit 方案

图??显示, 200 bit BCH(511,385) 整块方案在所考察的 AWGN 范围内可靠性最好, 其后依次为 BCH(511,421) 整块方案、BCH(255,207) 整块方案和 BCH(15,11) 分组方案。为给工程选型提供易比较的量化位置, 在相邻两个非零统计点之间对 $\log_{10}(\text{FER})$ 作线性插值。FER 为 10^{-2} 时, 四种方案所需 E_s/N_0 约为 2.95、3.52、4.39 和 5.98 dB。

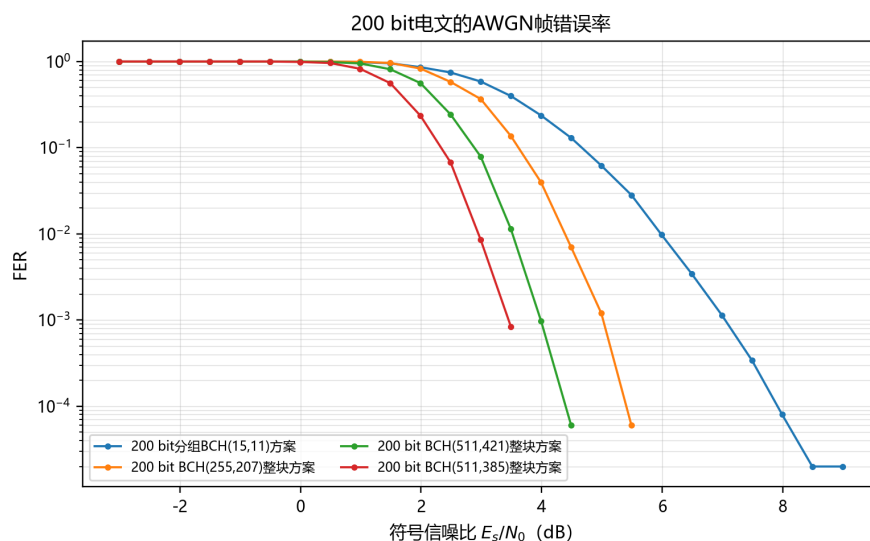


图 4.3: 200 bit 电文的 AWGN 帧错误率

上述插值只用于描述相邻采样点之间的曲线位置, 不等同于理论门限, 也没有扩大原始样本量。BCH(511,385) 整块方案通过 326 bit 发送长度和更高代数译码开销取得最靠左的曲线; BCH(255,207) 整块方案只发送 248 bit, 但 $t = 6$ 限制了低错误率区域的收益; 分组方案虽然单块译码简单, 却要连续正确恢复 19 个子块, 因此整帧 FER 较高。实际选择需要同时考虑链路余量、占用时长和处理资源。

4.4.3 300 bit 方案

图??给出 300 bit 电文结果。FER 为 10^{-2} 时, BCH(511,385) 整块方案、BCH(511,421) 整块方案、双块 BCH(255,207) 方案和 BCH(15,11) 分组方案的插值位置约为 3.46、4.03、4.27 和 6.18 dB。390 bit BCH(511,421) 整块方案比 396 bit 双块 BCH(255,207) 方案略靠左, 说明在当前分散噪声条件下, 把 $t = 10$ 作用于整个码字获得的联合纠错收益, 抵消了二者很小的码率差异。

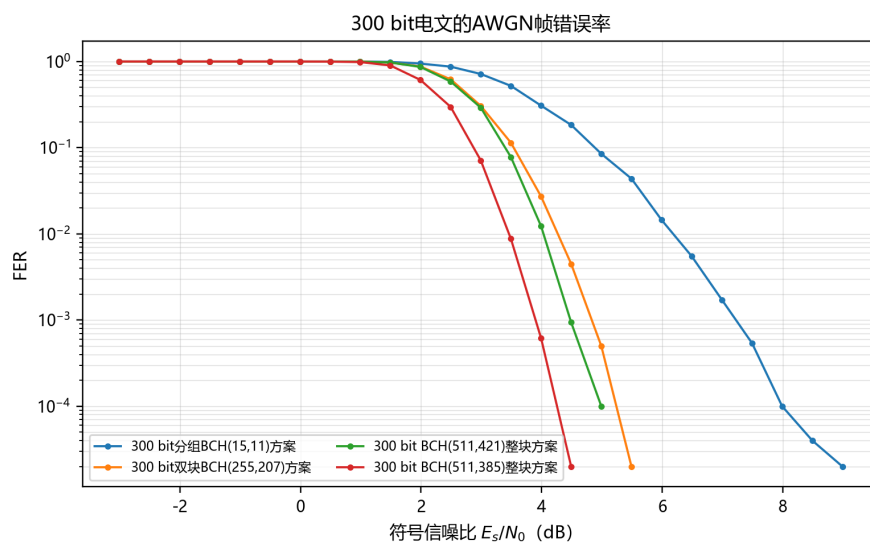


图 4.4: 300 bit 电文的 AWGN 帧错误率

426 bit BCH(511,385) 整块方案相对 390 bit 方案把 FER 为 10^{-2} 的位置改善约 0.57 dB, 但多发送 36 bit, 并增加代数译码运算。双块方案的两个子码字各自具有 $t = 6$, 如果错误恰好分散到两块, 它可能获得局部并行纠错的机会; 然而整帧需要两个子块都正确, 且当前软件按顺序完成两次代数译码, 所以不能仅按“每块 $t = 6$ ”与单块 $t = 10$ 直接换算优劣。

4.4.4 可靠性、码长与软件译码时间权衡

图??给出同一批 AWGN 仿真中的每帧平均软件译码时间。按全部信噪比点累计译码时间除以累计帧数，200 bit 四类方案从分组 BCH(15,11)、BCH(255,207)、BCH(511,421) 到 BCH(511,385) 约为 12.73、20.54、43.66 和 63.67 $\mu\text{s}/\text{帧}$ ；300 bit 对应约为 18.52、28.60、39.00 和 50.96 $\mu\text{s}/\text{帧}$ 。分组方案虽然需要 19 或 28 次子块译码，但每次只做短码综合与查表；长码方案则承担 BM 迭代和完整母码范围的 Chien 搜索。

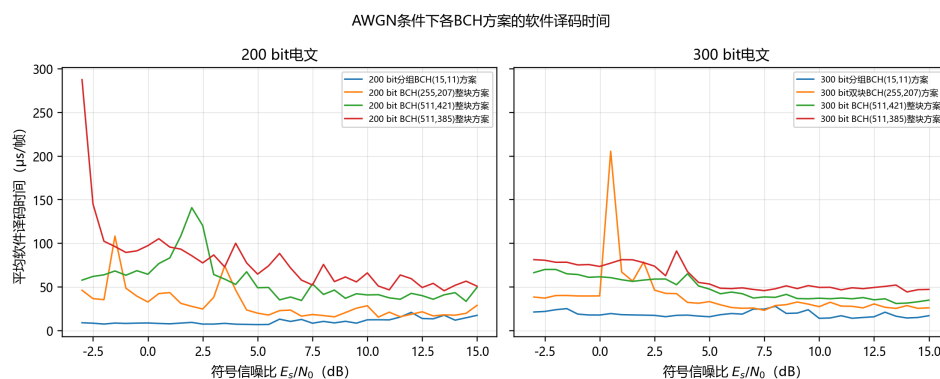


图 4.5: AWGN 条件下各 BCH 方案的软件译码时间

这些时间反映当前处理器、编译器和串行软件实现，不代表现场可编程门阵列或专用集成电路的流水线时延，也不应被视为与平台无关的算法常数。综合可靠性和代价，200 bit 场景若强调低复杂度，可采用分组 BCH(15,11)；若可靠性优先，则 BCH(511,385) 整块方案更有优势。300 bit 场景中，390 bit BCH(511,421) 整块方案在发送长度、AWGN 可靠性和软件时间之间形成较均衡的基线，426 bit BCH(511,385) 整块方案适用于愿意增加冗余以换取更低 FER 的场合。

4.5 不同信道条件下的 BCH 适应性

4.5.1 多信道仿真设置

多信道试验沿用表??中的八种编码方案，但根据失真机理分别设置横轴和接收处理，不能把不同图中的数值拼成统一排名。表??概括各场景。图??预留为 Visio 链路图，后续应画出编码、BPSK 调制、四类信道支路、相应接收处理、硬判决、BCH 译码和整帧统计之间的关系。

表 4.3: BCH 多信道适应性仿真设置

信道场景	信道模型	关键参数	接收处理	主指标
固定多径	四抽头实数信道叠加 AWGN	归一化多径抽头: 0.80452、0.52294、0、0.28158	已知信道, 线性 MMSE 均衡	FER
帧内相位累积	复基带符号相位线性增长并叠加 AWGN	帧首 0°、帧末累计 30°	不作频偏补偿, 取实部硬判决	FER
连续遮挡	连续调制符号幅度置零并叠加 AWGN	随机起点, 约占码长 10%	不提供受损位置, 不交织	FER
连续突发错误	连续硬比特人为翻转	随机起点, 0~50 bit, 不环回	无 AWGN, 不交织	FER



图 4.6: BCH 多信道适应性仿真链路

4.5.2 固定多径条件

固定多径信道表示为 $y_i = \sum_{\ell} h_{\ell} x_{i-\ell} + n_i$ 。四个多径抽头经过能量归一化，延迟副本之间的叠加造成码间干扰。接收端已知信道系数，采用线性

最小均方误差（Minimum Mean Square Error, MMSE）均衡恢复符号，再作硬判决和 BCH 译码。

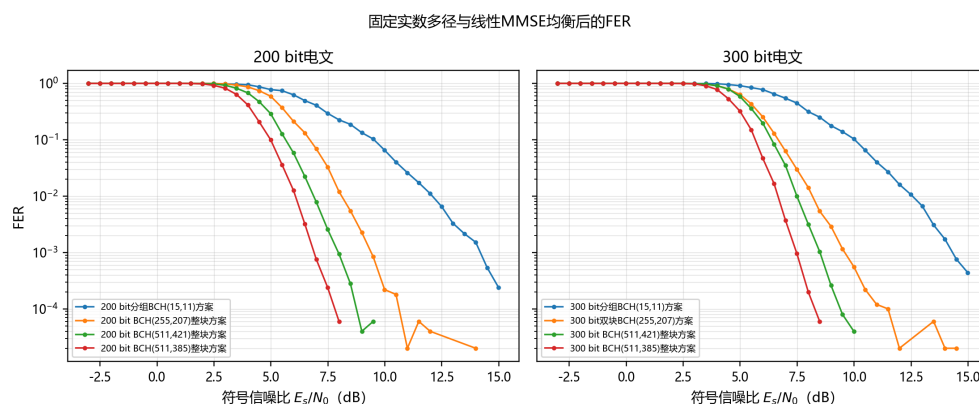


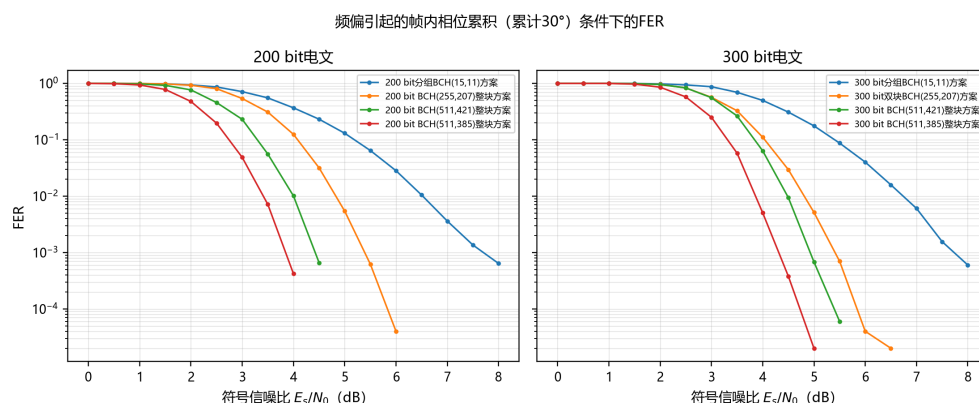
图 4.7: 固定实数多径与 MMSE 均衡后的帧错误率

图??中的曲线随信噪比提高而继续下降，说明已知信道条件下的 MMSE 均衡能够抑制主要码间干扰，没有形成明显不可消除的平台。与 AWGN 基准相比，下降区间整体后移，其原因是有限长度块均衡仍保留边界失真，同时 MMSE 均衡会带来残余干扰和噪声增强。各方案的相对趋势总体保持：较大的整块纠错能力更能吸收均衡后分散出现的硬判决错误。不过，该结论建立在固定实数多径抽头和完全已知信道上，不能直接外推到信道估计误差或快速时变环境。

4.5.3 频偏引起的帧内相位累积（累计 30° ）

频偏会使载波相位随符号时刻持续旋转。本试验不施加固定的初始相差，而是令每个编码帧从 0° 开始，相位按符号序号线性增加，并在帧末累计到 30° ；因此，“累计 30° ”描述的是一个编码帧内由频偏引起的相位增长强度。不同码长方案均在各自帧首和帧末之间完成相同的总相位累积，接收端关闭频偏补偿，只取复基带样值实部进行硬判决。

对理想 BPSK 符号 ± 1 ，旋转后的实部分量为 $\pm \cos \theta$ 。在帧末 $\theta = 30^\circ$ 时， $\cos 30^\circ \approx 0.866$ ，判决幅度和噪声裕量有所减小，但符号能量并未像遮

图 4.8: 频偏引起的帧内相位累积（累计 30° ）条件下的帧错误率

挡那样被直接删除。因此，随着信噪比提高，各方案的 FER 仍能继续下降，没有出现与 10% 连续置零相同的平台。较强整块码对相位累积造成的分散判决错误仍表现出更好的吸收能力，分组方案则更容易因某个短子块发生多错而产生整帧错误。这里评价的是编码在未补偿相位累积下的承受能力，并非载波同步算法本身的性能。

4.5.4 连续遮挡条件

遮挡试验从随机位置选择约占编码帧 10% 的连续调制符号，将其幅度直接置零，遮挡区仍叠加噪声。接收端既不知道受损位置，也没有擦除标记或交织处理。因此，该模型不是擦除码信道，而是“局部信号缺失后仍按普通硬判决输入 BCH 译码器”的压力试验。

图??表现出显著错误平台。提高 AWGN 信噪比只能减小未遮挡区域的噪声，却无法重建被置零的连续符号；当局部损伤产生的错误数超过 BCH 码纠错能力时，继续增加信噪比也难以改善整帧结果。BCH(511,385) 整块方案凭借 $t = 14$ 取得相对较低的 FER，但仍不能消除平台。该现象与累计 30° 相位旋转形成清楚对比：相位旋转主要压缩实部判决裕量，连续遮挡则直接丢失一段符号信息。对于强遮挡业务，必须考虑交织、受损位置辅助或更有针对性的接收结构。

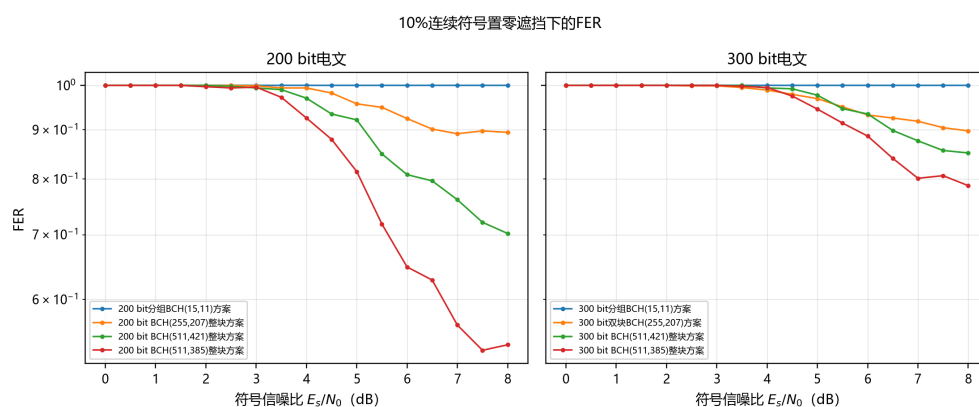


图 4.9: 约 10% 连续符号置零遮挡下的帧错误率

4.5.5 连续突发错误条件

连续突发错误试验不叠加 AWGN，而是在硬判决比特序列中从随机起点连续翻转 0–50 bit，不允许越过帧尾环回，也不采用交织。它是为了隔离“错误集中程度”而设置的人工模型，不能等同于自然无线信道中的独立随机误码。

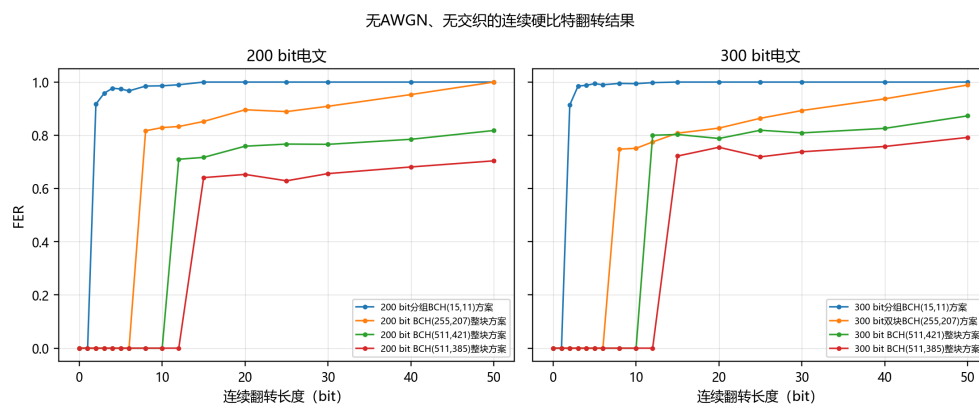


图 4.10: 无 AWGN、无交织条件下的连续突发错误帧错误率

当单个码字内的翻转数不超过 t 时，BCH 译码具有确定的纠错保证；超过 t 后，是否恢复成功还取决于突发起点、码字边界以及错误是否跨入

相邻子块。分组方案的每个 15 bit 子块只能保证纠正 1 bit，连续突发很容易在某一子块内超过能力。双块 BCH(255,207) 方案则应按每个子块分别判断：跨越两块边界的突发可能把错误分摊，但整帧仍要求两块都正确，不能把总突发长度简单等同于某一个 t 值。图中曲线因此是特定边界分布下的统计结果，而不是“突发长度等于纠错能力”的普遍规则。

4.5.6 不同错误形态对 BCH 性能的影响

从错误空间分布看，本章信道可归为两类。第一类以分散错误为主，包括 AWGN 硬判决错误、固定多径均衡后的残余错误以及累计 30° 相位旋转下的判决错误。这些错误可以分布在较长码字范围内，提高 t 通常能够使整块 BCH 方案吸收更多错误，FER 曲线也会随信噪比持续下降。多径中的残余码间干扰可能带有相关性，相位累积又使帧后部的判决裕量更小，但二者在当前条件下仍未表现为连续大段信息完全丢失。

第二类是局部集中的连续损伤，包括调制符号置零遮挡和人为连续翻转。它们可能一次向同一子码字注入超过 t 的错误，因而仅靠增加信噪比或适度增强 BCH 码不足以保证恢复。改善方向包括在编码前交织以重新分散错误、向译码器提供受损位置、利用导频或幅度检测识别遮挡区，以及设计面向连续损伤的接收处理。相关结构将在第??章结合交织进一步讨论。本章不建立跨不同横轴和不同损伤机理的统一排名，只在相同信道、相同判据下比较方案。

4.6 BCH 方案阶段性结论

表??把仿真结果转化为可执行的工程建议。推荐项均以本章已有数据为依据，不额外设定未经验证的工作门限。

总体而言，BCH(15,11) 分组方案适合强调实现简单和处理开销的低速链路；较长的缩短 BCH 码通过联合纠正更大范围内的错误，能够显著改善 AWGN 和其他分散错误条件下的整帧可靠性。对于 300 bit 电文，

表 4.4: BCH 方案的阶段性工程建议

业务条件	推荐方案	推荐理由	主要代价
200 bit, 低复杂度优先	分组 BCH(15,11)	短码查表路径简单, 软件译码时间最低	发送 285 bit; AWGN 可靠性较弱
200 bit, 可靠性优先	BCH(511,385) 整块方案	AWGN 曲线最靠左, 集中损伤下相对表现最好	发送 326 bit; 软件译码时间最高
300 bit, 综合平衡	BCH(511,421) 整块方案	390 bit 码长下兼顾 $t = 10$ 、FER 与软件时间	复杂度高于分组方案
300 bit, 可靠性优先	BCH(511,385) 整块方案	AWGN 下 FER 最低, 具有 $t = 14$ 纠错能力	比 390 bit 方案多发送 36 bit
强连续损伤	不依赖单一 BCH 方案	遮挡平台和突发结果表明局部错误可一次超过 t	需增加交织、位置辅助或结构化接收处理

BCH(511,421) 整块方案是较均衡的基准；对于可靠性优先的 200 bit 或 300 bit 业务，BCH(511,385) 整块方案更具优势。若链路存在强连续遮挡或长突发错误，则不应把提高 BCH 纠错能力作为唯一措施，而应把编码与交织、同步、受损位置检测和接收机处理联合设计。

需要强调的是，上述建议建立在系统编码、BPSK 硬判决以及本章给定停止条件之上。不同方案的实际码率并不相同，曲线差异同时包含冗余长度、纠错能力、单块或多块组织方式等因素，因此不能把任意两条曲线间距直接解释为与实现无关的固定编码增益。软件译码时间同样只用于当前实现的相对比较；若后续改用并行逻辑、流水线 Chien 搜索或专用有限域运算单元，复杂度排序可能保持，但绝对处理时间需要重新测量。

从系统应用角度看，BCH 码适合作为低速控制电文的基础纠错手段：在分散噪声占主导时，可根据允许的发送长度选择 $t = 10$ 或 $t = 14$ 的整块方案；在处理资源受限时，可退回结构清晰的分组方案。对连续损伤占主导的链路，后续设计应优先改变错误进入译码器之前的空间形态，再讨论是否继续增加 BCH 冗余。这样能够避免用过长码字换取有限改善，也使编码选择与实际信道机理保持一致。

第五章 卷积码方案与仿真分析

5.1 高速电文卷积码设计目标

本章面向长度固定为 300 bit 的高速二进制电文。与上一章讨论的低速电文相比，高速业务不仅要求降低整帧错误率，还要求编码器能够随数据到达连续工作，译码器能够在整帧结束前逐批给出稳定结果，并在发送效率、等待时间和存储资源之间作出可解释的取舍。若只追求最低帧错误率，可以增加冗余并等待完整码字；若只追求短发送时长，又会压缩纠错余量。因而本章不把某一个码率或某一组译码参数预设答案为唯一答案，而是先建立统一编码结构，再由正式仿真分别筛选可靠性、吞吐、时延、存储和综合平衡方案。

卷积码适合承担这一工程基线。编码输出由当前输入和有限长度历史共同决定，移位寄存器状态可以在数据连续到达时自然延续，不必把每个时隙重新初始化为独立码块。以码率 $1/2$ 母码为基础，还可以通过打孔删除预定位置的编码比特，在不改变主体网格结构和 Viterbi 译码框架的条件下形成约 $2/3$ 和约 $3/4$ 方案。这种做法使可靠性与发送效率的比较具有共同基础，也便于接收端在硬判决和软判决之间切换。

接收侧采用 Viterbi 算法在状态网格中寻找累计路径度量最优的输入序列。整块译码能够利用完整终止边界，适合作为可靠性基准；有限回溯和真滑窗译码则用受控的历史长度、接收窗口和滑动步长换取更早输出与更低存储。对于按时隙到达的高速电文，连续编码与滑窗译码还可把“等待完整 300 bit 后统一处理”改为“边到达、边更新、分批提交”。

因此,本章围绕四个问题展开:第一, $K = 7$ 、八进制生成多项式 171/133 如何适配 300 bit 电文并形成 612、459 和 408 bit 三种发送长度;第二,不同码率以及硬、软判决如何影响 BER 和 FER;第三,回溯深度、软信息量化位宽及滑窗 $W/S/D$ 参数如何改变可靠性与资源;第四,在最终码流不变的情况下,时隙组织为何保持 BER/FER 一致却产生不同输出时延。下文先从编码结构和实际码长开始。

评价这些问题时,FER 用于回答一封 300 bit 电文能否完整交付,BER 用于观察错误比特的总体密度,发送长度反映占用的调制符号数,首次输出和第 95 百分位决策时延则分别描述最早结果与大多数结果的等待。四类指标不能相互替代:较低 BER 并不必然保证整帧正确,较早出现第一个比特也不代表后续信息均已完成判决,而较高码率只有在给定信道余量下仍满足目标 FER 才具有业务价值。本章据此坚持在相同口径内比较,在目标改变时重新说明比较基准,避免用单一曲线或单一资源数值代替系统选型。

5.2 卷积编码结构与码长设计

5.2.1 约束长度与生成多项式

编码器采用约束长度 $K = 7$ 的二进制卷积母码。每个输出不仅由当前输入比特决定,还与前 6 bit 历史有关,故记忆长度为 6,状态数为 $2^6 = 64$ 。每输入 1 bit,编码器沿状态网格前进一步并产生两个母码输出比特。两个输出分别由八进制生成多项式 $G_1 = 171$ 和 $G_2 = 133$ 规定;它们对应二进制多项式

$$g_1(x) = x^6 + x^5 + x^4 + x^3 + 1, \quad g_2(x) = x^6 + x^4 + x^3 + x + 1. \quad (5.1)$$

这里的 171 和 133 是八进制表示,不是普通十进制系数。每个多项式标明当前输入和哪些记忆单元参加模 2 加法,两个异或网络由此产生一对编码比特。 $K = 7$ 使网格规模仍可由 64 状态 Viterbi 稳定处理,同时保留足够的路径记忆,是本项目 300 bit 高速电文的统一母码结构。

图??预留为编码器结构图。后续 Visio 绘制应完整表示当前输入、6 级记忆单元、两组抽头和两个输出，不能只画抽象方框而省略多项式含义。

待插入：Visio 绘制的卷积编码器移位寄存器与生成多项式结构图

输入信息比特 → 当前输入与 6 级记忆单元 → 两个异或网络
→ 两路编码输出

标注： $K = 7$ ， $G_1 = 171$ （八进制）， $G_2 = 133$ （八进制）

图 5.1: 卷积编码器移位寄存器与生成多项式结构

表 5.1: 卷积码基础编码参数

参数	冻结取值
原始信息长度	300 bit
约束长度、记忆长度	$K = 7$ ；记忆长度 6 bit
状态数	64
生成多项式	$G_1 = 171$ 、 $G_2 = 133$ （八进制）
母码率	1/2
尾比特	6 bit 零尾
编码器输入长度	306 个网格步
母码输出长度	612 bit

5.2.2 零尾终止与实际编码长度

整块编码在 300 bit 原始电文后追加 6 bit 零尾，使编码器从任意数据结束状态逐步返回全零状态。编码器实际处理

$$N_{\text{step}} = 300 + 6 = 306$$

(5.2)

个网格步。母码每步输出 2 bit，因此未打孔码长为

$$N_{\text{mother}} = 306 \times 2 = 612 \text{ bit.} \quad (5.3)$$

这 6 个尾步不属于原始业务信息，但为整块 Viterbi 提供已知终止状态。译码器可从零状态反向沿幸存路径回溯，避免把帧末状态当作未知量；恢复后只提交前 300 bit 原始电文。612 bit 远低于 1000 bit 码长上限，也为后续提高码率保留了打孔空间。

零尾是整块组织的终止手段，不等于每个时隙都重新补 6 bit。若把 50、100 或 150 bit 时隙各自独立终止，不仅会反复增加尾比特，还会切断跨时隙状态记忆，形成与本章正式方案不同的多个短卷积码块。连续时隙方案只在整封 300 bit 电文的最后一个时隙追加零尾，具体机制见第??节。

5.2.3 打孔与实际码率

母码输出按“每个网格步先输出 G_1 、再输出 G_2 ”的顺序排列。打孔器周期性保留或删除这些位置：图样 11 保留全部母码比特；图样 1101 每 4 个母码位置保留 3 个；图样 110110 每 6 个位置保留 4 个。接收端解打孔时，被删除的位置以“无观测”标志送入路径度量计算，并不把它们错误补成判决比特 0。因此，打孔改变的是可用观测数量和冗余，而不是强行引入固定比特。

实际帧级码率统一定义为

$$R = \frac{K_{\text{info}}}{N_{\text{tx}}}, \quad (5.4)$$

其中 $K_{\text{info}} = 300$ ， N_{tx} 是实际发送编码长度。表??中的“1/2、约 2/3、约 3/4”是母码或目标打孔方案名称；由于 6 个零尾不计入原始信息，实际帧级码率分别为 0.4902、0.6536 和 0.7353，不能直接写成理想分数。

表 5.2: 不同打孔方案的实际编码长度与码率

方案	打孔图样	发送长度/bit	实际码率	工程意义
1/2 母码	11	612	0.4902	冗余最多,可靠性优先
约 2/3 打孔	1101	459	0.6536	可靠性与发送效率折中
约 3/4 打孔	110110	408	0.7353	发送长度最短,吞吐优先

5.3 Viterbi 译码与连续处理机制

5.3.1 网格状态、路径度量与回溯

编码器的 6 bit 记忆内容构成网格状态。每到来 1 bit 输入，当前状态按寄存器移位规则转移到下一状态，并产生一对候选编码输出；这一过程称为一个网格步。每个状态有两条可能输入分支，连续分支连接成候选路径。接收端根据观测与候选输出的差异计算分支度量，再把它加到前一状态的累计路径度量上。到达同一新状态的多条路径中，仅保留累计度量较小者及其前驱信息，称为幸存路径。

Viterbi 算法并不逐个比特作孤立判决，而是比较完整状态约束下的路径。噪声可能使某一步的局部观测倾向错误分支，但后续观测继续累积后，不符合卷积约束的路径往往付出更大度量而被淘汰。经过足够深度后，从当前优胜状态向前回溯，早期路径通常已经合并，此时才能把对应信息比特作为稳定结果提交。图??将用少量状态示意分支竞争和幸存路径；实际 $K = 7$ 译码器始终包含 64 状态，示意图中的状态数量不代表真实规模。

路径合并也是理解回溯和滑窗参数的关键。若两条候选路径在较早时刻已经进入同一状态，则该状态之前的共同部分不会再被后续输入分开；反之，若多条竞争路径在有限历史内仍保持接近的累计度量，立即输出就可能选错。回溯深度给竞争路径留下继续分化的观察时间，窗口长度提供可

用于比较的接收范围，滑动步长决定稳定结果以多大批次被提交。三者分别作用于路径稳定、观察范围和输出节奏，不能把其中一个参数的增大简单视为对另外两个参数的替代。

待插入：Visio 绘制的网格状态转移与 Viterbi 幸存路径示意图

少量状态仅用于说明分支、累计路径度量、路径竞争与回溯；
实际译码器共有 64 状态

图 5.2: 网格状态转移与 Viterbi 幸存路径示意

5.3.2 硬判决与软判决

硬判决先把 BPSK 接收样值量化为 0 或 1，再统计候选编码输出与接收判决之间不一致的位置数，形成汉明型分支度量。该接口简单，每个观测只需 1 bit，但量化后无法区分“接收点刚刚越过判决门限”和“接收点远离门限”这两种可靠性完全不同的情况。

浮点软判决保留 BPSK 接收样值。对每个候选编码输出，先映射到理想符号 +1 或 -1，再累计接收样值与候选符号之间的平方欧氏距离。当前实现不是直接输入对数似然比 (LLR)，而是用连续接收样值中的幅度距离表达可靠性。对于打孔删除的位置，观测标志为 0，该位置不参与硬判决或软判决的分支度量。软判决因此没有改变卷积码和状态网格，只是用更完整的信道信息提高了路径区分能力。

5.3.3 整块、有限回溯与滑窗译码

整块译码接收完整 306 步网格，保存全帧幸存路径，从已知零终止状态回溯并一次输出 300 bit。它最大限度利用完整帧边界，是比较码率和判

决方式时的可靠性基准，但需要保存约 306 步的幸存信息，且最早结果必须等到帧末。

有限回溯仍持续更新 64 个状态度量，但只保留最近 D 步路径历史；当路径足够稳定时提交较早比特。增大 D 通常有助于降低过早判决风险，却同步增加存储和等待，并可能带来重复回溯操作。真滑窗在此基础上引入接收窗口长度 W 和滑动步长 S ：窗口缓存连续观测，更新路径度量，向前回溯 D 步，每个非最终窗口提交约 S 个稳定信息比特，然后前移 S 继续处理。参数必须满足

$$W > D, \quad S \leq W - D. \tag{5.5}$$

最终窗口利用零终止状态完成剩余输出，并检查 300 bit 是否无遗漏、无重复。这一过程是真正的增量译码，不是先做整帧译码再人为截断结果。图??预留两条流程，后续应明确整块一次输出与滑窗分批提交的差别。

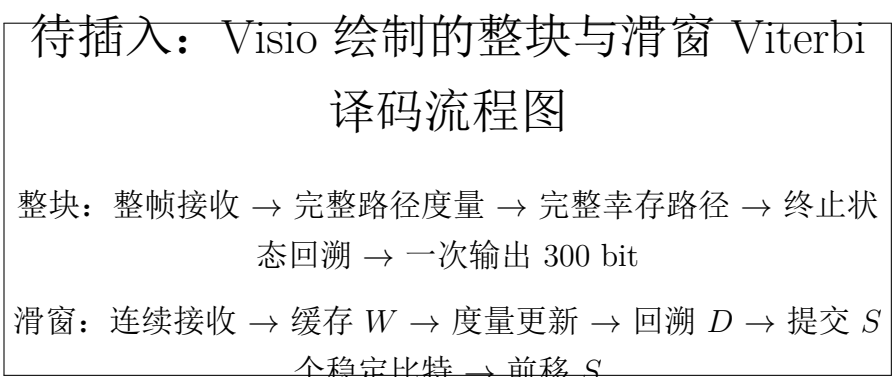


图 5.3: 整块与滑窗 Viterbi 译码流程

5.4 码率与判决方式性能分析

5.4.1 仿真条件

基础性能比较采用 300 bit 独立等概率二进制电文、BPSK 调制和实值 AWGN 信道。主横轴为 E_s/N_0 ，单位 dB；幅度为 ± 1 时，噪声方差为

$$\sigma^2 = \frac{1}{2 \times 10^{(E_s/N_0)/10}}. \tag{5.6}$$

若需要换算为 E_b/N_0 ，应对每个方案分别使用 $E_b/N_0 = E_s/N_0 - 10 \log_{10} R$ ，其中 R 取表??的实际帧级码率，不能对三码率统一平移。

表 5.3: 基础性能仿真参数

项目	设置
原始电文与编码	300 bit; $K = 7$; 171/133 (八进制); 6 bit 零尾
调制与信道	BPSK; 实值 AWGN
主横轴	E_s/N_0
基础粗网格	-5 ~ 10 dB, 步长 0.5 dB
粗网格停止条件	每点至少 1000 帧; 累计 200 个错误帧后可停止; 最多 50000 帧
密集区用途	每点至少 5000 帧, 用于瀑布区及 FER=0.1 工作点插值
码率方案	1/2 母码、约 2/3 打孔、约 3/4 打孔
判决方式	硬判决、浮点软判决
主要指标	BER、FER; 资源与时延另行统计

粗网格用于覆盖完整工作区间，密集区数据用于提高瀑布段工作点插值的分辨率，两者停止条件不能混写。FER=0.1 所需信噪比在相邻非零统计点间对 $\log_{10}(\text{FER})$ 线性插值。若高信噪比点在有限帧数内未观察到错误，只能表述为“零误帧观测”；在对数图中省略该点，不把它解释为理论 FER 等于零。

5.4.2 不同码率可靠性分析

图??在整块浮点软判决条件下比较三码率。提高码率会减少发送冗余和实际码长：约 3/4 方案只发送 408 bit，约 2/3 方案发送 459 bit，1/2 母码发送 612 bit。相应地，在相同 E_s/N_0 下，打孔方案可供路径度量使用的观测更少，瀑布区依次向右移动。

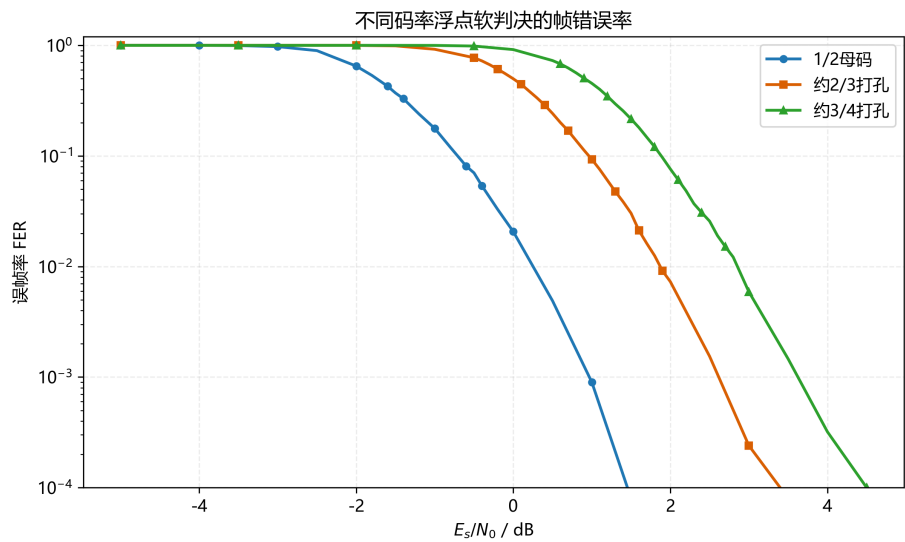


图 5.4: 不同码率浮点软判决的帧错误率

表 5.4: 不同码率的 FER=0.1 工作点

方案	发送长度/bit	所需 E_s/N_0 /dB	工程解释
1/2 母码	612	-0.709	冗余最多，可靠性余量最大
约 2/3 打孔	459	0.963	减少 153 bit，形成折中
约 3/4 打孔	408	1.885	发送最短，以可靠性换取效率

表??量化了这一取舍。相对 1/2 母码，约 2/3 和约 3/4 方案达到

FER=0.1 分别需要更高的 E_s/N_0 ，但不能据此简单称约 3/4 方案“最差”：其代价对应 204 bit 的码长缩短和更高实际码率，适用于链路余量充足、发送占用或吞吐优先的场合。码率选择的实质是在可用信噪比下决定把资源留给冗余还是业务传输。

这里的公平性边界必须明确：比较固定的是每个实际发送 BPSK 符号的 E_s/N_0 。1/2 母码、约 2/3 和约 3/4 方案每帧分别发送 612、459 和 408 个符号，故低码率方案不仅冗余更多，整帧占用的发送符号也更多。本节回答的是固定单符号能量条件下的“发送占用—可靠性”权衡，而不是固定每信息比特能量或固定整帧总能量条件下的纯编码增益。

5.4.3 硬判决与软判决性能分析

图??显示，对三个码率，浮点软判决瀑布区均明显早于硬判决进入下降段。两种译码器采用相同卷积码、打孔图样和状态转移，差别只在分支度量：硬判决丢弃门限两侧的距离，软判决保留接收点相对 ± 1 理想符号的连续幅度信息，因此能更早排除不可信路径。

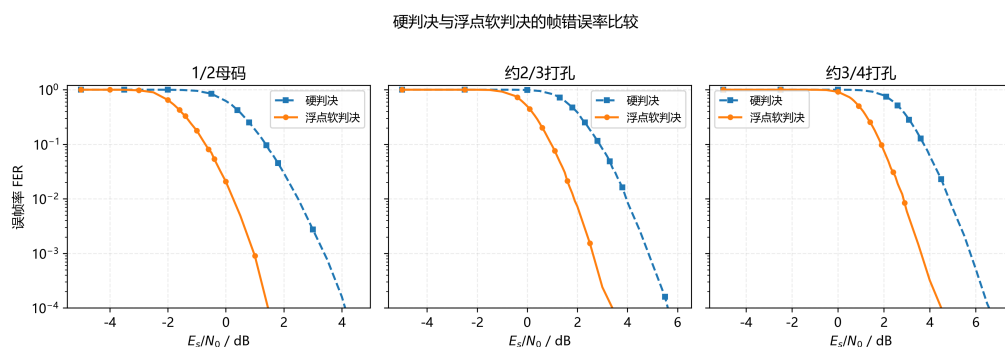


图 5.5: 硬判决与浮点软判决的帧错误率比较

在 FER=0.1 处，软判决相对硬判决的 E_s/N_0 收益为 1.857–2.085 dB。它不需要增加发送冗余，而是以更精细的接收输入、路径度量运算和存储精度换取可靠性。因此，后续回溯、量化、滑窗和时隙研究均以软判决作为主要性能路线，同时保留硬判决作为低接口复杂度基准。

表 5.5: 软判决相对硬判决的 FER=0.1 性能收益

方案	E_s/N_0 收益/dB	解释
1/2 母码	2.085	利用幅度可靠性后所需信噪比显著下降
约 2/3 打孔	1.927	打孔观测减少,但软信息优势仍保持
约 3/4 打孔	1.857	在最高码率下仍获得明显收益

5.5 回溯深度与软信息量化

5.5.1 回溯深度对可靠性的影响

完整回溯保存全帧 306 步幸存路径并利用零尾终止状态；有限回溯正式比较 $D = 35, 49, 70, 84, 98, 112$ 。图??给出三码率的全部正式 FER 点。随着 D 增加，过早提交尚未合并路径的风险下降，各有限深度曲线逐步逼近完整回溯，而不是只有 $D = 112$ 一个孤立结果。

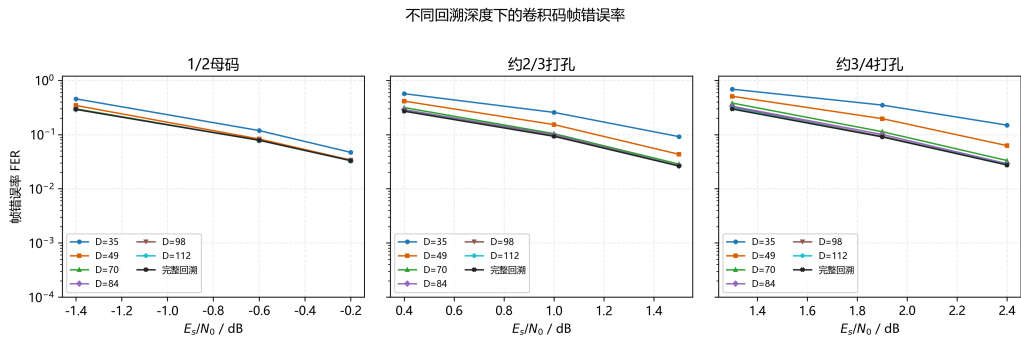


图 5.6: 不同回溯深度下的卷积码帧错误率

5.5.2 回溯深度对时延、存储和软件处理的影响

图??是当前处理器、编译器和串行实现上的软件处理时间，并非硬件固有时延。符号级首次输出时延则由译码结构决定，是更稳定的算法指标。

有限回溯还会重复执行局部回溯，因此深度缩短并不保证本机软件时间同步下降。

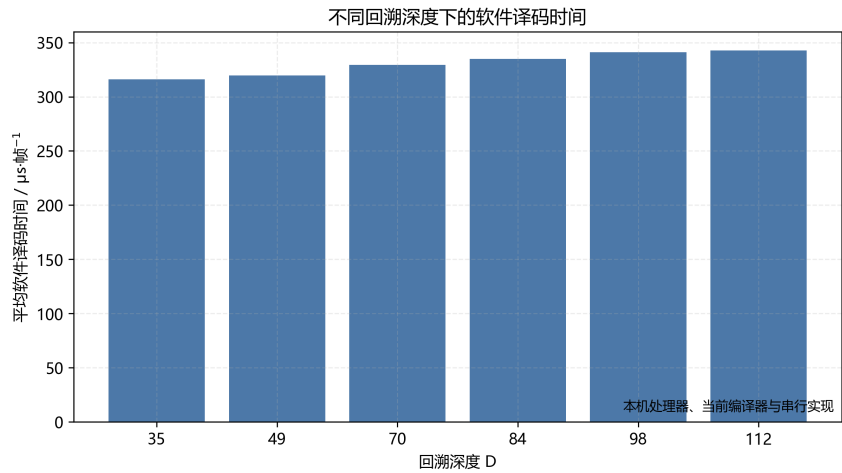


图 5.7: 不同回溯深度下的软件译码时间

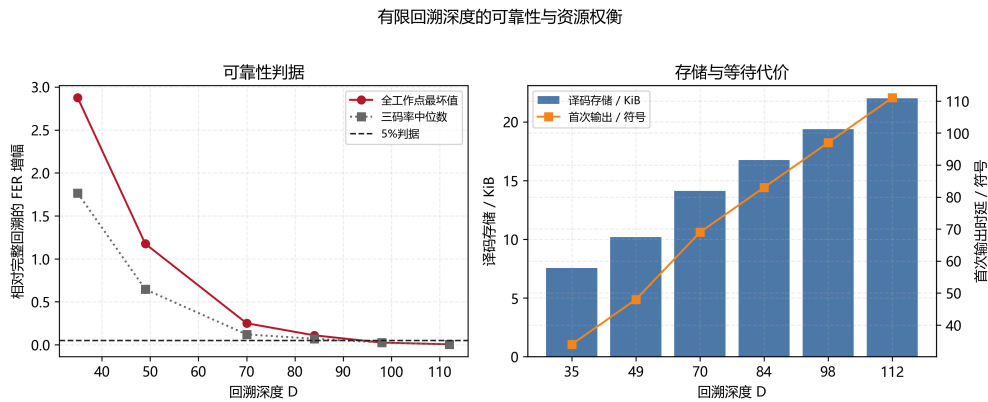


图 5.8: 有限回溯深度的可靠性与资源权衡

$D = 112$ 是当前测试集合中第一个通过全工作点 5% 最坏相对 FER 增幅门限的有限深度； $D = 98$ 仍略高于门限。相对完整回溯， $D = 112$ 将译码存储降低约 62.3%，首次输出提前 194 个符号，故通用选择保持 $D = 112$ 。

表 5.6: 不同回溯深度的可靠性、资源与时延比较

D	最坏 FER 增幅	5% 判据	存储/B	首次输出/符号	软件时间/ μs	评价
35	445%	不通过	7744	34	315.919	可靠性损失很大
49	128%	不通过	10432	48	319.674	与完整回溯差距大
70	29.1%	不通过	14464	69	329.360	可靠性继续改善
84	12.5%	不通过	17152	83	334.853	资源较低, 未过门限
98	6.76%	不通过	19840	97	341.030	略高于 5% 门限
112	3.38%	通过	22528	111	首个达标有限深度	
完整	基准	基准	59776	305	288.388	全帧可靠性基准

5.5.3 不同量化位宽的完整 FER 性能

正式试验覆盖浮点以及 3–8 bit 六种量化输入。图??直接展示全部逐点 FER 曲线，未用插值生成曲线点；高信噪比的零误帧观测在对数轴上省略。

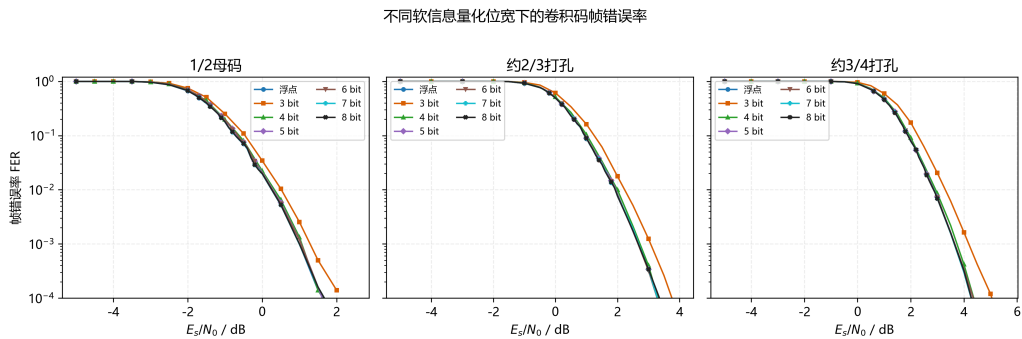


图 5.9: 不同软信息量化位宽下的卷积码帧错误率

5.5.4 量化性能损失与位宽选择

图??在 FER=0.1 处比较量化相对浮点的信噪比差值，逻辑顺序是“完整曲线—工作点差值—工程选择”。

Q8 在三码率上的损失分别为 0.006214、0.008130 和 -0.000741 dB，已接近统计与插值分辨率，因此量化实现推荐仍为 Q8；系统级五类方案仍以浮点软判决为正式比较口径。

表 5.7: 软信息量化实现参数

项目	正式实现
位宽含义	Qb 为 b bit 有符号输入码, $q_{\max} = 2^{b-1} - 1$, 使用对称整数区间 $[-q_{\max}, q_{\max}]$
输入与缩放	对解打孔后的实值接收样值量化;裁剪幅度 Q3–Q6 为 1.5, Q7–Q8 为 2.0; 步长 $\Delta = A/q_{\max}$
舍入与饱和	先按 $\text{lround}(y/\Delta)$ 舍入到最近整数, 再限幅到 $[-q_{\max}, q_{\max}]$
路径度量	量化输入采用平方欧氏分支度量, 累计量为 32 bit 有符号整数; 每个网格步减去最小度量进行归一化
溢出策略	候选值以 64 bit 临时量计算, 超出 32 bit 或 10^9 上限时饱和; 正式结果的两类计数均为 0
研究边界	不仅输入被量化, 量化译码支路的路径度量也为整数; 浮点基准另用浮点软译码器

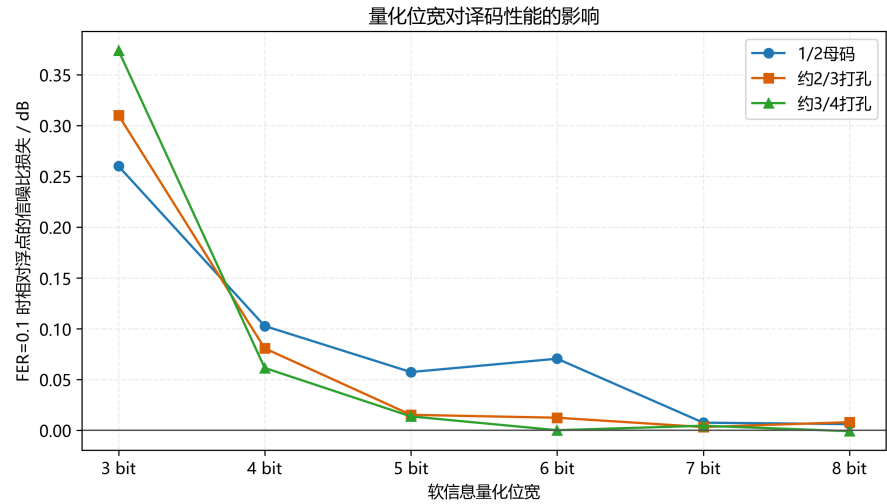


图 5.10: 软信息量化位宽的 FER=0.1 性能损失

表 5.8: 代表性软信息精度的 FER=0.1 性能损失与用途

精度	三码率相对浮点损失/dB	表示代价	推荐用途
浮点	0 (基准)	输入与度量精度最高	系统级软件基准
Q4	0.103 / 0.081 / 0.061	4 bit 输入与整数度量	存储受限候选
Q6	0.071 / 0.013 / 0.000	6 bit 输入与整数度量	中等位宽候选
Q8	0.006 / 0.008 / -0.001	8 bit 输入与整数度量	量化实现推荐

5.6 滑窗译码参数分析

5.6.1 窗口长度影响

窗口长度试验取 $W \in \{96, 128, 160, 192\}$ ，固定 $S = 16, D = 70$ 。图??表明，在当前控制条件下各曲线已高度接近： W 达到足够观察范围后，继续增加窗口的可靠性收益趋于饱和，却仍增加缓存和首窗口等待。

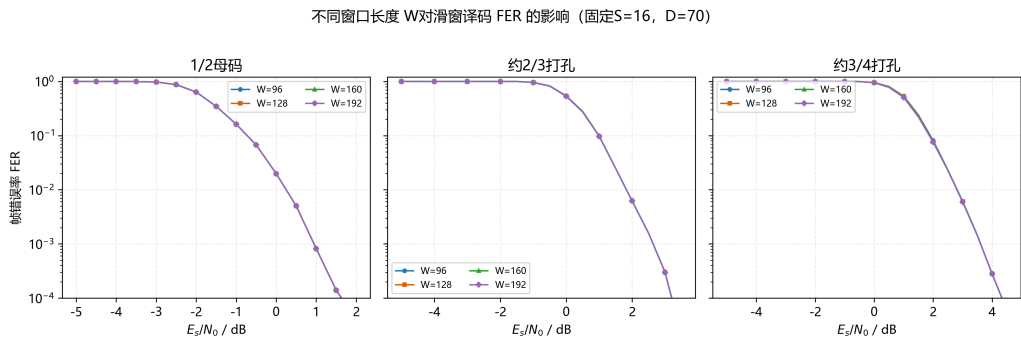


图 5.11: 不同窗口长度对滑窗译码 FER 的影响

5.6.2 滑动步长影响

步长试验取 $S \in \{8, 16, 25, 50\}$ ，固定 $W = 160, D = 70$ 。 S 主要控制输出批量和触发频率：较小 S 触发更频繁、重复计算更多；较大 S 扩大单批提交量并改变等待分布。它不是纠错深度，不能把“ S 越大”写成“纠错能力越强”。

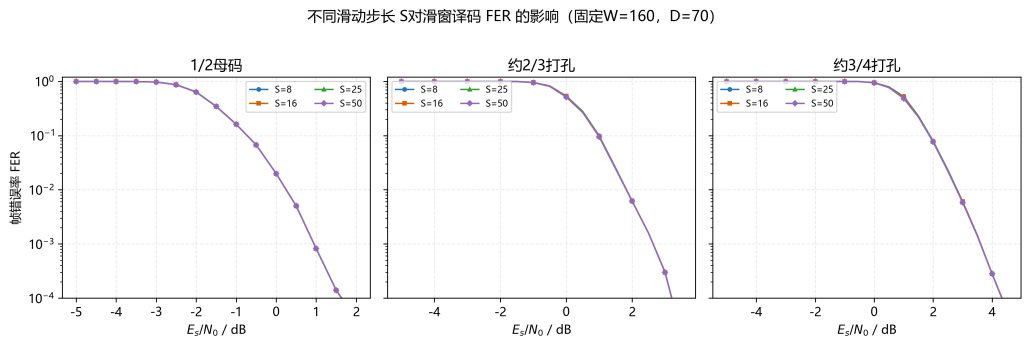


图 5.12: 不同滑动步长对滑窗译码 FER 的影响

5.6.3 回溯深度影响

滑窗回溯试验取 $D \in \{35, 49, 70, 84, 98, 112\}$ ，固定 $W = 160, S = 16$ 。小 D 增加过早提交风险，增大 D 后可靠性逐渐逼近饱和，同时提高等待、幸存路径存储和回溯工作量。约 3/4 最终候选的 $D = 126$ 没有进入这组完整控制扫描，只参加最终候选比较。

表 5.9: 滑窗控制变量试验摘要

变量	正式集合	固定条件	结构统计	工程解释
W	96/128/160/192	$S = 16, D = 70$	首次输出: 170/127/113 符号	观察充分后 FER 收益趋于饱和
S	8/16/25/50	$W = 160, D = 70$	$S = 16$ 时 P95: 260/195/173 符号	控制输出节奏与触发频率
D	35, 49, 70, 84, 98, 112	$W = 160, S = 16$	逐点曲线覆盖六个深度	增深降低过早提交风险并增加等待、存储

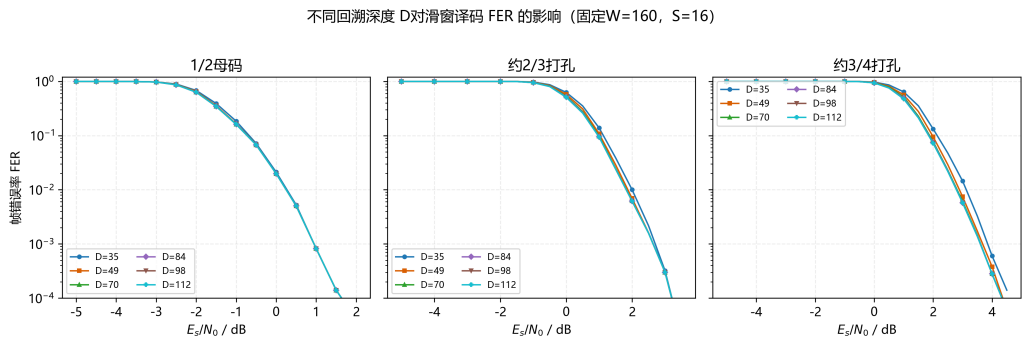


图 5.13: 不同回溯深度对滑窗译码 FER 的影响

5.6.4 滑窗参数综合选择

筛选遵循先满足可靠性、再比较等待、存储和处理量的顺序，不对 $W/S/D$ 机械取最大或最小。

表 5.10: 不同码率 and 目标下的滑窗参数推荐

码率方案	目标	W	S	D	选择含义
1/2 母码	综合/性能	128	25	70	冗余充足，较浅回溯即可稳定
1/2 母码	时延/存储	96	16	70	缩小缓存并保持可接受可靠性
约 2/3 打孔	综合/性能	128	25	98	以更深回溯补偿观测减少
约 2/3 打孔	时延/存储	128	25	84	在等待与可靠性间折中
约 3/4 打孔	综合/性能	160	25	126	最少冗余需要更长观察与回溯
约 3/4 打孔	时延/存储	128	25	98	接受性能代价以减少资源

不存在跨三码率统一最优的 $W/S/D$ 。打孔越多，竞争路径通常需要更长观察或回溯；目标函数不同，也会在相同码率下给出不同配置。

5.7 时隙组织与连续译码性能

5.7.1 连续编码与时隙组织方式

正式比较包括整块 300 bit，以及 50×6 、 100×3 、 150×2 三种连续时隙组织。连续方案跨时隙保持编码器状态和打孔相位，只有最后一个时隙

追加 6 bit 零尾；三种连续组织编码的是同一封 300 bit 电文，而不是多个独立短码块。

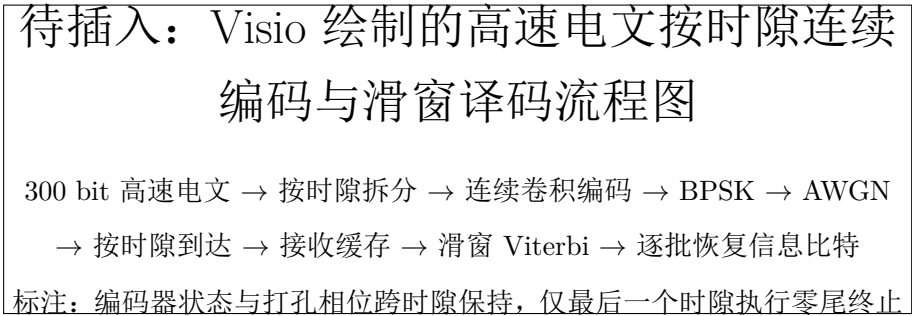


图 5.14: 高速电文按时隙连续编码与滑窗译码流程

5.7.2 硬判决下的时隙可靠性

图??比较整块与三种连续组织的硬判决 FER。对应 BER 组合图也已按同一正式 CSV 生成，并为控制正文篇幅移至附录??；300 bit 电文的主要交付指标采用 FER。

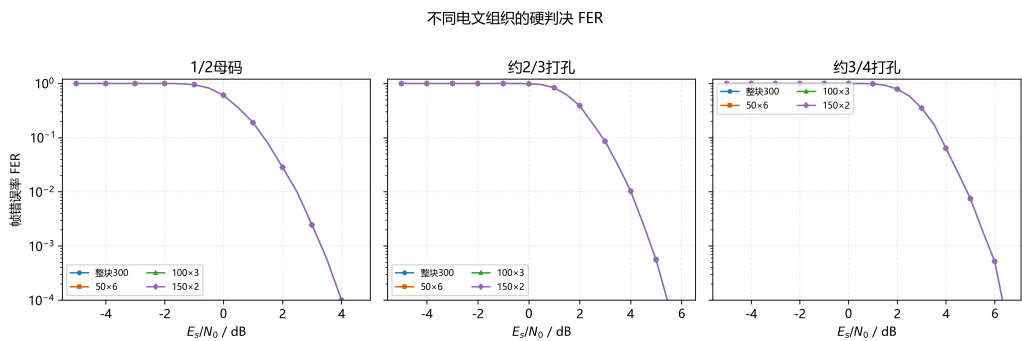


图 5.15: 不同电文组织的硬判决帧错误率

5.7.3 浮点软判决下的时隙可靠性

图??给出浮点软判决 FER。相同码率、判决方式和 E_s/N_0 下, 50×6 、 100×3 和 150×2 三种连续组织的 BER 与 FER 逐点完全重合。原因是最终发送码流、编码器状态、打孔相位、仅末时隙终止、接收序列、噪声、滑窗规则均一致; 时隙组织改变的是接收数据何时到达, 而不是最终卷积码字。不能据此声称 50×6 纠错更强。

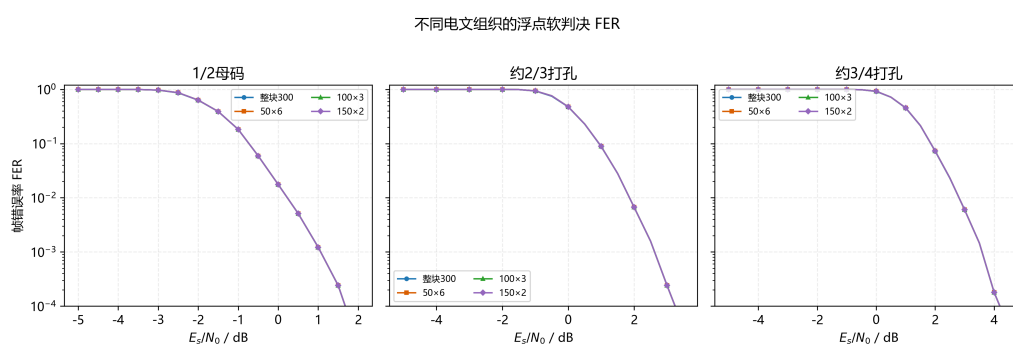


图 5.16: 不同电文组织的浮点软判决帧错误率

5.7.4 不同时隙组织的输出时延

时延以接收符号计数。首次输出反映最早结果, 第 95 百分位反映绝大多数信息比特的等待, 整帧最后决策符号反映全部交付时刻。

约 2/3 软判决的 50×6 和 150×2 均在 223 个符号后首次输出, 但 P95 分别为 256 和 397 个符号。 50×6 的低时延结论来自整体等待分布, 而不是可靠性差异。

5.7.5 不同时隙组织的有效吞吐率

归一化有效吞吐率同时受实际码率与 FER 影响。高码率并不自动产生更高成功业务交付; 只有可靠性仍可接受时, 较短发送长度才会转化为有效交付优势。

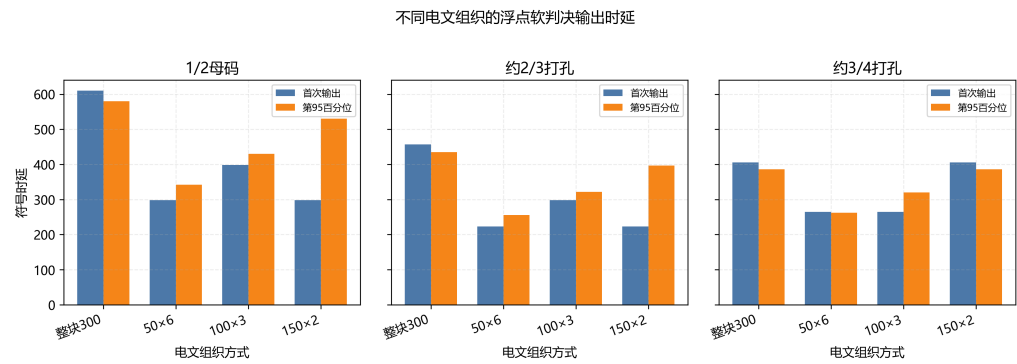


图 5.17: 不同时隙组织的浮点软判决输出时延

表 5.11: 不同时隙组织的浮点软判决输出时延

码率	组织方式	首次	平均	P95	最大	整帧最后符号
1/2	整块 300	610	311.0	580	610	611
1/2	50 × 6	298	231.0	342	360	611
1/2	100 × 3	398	258.0	430	460	611
1/2	150 × 2	298	285.0	530	560	611
约 2/3	整块 300	457	233.0	435	457	458
约 2/3	50 × 6	223	173.0	256	270	458
约 2/3	100 × 3	298	193.25	322	345	458
约 2/3	150 × 2	223	213.5	397	420	458
约 3/4	整块 300	406	207.0	386	406	407
约 3/4	50 × 6	265	171.17	262	273	407
约 3/4	100 × 3	265	183.5	320	340	407
约 3/4	150 × 2	406	207.0	386	406	407

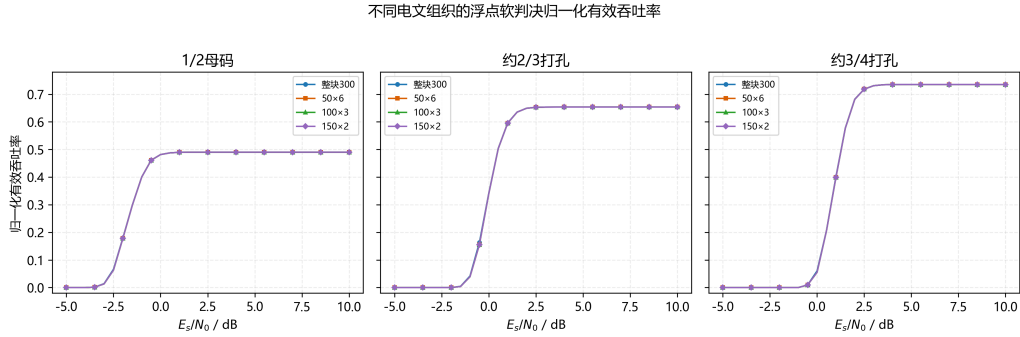


图 5.18: 不同电文组织的浮点软判决归一化有效吞吐率

硬判决有效吞吐率图已有正式数据支撑，但与硬判决 FER 图传达的信息重复，故不进入正文；正文保留浮点软判决主路线。

5.7.6 时隙组织选择

在三种连续组织可靠性完全相同的前提下，选择由到达节奏和时延分布决定。 50×6 更频繁地向接收缓存提供新符号，在约 2/3 方案中保持 223 符号的最早首次输出，并把 P95 压缩到 256 符号，因此继续作为低时延推荐。该结论依赖跨时隙状态与打孔相位连续；若各时隙独立清零或重新起相位，必须重新评估。

5.8 卷积码方案阶段性结论

表??把本章各控制问题压缩为可执行选择。它们共同构成一条完整工程链：300 bit 电文采用 $K = 7$ 、171/133 母码和 6 bit 零尾；通过打孔获得 612、459 和 408 bit 三种发送长度；浮点软判决建立性能基准；有限回溯、量化和滑窗再按资源目标降维；连续时隙在保持可靠性的同时调整交付时延。

五类推荐不是一个“统一最优方案”。可靠性与吞吐方案在固定 E_s/N_0 下比较，时延、存储和综合方案在固定 FER=0.1 下比较；基准不同，不能

表 5.12: 卷积码主要参数选择结论

设计问题	最终结论	工程原因
可靠性码率	1/2 母码	冗余最多，FER 门限最低
吞吐码率	约 3/4 打孔	408 bit 发送长度，实际码率最高
判决方式	浮点软判决	FER=0.1 处获得约 1.86–2.08 dB 收益
通用有限回溯	$D = 112$	唯一通过 5% 最坏相对 FER 增幅判据，存储降约 62.3%
量化实现	Q8	三码率相对浮点损失不超过约 0.008 dB 量级
滑窗配置	按码率和目标分别设置	冗余度改变路径收敛，时延与存储目标也不同
连续时隙	低时延优先 50×6	可靠性与其他连续组织相同,第 95 百分位等待更低

表 5.13: 卷积码五类系统级推荐

目标	推荐方案	比较基准	选择理由
可靠性优先	1/2 母码整块浮点软判决	固定 $E_s/N_0 = 2$ dB	共同信噪比下 FER 最低
吞吐优先	约 3/4 打孔滑窗, $W = 160, S = 25, D = 126$	固定 $E_s/N_0 = 2$ dB	实际码率与有效吞吐率最高
时延优先	约 2/3 打孔滑窗, $W = 128, S = 25, D = 84$	固定 FER=0.1	覆盖 FER 门限且首次输出最早
存储优先	1/2 母码滑窗, $W = 96, S = 16, D = 70$	固定 FER=0.1	覆盖 FER 门限且译码存储最低
综合平衡	1/2 母码滑窗, $W = 96, S = 16, D = 70$	固定 FER=0.1	综合四项指标最优

声称其中某一项全面优于其余方案。类似地，Q8 量化建议回答的是定点输入位宽问题，不改变表??以浮点软判决建立的系统级推荐口径。

工程部署时还应把这些推荐视为可复核的起点，而不是脱离链路条件的永久常数。若业务信噪比裕量降低，应优先检查高码率方案是否仍覆盖目标 FER；若接收缓存成为主要约束，可在同一码率内选择较小窗口配置；若处理器或硬件采用并行回溯，软件测得的时间排序也需要重新验证。尽管绝对资源会随平台变化，本章冻结的码长、状态数、路径度量、FER 工作点和符号时延仍提供了统一接口，使后续比较能够明确区分“编码方案差异”和“实现平台差异”。

本章已经完成卷积码内部设计和筛选。后续横向比较将使用冻结的 1/2 母码 612 bit 与约 2/3 打孔 459 bit 整块浮点软判决候选，并采用完整 306 步终止回溯；复杂度分析和突发错误交织专题也分别沿用已冻结配置。此处只给出输入边界，不提前展开多信道结果、译码算法横向复杂度或交织收益。由此，第 5 章回答了“为什么这样设计、正式结果如何、差异为何出现以及不同工程目标如何选择”，并为后续系统级比较提供一致的高速卷积码基线。

第六章 LDPC 方案与仿真分析

6.1 高速电文 LDPC 设计方案

面向 300 bit 高速二进制电文, LDPC 码可作为卷积码之外的高可靠短块编码方案。本章采用整块编码组织, 300 bit 信息在一个 LDPC 码字内完成编码与恢复, 不进行 LDPC 分块, 也不配置交织。编码器基于 5G NR BG2 准循环结构进行短块构造, 比较实际码长 480 bit、560 bit 和 640 bit, 并分别采用 BP 与 NMS 软译码器。评价内容包括 BER、FER、编码增益、迭代次数、平均与尾部软件译码时间、最大观测时间以及运算规模。

本章实现主要复用 BG2 结构、提升过程和稀疏校验矩阵构造, 不包含完整 5G NR 物理层中的传输块分段、HARQ 与完整速率匹配流程。三种方案共享 300 bit 原始信息、BPSK 调制和 AWGN 信道, 使可靠性、传输效率与译码负担之间的差异能够在统一链路条件下比较。

6.2 BG2 短块编码结构与参数

设 BG2 信息列数为 k_b , 选取的基图列数和行数分别为 n_b 与 m_b , 提升因子为 Z_c 。系统信息容量 K_{cap} 、填充位数 F 、实际码长 N 与实际码率 R 为

$$K_{\text{cap}} = k_b Z_c, \quad F = K_{\text{cap}} - 300, \quad N = n_b Z_c, \quad R = \frac{300}{N}. \quad (6.1)$$

300 bit 原始信息放入系统信息区, 其余位置填零, 再由准循环校验矩阵生成校验位并形成完整码字。该组织方式保留了系统码结构, 接收端译码后

可直接从信息区恢复 300 bit 电文。

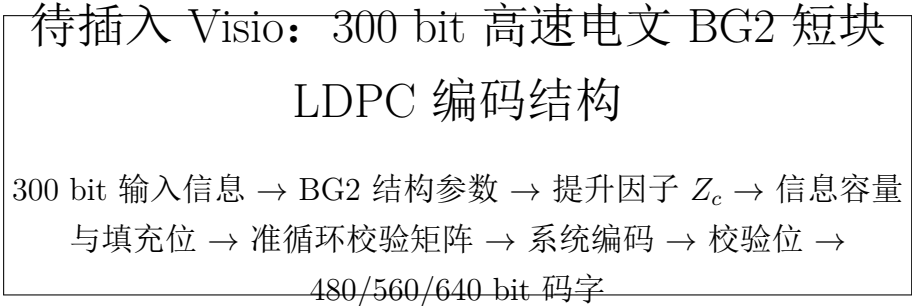


图 6.1: 300 bit 高速电文 BG2 短块 LDPC 编码结构

表 6.1: 300 bit 高速电文 LDPC 编码参数

方案	目标码长	实际码长	Z_c	k_b	n_b	m_b	信息容量	填充位	校验位	实际码率
N480	480	480	48	8	10	2	384	84	96	0.6250
N560	576	560	56	8	10	2	448	148	112	0.5357
N640	≤ 640	640	40	8	16	8	320	20	320	0.4688

教师给出的中间目标码长为 576 bit。BG2 短块构造受到提升因子与校验矩阵可编码条件的共同约束，精确 576 bit 候选未满足本系统的编码条件，因此采用距离目标最近的 560 bit 合法方案。后续仿真、码率和性能比较均按 560 bit 实际发送长度计算。

三种构造的差异不仅体现为发送长度变化，还包括提升因子、基图截取范围、填充量和校验约束分布的同步变化。N640 包含 320 个校验位且填充位仅 20 bit；N560 虽然发送长度居中，但信息容量达到 448 bit，其中 148 bit 为填充位。完整校验图结构由此成为短块性能分析的重要基础。

6.3 BP 与 NMS 分层译码

BP 译码器采用分层和积更新。每一层先从变量节点后验 LLR 中扣除旧校验消息，校验节点再通过双曲正切与反双曲正切运算形成新消息，并

立即更新变量节点后验信息。NMS 译码器保持相同分层顺序，以绝对值、最小值、次小值和符号积近似校验节点计算，并用归一化因子 α 修正消息幅度，其核心形式为

$$L_{c \rightarrow v}^{\text{NMS}} = \alpha s_{c \rightarrow v} \min_{v' \neq v} |L_{v' \rightarrow c}|. \tag{6.2}$$

每轮分层更新结束后，两种译码器均执行硬判决和综合校验。校验方程全部满足时提前输出，否则进入下一轮，最大迭代次数为 32。

待插入 Visio：BP 与 NMS 分层迭代译码流程

接收 LLR → 校验节点分层处理；BP：双曲正切/反双曲正切，
NMS：绝对值/最小值/次小值/符号/缩放 → 变量节点更新 →
硬判决 → 校验方程

通过 → 输出；未通过 → 下一迭代
图 6.2: BP 与 NMS 分层迭代译码流程

表 6.2: BP 与 NMS 译码运算结构

项目	BP	NMS
校验节点	双曲正切、乘积、反双曲正切	绝对值、最小值/次小值、符号、缩放
变量节点	外信息形成与后验 LLR 更新	外信息形成与后验 LLR 更新
收敛控制	硬判决、综合校验、提前停止	硬判决、综合校验、提前停止
性能定位	和积软译码基准	低代价近似软译码

BP 与 NMS 的基础运算类型存在明显差异。BP 校验节点包含双曲正切与反双曲正切等非线性运算，NMS 主要由绝对值、比较、符号判断和缩放乘法构成。因此，消息更新次数表示传播规模，基础运算事件数表示计算事件规模，而软件译码时间进一步反映这些运算在处理器上的综合执行代价。

6.4 NMS 归一化因子性能分析

NMS 以 α 修正最小和近似产生的消息幅度。较小的 α 会压缩校验消息，使部分帧需要更多迭代才能收敛；较大的 α 提高消息幅度，同时也增强错误信息在图中的传播。归一化因子在 0.60 ~ 1.00 范围内按 0.05 步长比较，表??列出四个关键取值。平均 FER、平均迭代和平均时间由每个码长的五个扫描点计算；错误合法码字率来自瀑布区末端两个工作点的 600 帧专项分类统计。

表 6.3: NMS 归一化因子关键性能比较

码长	α	扫描 $E_s/N_0/\text{dB}$	平均 FER	平均迭代	平均时间/ μs	错误合法码字率
480	0.80	2.5 ~ 4.5	0.4707	11.437	90.65	7.33%
480	0.90	2.5 ~ 4.5	0.4427	7.187	58.10	11.00%
480	0.95	2.5 ~ 4.5	0.4273	4.279	35.83	14.17%
480	1.00	2.5 ~ 4.5	0.4200	1.485	13.69	17.33%
560	0.80	2.5 ~ 4.5	0.5047	13.753	120.38	7.17%
560	0.90	2.5 ~ 4.5	0.4753	8.448	77.02	11.50%
560	0.95	2.5 ~ 4.5	0.4673	5.287	49.68	14.67%
560	1.00	2.5 ~ 4.5	0.4613	1.537	16.45	19.50%
640	0.80	-2.5 ~ -0.5	0.3653	15.949	542.20	0
640	0.90	-2.5 ~ -0.5	0.3820	16.611	567.07	0
640	0.95	-2.5 ~ -0.5	0.4153	17.592	604.48	0
640	1.00	-2.5 ~ -0.5	0.4760	19.243	657.36	0

N480 在 $\alpha = 0.90$ 与 0.95 时的平均 FER 分别为 0.4427 和 0.4273，绝对差为 0.0154；N560 对应差值为 0.0080。0.95 使两种码长的平均迭代和软件时间进一步下降，但错误合法码字率均比 0.90 增加 3.17 个百分点。因此，N480 与 N560 在 0.90 ~ 0.95 附近形成可靠性、收敛速度与错误收敛风险之间的综合取舍区间，后续大规模性能比较采用 $\alpha = 0.95$ 。

N640 的变化方向不同： $\alpha = 0.80$ 的平均 FER 为 0.3653，低于 0.90、

0.95 和 1.00，对应较优区间向较小系数移动，性能比较采用 $\alpha = 0.80$ 。N480 与 N560 在 $\alpha = 1.00$ 时表现出最低的表面平均 FER 和最短时间，同时错误合法码字率升至 17.33% 和 19.50%。该现象说明较大的消息幅度会使部分错误帧快速进入错误但满足校验方程的码字，较短的迭代过程由此未必对应更可靠的恢复结果。

6.5 仿真条件与评价指标



图 6.3: LDPC 仿真链路与软件译码时间统计位置

表 6.4: 软件译码时间统计范围与运行环境

项目	内容
计时范围	BP/NMS 迭代、校验节点更新、变量节点更新、硬判决、综合校验与提前停止
计时外环节	LDPC 编码、BPSK、AWGN、LLR 生成、BER/FER 汇总、文件 I/O 与绘图
操作系统	Windows 10.0.26200
处理器架构	Intel64 Family 6 Model 191
编译与并行	Release/O3; 6 进程
计时器与单位	std::chrono::steady_clock; μs

运行记录保存了处理器架构信息，未包含完整商品型号，因此软件译

码时间用于同一实现环境中的相对比较。计时边界位于译码函数入口和返回位置，反映软件译码内核时间，而非端到端通信时延。

表 6.5: LDPC 性能仿真参数

参数	设置
信息长度与实际码长	300 bit; 480/560/640 bit
调制与信道	BPSK; AWGN
信噪比范围与步长	$E_s/N_0 = -5 \sim 10$ dB; 0.5 dB
仿真帧停止条件	最少 1000 帧; 目标 200 错误帧; 最多 50000 帧
译码控制	最大 32 次迭代; 每个完整迭代后进行综合校验并提前停止
公平比较条件	同一码长、信噪比与帧索引共享信息、码字、噪声样本和 LLR

对幅度为 ± 1 的 BPSK 符号，噪声方差与输入译码器的 LLR 为

$$\sigma^2 = \frac{1}{2} 10^{-(E_s/N_0)/10}, \quad L = \frac{2y}{\sigma^2}, \quad (6.3)$$

其中 y 为接收符号。BER 与 FER 基础曲线采用 E_s/N_0 ，用于观察相同符号能量条件下的译码表现。不同实际码率之间的编码增益采用

$$(E_b/N_0)_{\text{dB}} = (E_s/N_0)_{\text{dB}} - 10 \log_{10} R, \quad (6.4)$$

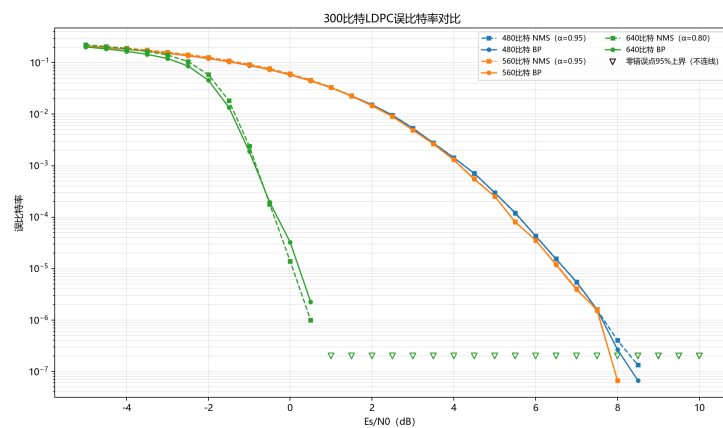
并定义候选方案相对参考方案的增益为

$$G = (E_b/N_0)_{\text{reference}} - (E_b/N_0)_{\text{candidate}}. \quad (6.5)$$

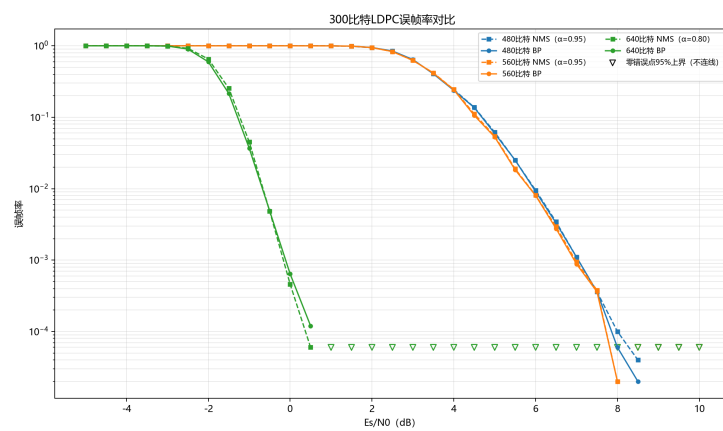
正增益表示候选方案在相同目标误差率下所需 E_b/N_0 更低。

6.6 误码性能分析

图??显示，三种码长均随 E_s/N_0 提高进入明显瀑布区，N640 的 BER 与 FER 曲线整体位于 N480 和 N560 左侧。表??采用相邻非零 FER 点的



(a) BER



(b) FER

图 6.4: 不同 LDPC 码长及 BP/NMS 译码的误码性能

对数线性插值, 给出统一可靠性目标下所需的 E_s/N_0 ; 全部目标均位于原始采样区间内。

表 6.6: 不同方案典型 FER 工作点

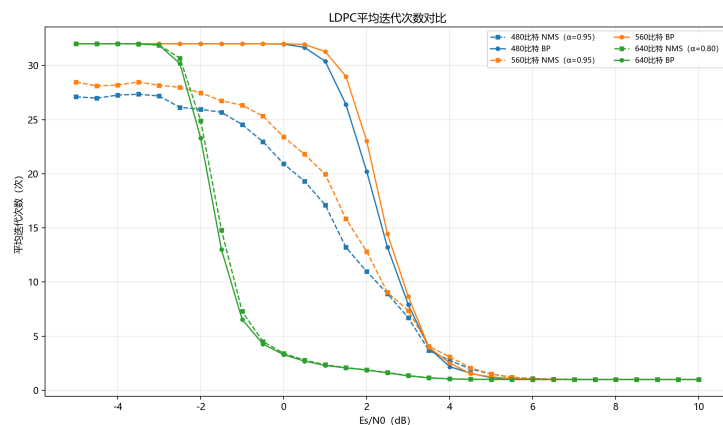
码长	算法	FER= 0.1/ E_s/N_0 /dB	FER= 0.01/ E_s/N_0 /dB	实际码率
480	BP	4.687	5.956	0.6250
480	NMS	4.700	5.975	0.6250
560	BP	4.541	5.866	0.5357
560	NMS	4.577	5.875	0.5357
640	BP	-1.283	-0.678	0.4688
640	NMS	-1.230	-0.663	0.4688

在 FER= 0.1 处, N640 相对 N480 所需 E_s/N_0 分别降低 5.970 dB (BP) 和 5.930 dB (NMS); FER= 0.01 处对应降低 6.634 dB 和 6.638 dB。该优势来自更低实际码率、更多校验约束、不同提升因子、准循环结构以及较小填充占比的共同作用。当前完整 N640 构造由此适合可靠性优先的高速电文。

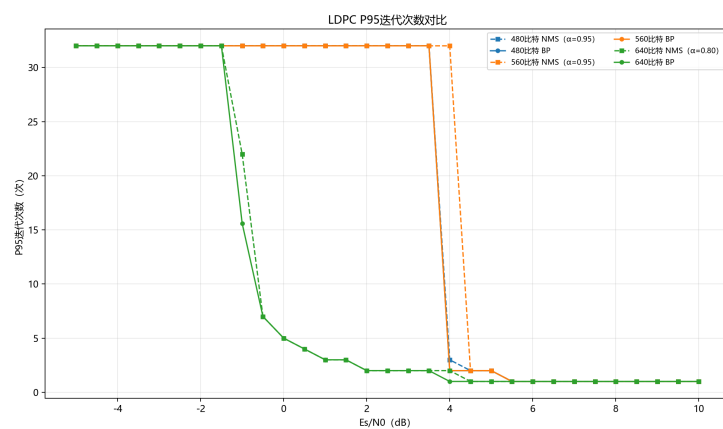
N560 在 FER= 0.1 处相对 N480 仅降低 0.146 dB (BP) 和 0.123 dB (NMS), FER= 0.01 处仅降低 0.090 dB 和 0.100 dB, 没有形成随发送长度增加而自然增长的中间可靠性收益。N560 的 $Z_c = 56$ 、信息容量 448 bit 和填充位 148 bit 均与 N480 不同, 截取矩阵结构与校验约束分布也同步改变。较大的填充量是影响短块结构与有效约束利用的重要差异之一, 但性能变化由完整校验图共同决定。

同一码长下, BP 与 NMS 的 FER 目标点十分接近: FER= 0.1 时所需 E_s/N_0 差值为 0.013~0.053 dB, FER= 0.01 时为 0.009~0.019 dB。NMS 在适当归一化系数下保持了接近 BP 的可靠性, 为后续降低软件计算时间提供了基础。

6.7 迭代次数与软件译码时间



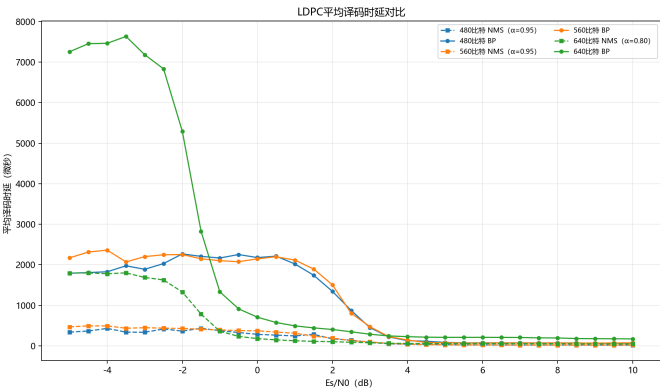
(a) 平均迭代次数



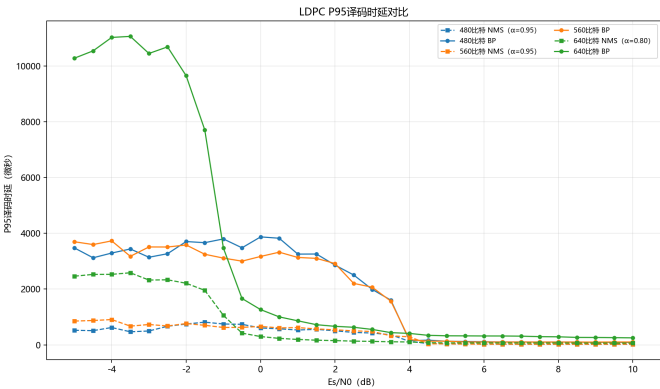
(b) P95 迭代次数

图 6.5: BP 与 NMS 译码迭代特性

低信噪比区域的 LLR 可靠性较弱, 校验方程长期未满足, 大量帧运行至 32 次迭代上限, 平均与 P95 迭代数处于高位。进入瀑布区后, 更多帧在较少迭代内通过综合校验, 提前停止比例提高, 平均迭代次数和软件时间随之快速下降。高信噪比区域的可靠 LLR 使多数帧在 1~2 次迭代内收敛, 形成曲线右侧的平台。



(a) 平均软件译码时间



(b) P95 软件译码时间

图 6.6: BP 与 NMS 软件译码时间

在 FER= 0.1 的统一可靠性条件下，NMS 平均迭代数比 BP 高 5.3%~31.5%，但平均软件时间分别降低 69.2%（N480）、64.9%（N560）和 74.5%（N640）；P95 时间分别降低 72.0%、71.0% 和 74.9%。NMS 单次校验节点更新由比较、绝对值、符号和缩放构成，其较低单次计算代价抵消了增加的迭代次数。

表??保留了最大观测软件译码时间。最大值同时受到译码负载与 Windows 任务调度影响，反映观测期间的极端样本；平均值与 P95 对典型软件执行时间的描述更稳定。N640 在相同 FER 目标下仍具有较高迭代和时间负担，源于其 2240 条边和更大的校验图规模，体现了可靠性提升与译码资

表 6.7: 相同 FER 条件下的 BP/NMS 迭代与软件时间比较

码长	算法	FER 目标	$E_s/N_0/\text{dB}$	平均迭代	P95 迭代	平均时间/ μs	P95 时间/ μs	最大观测/ μs
480	BP	0.1	4.687	1.418	2.00	103.17	155.03	4200.3
480	NMS	0.1	4.700	1.779	2.00	31.76	43.34	1950.2
560	BP	0.1	4.541	1.508	2.00	82.06	132.49	4068.7
560	NMS	0.1	4.577	1.983	2.00	28.84	38.44	750.6
640	BP	0.1	-1.283	10.205	24.88	2178.41	5869.05	15850.9
640	NMS	0.1	-1.230	10.741	26.59	554.95	1471.03	5185.5

源之间的交换关系。

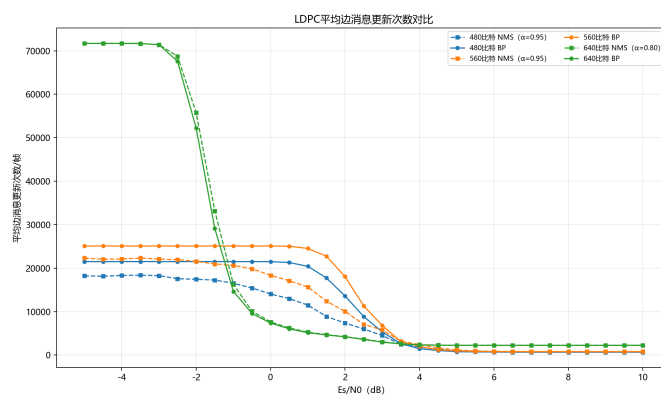
6.8 译码复杂度与错误合法码字特性

表 6.8: BP/NMS 复杂度与错误合法码字关键结果

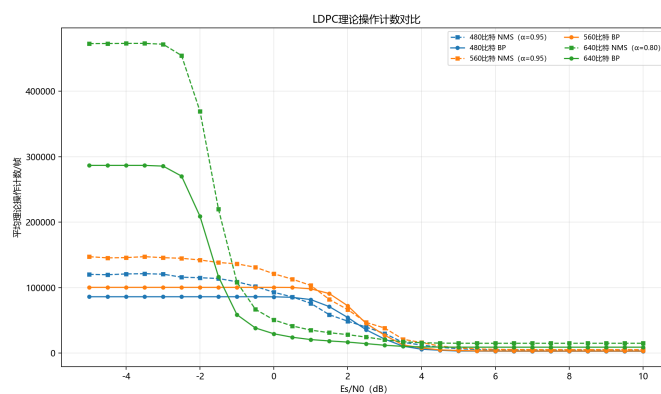
码长	算法	FER 工作点	平均迭代	平均边消息更新	基础运算事件数	平均时间/ μs	错误合法码字率
480	BP	0.1	1.418	952.6	3810.5	103.17	10.017%
480	NMS	0.1	1.779	1195.2	7910.8	31.76	9.164%
560	BP	0.1	1.508	1182.5	4730.1	82.06	9.396%
560	NMS	0.1	1.983	1555.0	10290.3	28.84	8.641%
640	BP	0.1	10.205	22859.2	91436.7	2178.41	0
640	NMS	0.1	10.741	24060.3	159431.0	554.95	0

迭代次数决定消息传播轮数，边消息更新数进一步体现每帧执行了多少次稀疏图消息传播。基础运算事件数加入不同算法的操作项目后，NMS 在 FER= 0.1 处的未加权事件数高于 BP；然而 NMS 的平均软件时间仍显著更低。该差异表明基础事件的单次代价并不相同，BP 的非线性函数计算在当前 CPU 实现中占据了更高成本。

错误合法码字表示译码输出满足全部校验方程，但恢复的 300 bit 信息仍然错误。图??呈现出明显的信噪比分区：低信噪比下，大量帧无法进入任何合法码字，错误合法码字比例处于低位；中等信噪比下，部分错误帧



(a) 平均边消息更新次数



(b) 基础运算事件数

图 6.7: BP 与 NMS 译码计算规模

能够收敛到错误但合法的码字，N480 与 N560 形成峰值；高信噪比下，正确码字成为主要收敛结果，错误合法码字率随总错误数共同下降。

N480 的峰值为 49.1% (BP, 2.5 dB) 和 64.5% (NMS, 2.0 dB)，N560 对应为 44.9% 和 60.8% (均在 2.5 dB)。N640 仅在 -0.5 dB 附近出现约 2.41×10^{-5} 的极低观测值。归一化因子分析中 N480 与 N560 在 $\alpha = 1.00$ 时错误合法码字增多，与中等信噪比下较强消息幅度引起的错误收敛相吻合。工程接收机因而同时利用信息正确性统计与综合校验结果评价译码质量。

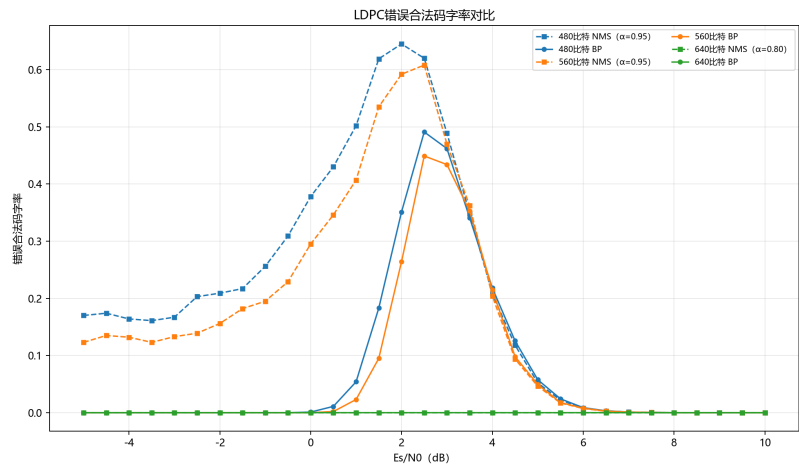
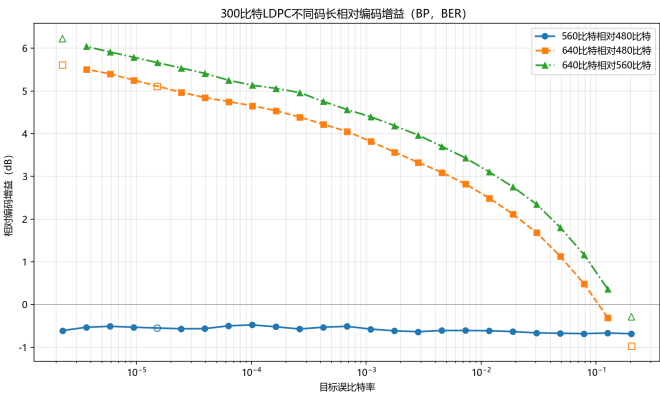
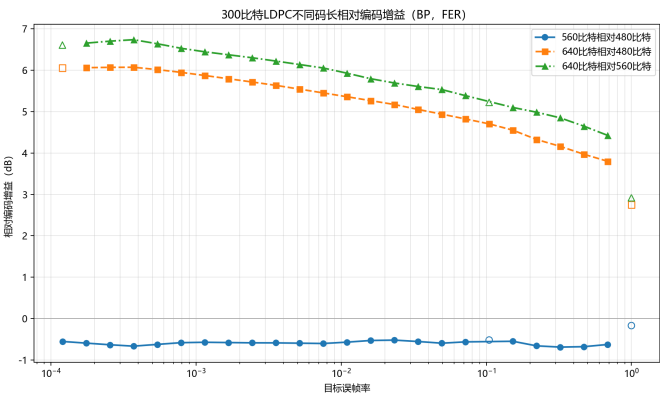


图 6.8: BP 与 NMS 错误合法码字率

6.9 不同码长及编码增益分析

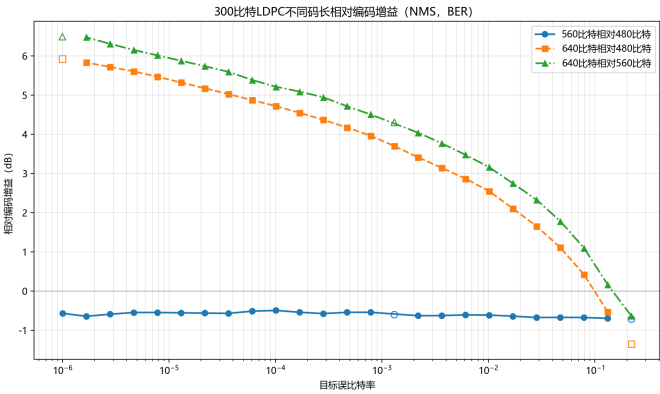


(a) BER 相对编码增益

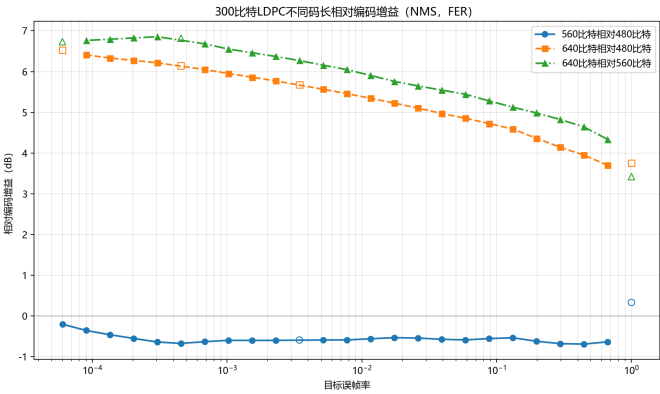


(b) FER 相对编码增益

图 6.9: BP 译码下不同 LDPC 方案的相对编码增益



(a) BER 相对编码增益



(b) FER 相对编码增益

图 6.10: NMS 译码下不同 LDPC 方案的相对编码增益

表 6.9: 三码长平均相对编码增益

算法	指标	N560 相对 N480/dB	N640 相对 N480/dB	N640 相对 N560/dB
BP	BER	-0.587	3.462	4.049
BP	FER	-0.576	5.167	5.742
NMS	BER	-0.598	3.591	4.189
NMS	FER	-0.532	5.318	5.849

在 25 个共同目标误差率点上, N560 相对 N480 的平均增益约为 $-0.5 \sim -0.6$ dB。固定 FER 时, N560 在 E_s/N_0 坐标上仅获得 $0.09 \sim 0.15$ dB 改善; 换算到每信息比特能量后, 其较低码率对应的能量归一化量抵消了这部分改善, 平均 E_b/N_0 增益由此为负。N560 的提升因子、信息容量、148 bit 填充量、基图截取范围和校验约束分布同时发生变化, 额外发送长度没有转化为同比例的可靠性收益。

N640 相对 N480 的平均 BER 增益为 3.462 dB (BP) 和 3.591 dB (NMS), 平均 FER 增益达到 5.167 dB 和 5.318 dB; 相对 N560 的 FER 增益进一步达到 5.742 dB 和 5.849 dB。当前完整 N640 构造具有 0.46875 的实际码率、320 个校验位、20 个填充位、 $Z_c = 40$ 以及 $n_b/m_b = 16/8$ 结构, 其优势由更低码率、更多校验约束、不同准循环结构和较小填充占比共同产生。相应代价是 640 bit 发送长度和更大的译码图规模。

6.10 LDPC 方案选择

N480 以最高码率和最短发送长度满足传输效率优先场景; N640 以更长码字和更大校验图换取显著可靠性增益, 适合链路可靠性优先的高速电文。N560 接近 576 bit 目标, 但在当前短块构造中没有形成明显的中间性能优势, 其价值主要体现在特定码长接口适配。

BP 提供和积译码性能基准, NMS 在适当归一化系数下维持接近 BP 的 BER 与 FER, 同时显著降低平均与 P95 软件译码时间。综合可靠性、

表 6.10: 不同 LDPC 码长方案工程取舍

方案	码长	实际码率	主要优势	适用场景
N480	480	0.6250	发送最短、码率最高	链路余量充足、传输效率优先
N560	560	0.5357	接近 576 bit 目标	特定码长接口适配；非默认性能折中
N640	640	0.4688	误码性能最强	高速电文可靠性优先

表 6.11: BP 与 NMS 译码工程比较

算法	误码与迭代特性	主要运算	软件时间	工程定位
BP	可靠性基准；迭代数通常较少	双曲正切、反双曲正切与消息更新	同等 FER 下较长	性能基准与高精度软译码
NMS	性能接近 BP；迭代数略高	最小值、比较、符号与缩放	同等 FER 下降约 65%~75%	低计算代价工程实现

软件实时性与实现复杂度，NMS 适合作为工程译码器，BP 适合作为性能参照；码长则依据传输效率与可靠性优先级在 N480 和 N640 之间选择。

以 FER= 0.1 为链路设计点时，N480 的 BP/NMS 门限约为 4.69/4.70 dB，N640 约为 -1.28/ - 1.23 dB，两种完整构造之间形成约 5.9 dB 的 E_s/N_0 余量差。链路可提供较高接收信噪比时，N480 以 160 bit 的码长节省获得更高传输效率；弱信号或衰落余量较小时，N640 用较低实际码率换取更早进入低 FER 区域。N560 的门限仅比 N480 降低约 0.12~0.15 dB，其使用条件更多取决于 576 bit 附近的接口长度需求。

软件实时性进一步区分了译码器选择。在 FER= 0.1 处，N640 采用 NMS 时的平均与 P95 译码时间约为 555 μs 和 1471 μs ，明显低于 BP 的 2178 μs 和 5869 μs ；N480 与 N560 的 NMS 平均时间均低于 32 μs 。由此，可靠性优先且具有毫秒级软件预算的场景可采用 N640 与 NMS 组合，传输效率和低处理负担优先的场景可采用 N480 与 NMS 组合。

码长由目标 FER、可用 E_s/N_0 与发送长度共同确定，译码算法由软件时间预算和实现资源确定。

第七章 高速电文不同信道下卷积码与 LDPC 对比

7.1 对比方案与系统级评价指标

本章面向 300 bit 高速二进制电文，比较前文确定的卷积码与 LDPC 工程候选。比较对象按实际码率组织为两组：近 2/3 码率组由卷积码 R2/3 与 LDPC N480 构成，实际码率分别为 0.6536 和 0.6250；近 1/2 码率组由卷积码 R1/2 与 LDPC N640 构成，实际码率分别为 0.4902 和 0.46875。这样的分组避免将冗余度差异过大的方案直接横向排序，使可靠性、发送开销和软件处理代价具有可解释的比较基础。

卷积码采用约束长度 $K = 7$ 、生成多项式 171/133（八进制）和 6 bit 零尾终止。R2/3 方案由母码输出按 1101 模式打孔，发送长度为 459 bit；R1/2 方案发送完整 612 bit 母码。接收端把信道 LLR 换算为 BPSK 软符号，R2/3 方案先恢复母码域观测掩码，再以软符号与支路参考符号之间的欧氏距离构造整块浮点软判决 Viterbi 分支度量。LDPC 采用 BG2 Direct QC-LDPC 短块构造和分层归一化最小和译码，N480 与 N640 的归一化系数分别为 0.95 和 0.80，最大迭代次数均为 32。四种方案均不配置交织。

系统评价分别报告 AWGN 基准、各非 AWGN 信道条件下的 BER/FER 与目标 FER 门限、相对本方案 AWGN 门限的附加 E_s/N_0 ，以及软件译码时间、接收算法时间和 LDPC 迭代负担。绝对门限表征方案在当前信道条件下达到目标 FER 所需的 E_s/N_0 ，相对 AWGN 附加 E_s/N_0 表征该信道

表 7.1: 高速电文正式比较方案

比较组	方案	信息长度/bit	发送长度/bit	实际码率	译码器	关键参数
近 2/3	卷积码 R2/3	300	459	0.6536	整块浮点软判决 Viterbi	$K = 7, 171/133, 1101$ 打孔, 6 bit 零尾
近 2/3	LDPC N480	300	480	0.6250	分层 NMS	BG2 Direct, $\alpha = 0.95$, 最多 32 次
近 1/2	卷积码 R1/2	300	612	0.4902	整块浮点软判决 Viterbi	$K = 7, 171/133, 6$ bit 零尾
近 1/2	LDPC N640	300	640	0.46875	分层 NMS	BG2 Direct, $\alpha = 0.80$, 最多 32 次

相对于方案自身基线引入的额外代价；两类指标描述不同的性能维度。本章保留各项指标的独立物理含义，不构造缺少业务权重依据的综合分数。



图 7.1: 高速电文卷积码与 LDPC 多信道对比仿真链路

7.2 多信道仿真模型与公平比较条件

发射端采用 BPSK 映射，信息比特 0 映射为 +1，比特 1 映射为 -1。横轴统一采用 E_s/N_0 ，复基带 AWGN 的同相和正交分量由独立标准高斯样本生成，每一实分量的方差为

$$\sigma^2 = \frac{1}{2 \times 10^{(E_s/N_0)/10}}.$$

(7.1)

除纯 AWGN 基准外，其余五类条件均先施加确定性信道失真或随机干扰，再叠加同一口径的 AWGN。表??给出各模型及接收处理条件。

固定多径采用抽头 [1, 0.65, 0.35]、时延 [0, 1, 3] 的线性卷积，抽头按总能量归一化。接收端已知固定抽头并执行实轴 MMSE 处理；若均衡输出、等效增益和等效方差分别记为 $\hat{x}_k$ 、 g_k 和 v_k ，则软信息为 $L_k = 2g_k\hat{x}_k/v_k$ 。

因此，多径结果是“多径信道、已知 MMSE 接收处理和编码方案”的联合系统表现，不能把全部差异归因于编码器。

固定频偏模型在一帧内部令相位从 0° 线性累积至 30° ，并未把所有符号统一旋转 30° 。为表征多普勒类时变频偏对高速电文的影响，本章另采用线性时变频偏简化模型。对于 $N_{\text{tx}} > 1$ ，其瞬时归一化频偏为

$$\epsilon_i = \frac{2}{3(N_{\text{tx}} - 1)} \left(\frac{i}{N_{\text{tx}} - 1} - \frac{1}{2} \right), \quad \phi_0 = 0, \quad \phi_i = \phi_{i-1} + 2\pi\epsilon_{i-1}, \quad (7.2)$$

即 ϵ_i 随符号索引线性变化，累计相位由逐符号积分形成。该受控基带模型未输入载波频率、相对径向速度、轨道参数或多普勒变化率，因而不等同于依据具体运动学参数建立的实际多普勒物理信道。固定频偏与线性时变频偏均不执行载波补偿；后者的频偏与相位轨迹还随发送长度变化，因此 459、480、612 和 640 bit 码字经历的轨迹并不完全相同。

局部连续非理想条件包含已知 5% 连续遮挡与未知 5% 突发干扰。遮挡长度取 $\text{round}(0.05N_{\text{tx}})$ ，作用在实际发送符号域，接收端知道遮挡位置并将其 LLR 置为 0。未知突发干扰在连续 5% 发送符号上叠加 ISR 为 10 dB 的独立复高斯干扰，接收端不知道污染位置，仍按名义 AWGN 方差生成 LLR。前者提供中性可靠度，后者把受污染观测作为有效软信息送入译码器，因而二者对软译码的影响机制不同。

表 7.2: 六类信道模型与接收处理

信道	模型与参数	AWGN 叠加	接收端已知条件	接收处理与软信息
AWGN	无附加信道扰动	是	噪声方差	实轴投影, $L = 2\text{Re}\{y\}/\sigma^2$
固定多径	归一化三抽头 [1, 0.65, 0.35], 是时延 [0, 1, 3]	是	固定抽头	实轴 MMSE, $L_k = 2g_k \hat{x}_k / v_k$
固定频偏	帧内相位 $0^\circ \rightarrow 30^\circ$	是	不补偿相位	实轴投影, 按名义 AWGN 方差生成 LLR
时变频偏	瞬时归一化频率线性变化, 相位逐符号累积	是	不补偿频率	实轴投影, 按名义 AWGN 方差生成 LLR
已知遮挡	连续 $\text{round}(0.05N_{\text{tx}})$ 个发送符号置零	是	遮挡掩码	遮挡位置 LLR 置 0
未知突发	连续 5% 发送符号叠加复高斯干扰, ISR=10 dB	是	不知道污染掩码	受污染样本仍按名义 AWGN 模型生成 LLR

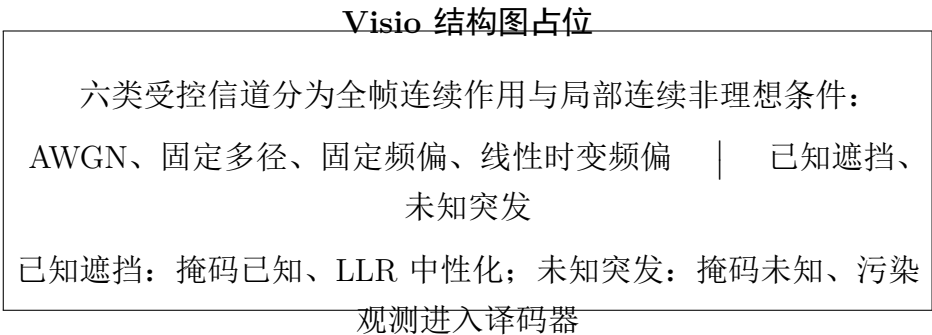


图 7.2: 六类受控信道及接收处理关系

正式仿真在 $-5 \sim 10$ dB 范围内按 0.5 dB 步长设置 31 个 E_s/N_0 点。每个“比较组—信道—信噪比”任务同时推进两种候选，至少运行 1000 帧；当两种方案都累计到 200 个错误帧时停止，否则最多运行 50000 帧。这种配对停止使同一横向比较点的两种方案具有相同最终帧数，避免某一方案因独立提前停止而获得不同的统计样本量。

相同帧索引下，各方案使用同一 300 bit 电文生成规则。噪声由主种子、噪声组、帧索引、符号索引和噪声策略版本共同确定；由于发送长度不同，四种方案的全部噪声样本并非逐个相同，但相同索引遵循一致的生成规则。连续遮挡与突发干扰使用同一相对起点，再映射到不同 N_{tx} 的发送域，因此绝对起点可以不同而相对位置保持可比。

7.3 AWGN 基准性能

AWGN 是后续相对信道损失的基准。图??单独展示两组 FER 瀑布曲线。近 2/3 组中，卷积码 R2/3 达到 FER= 0.1 和 0.01 所需的 E_s/N_0 门限估计分别为 0.912 dB 和 1.857 dB，而 LDPC N480 分别为 4.671 dB 和 6.027 dB，卷积码门限低 3.759 dB 和 4.170 dB。该组的绝对可靠性由卷积码占优，且差异远大于同组 0.0286 的实际码率差。

近 1/2 组呈现相反关系。卷积码 R1/2 达到 FER= 0.1 和 0.01 分别需要 -0.724 dB 和 0.244 dB，LDPC N640 分别需要 -1.241 dB 和 -0.683 dB。N640 所需门限低 0.517 dB 和 0.926 dB，说明在该组冻结的短块构造

表 7.3: 多信道正式仿真公共参数与公平性条件

项目	设置
信息与调制	300 bit 高速电文; BPSK, 0 映射为 +1、1 映射为 -1
横轴与范围	$E_s/N_0 = -5 \sim 10$ dB, 步长 0.5 dB, 共 31 点
停止条件	最少 1000 帧; 两方案均达到 200 个错误帧后配对停止; 最多 50000 帧
比较规模	2 个近码率组、6 类信道、31 个信噪比点, 共 372 个配对任务、744 个方案点
电文公平性	相同帧索引使用同一 300 bit 电文生成规则
噪声公平性	共享主种子、噪声组、帧索引和符号索引规则; 发送长度不同
局部位置公平性	相同帧索引共享相对起点, 再映射至各方案实际发送长度
统计量	BER、FER、译码时间、接收算法时间; LDPC 另统计迭代次数和达到上限的帧比例

与译码配置下,增加到 640 bit 码长后,LDPC 的分布式校验约束在 AWGN 中形成了更有利的瀑布区。上述门限均由夹住目标值的相邻非零 FER 实测点在 FER 对数域内插值得到,不是原始采样点。

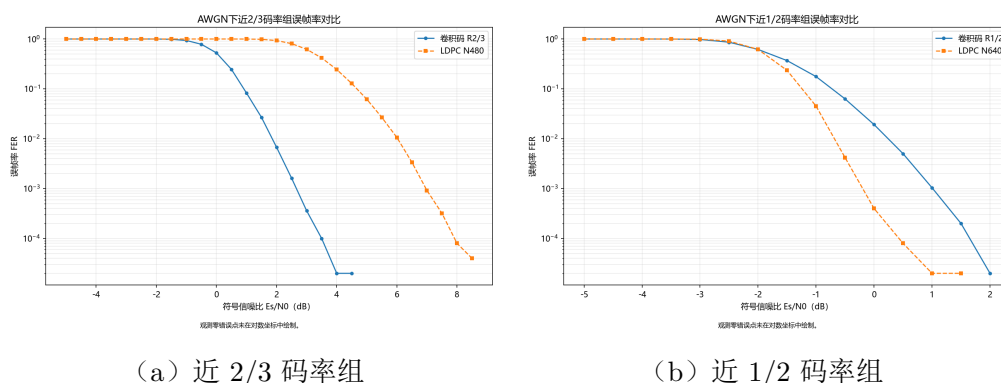


图 7.3: 不同近似码率组在 AWGN 信道下的误帧性能

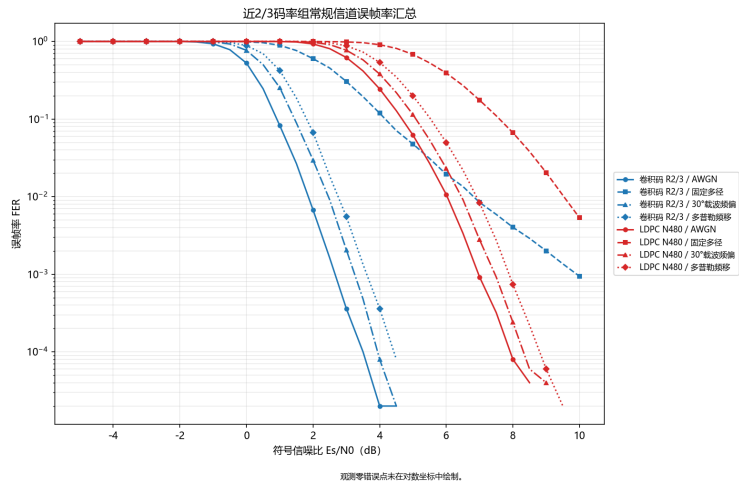
AWGN 基准还表明,码型名称不足以单独决定性能排序。N480 在近 2/3 组的两个目标门限分别高于卷积码 3.759 dB 和 4.170 dB,而 N640 在近 1/2 组分别低 0.517 dB 和 0.926 dB;该差异同时受到实际码率、短块校验结构、归一化系数和码长影响。各非 AWGN 条件下的结果同时报告绝对门限和相对各自 AWGN 基线的附加 E_s/N_0 。

7.4 多径与频偏条件下的可靠性

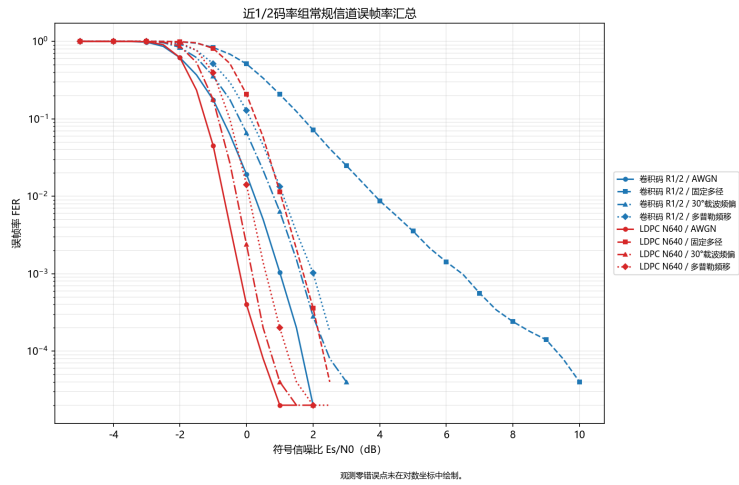
图??和图??汇总 AWGN、固定多径、固定频偏和线性时变频偏四种条件。FER 直接反映 300 bit 高速电文能否整帧正确恢复,BER 补充描述错误帧内部的错误比特密度。固定多径和两类频偏条件均使瀑布区相对 AWGN 右移,并改变同一近码率组内两种方案之间的门限间距。

7.4.1 固定多径信道

固定多径在已知三抽头和实轴 MMSE 处理后仍产生最大的常规信道门限右移。图??显示,近 2/3 组中卷积码达到 FER= 0.1/0.01 需要

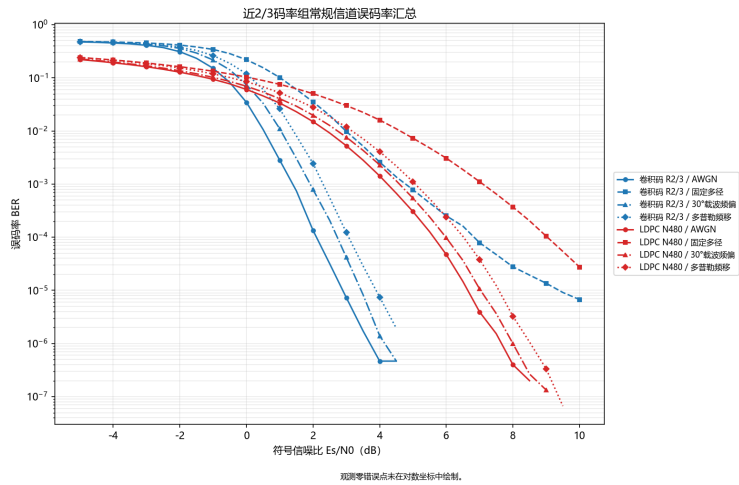


(a) 近 2/3 码率组

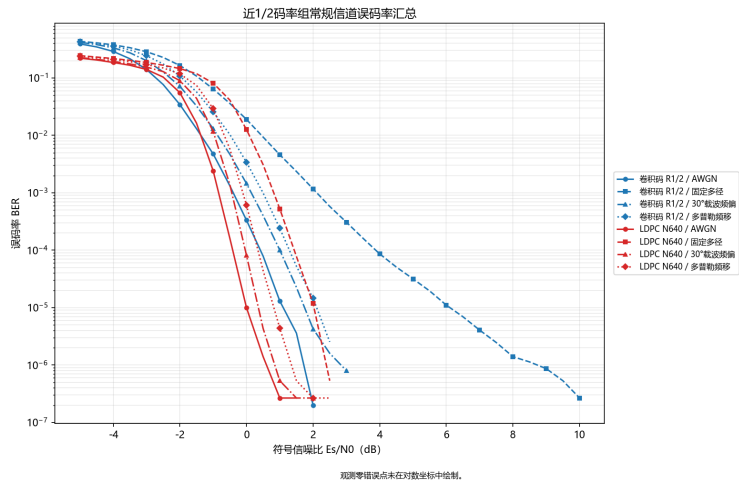


(b) 近 1/2 码率组

图 7.4: 常规信道下两组候选的 FER 汇总



(a) 近 2/3 码率组



(b) 近 1/2 码率组

图 7.5: 常规信道下两组候选的 BER 汇总

4.168/6.819 dB, LDPC N480 需要 7.597/9.529 dB; 卷积码仍保持更低绝对门限。近 1/2 组中, 卷积码需要 1.699/3.871 dB, LDPC N640 只需 0.288/1.040 dB, N640 保持更低门限。

与各自 AWGN 比较时, 近 2/3 组卷积码的附加损失为 3.256/4.962 dB, N480 为 2.926/3.502 dB; N480 相对自身基线的退化少 0.330 dB 和 1.459 dB。近 1/2 组卷积码损失为 2.423/3.628 dB, N640 为 1.528/1.722 dB, N640 少 0.895 dB 和 1.905 dB。由此可见, 卷积码 R2/3 在多径中的绝对 FER 仍好于 N480, 但 N480 从自身 AWGN 基线出发的相对退化更小; 绝对性能与相对鲁棒性的判断确实可能不同。

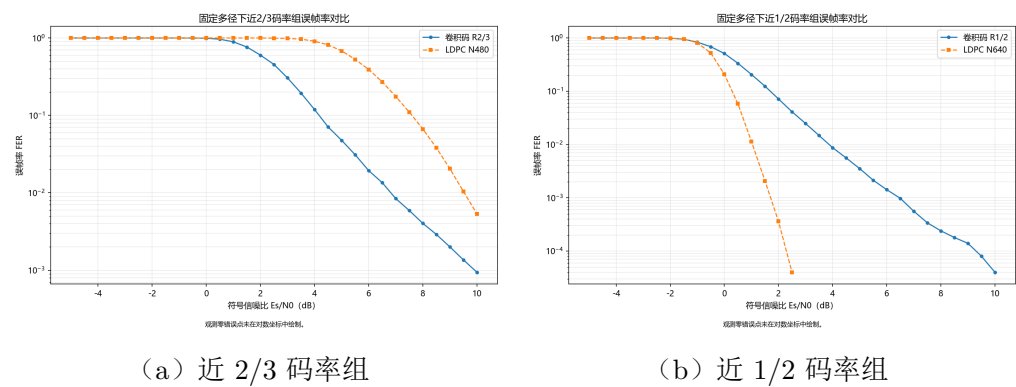


图 7.6: 固定多径与实轴 MMSE 接收处理下的误帧性能

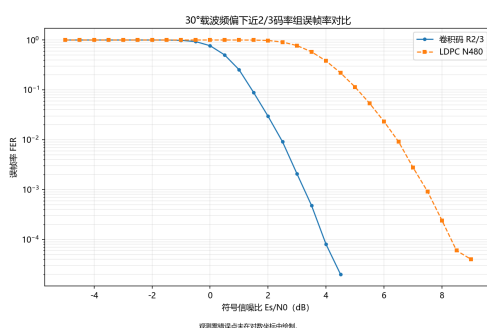
多径曲线反映信道、已知抽头实轴 MMSE 处理与译码器的联合系统性能。MMSE 矩阵同时取决于归一化抽头、码字长度和噪声方差, 均衡后的等效增益与方差又参与 LLR 形成。在当前已知抽头条件下, LDPC 两种长度的相对损失均较小; 该结论限于冻结的抽头和接收机配置, 不覆盖未知时变多径或其他均衡器。

7.4.2 固定频偏信道

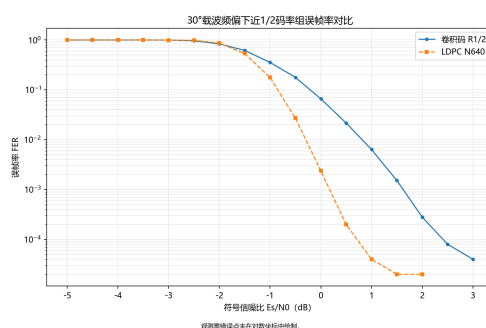
固定频偏使码字内相位从 0° 连续漂移到 30°。图??中, 近 2/3 组卷积码与 N480 的 FER= 0.1 门限为 1.445 dB 和 5.089 dB, FER= 0.01 门限为

2.459 dB 和 6.454 dB；近 1/2 组卷积码与 N640 对应为 $-0.208/-0.848$ dB 和 $0.815/-0.295$ dB。绝对排序与 AWGN 一致：近 2/3 组卷积码占优，近 1/2 组 N640 占优。

相对损失的量级低于固定多径。近 2/3 组卷积码在两个目标处损失 0.533 dB 和 0.603 dB，N480 为 0.418 dB 和 0.427 dB；近 1/2 组卷积码为 0.516 dB 和 0.571 dB，N640 为 0.393 dB 和 0.387 dB。LDPC 在四个对比中的附加代价低 0.115~0.184 dB，属于较小差异。



(a) 近 2/3 码率组



(b) 近 1/2 码率组

图 7.7: 帧内 $0^\circ \rightarrow 30^\circ$ 固定频偏相位漂移下的误帧性能

7.4.3 线性时变频偏模型

线性时变频偏的门限损失约为 0.74~0.90 dB，比固定频偏更大。图??中，近 2/3 组卷积码与 N480 达到 $FER=0.1$ 需要 1.806 dB 和 5.529 dB，达到 0.01 需要 2.753 dB 和 6.905 dB；近 1/2 组卷积码与 N640 对应为 $0.120/-0.502$ dB 和 $1.108/0.075$ dB。绝对排序仍与各自 AWGN 基准一致。

近 2/3 组在 $FER=0.1$ 处的相对损失为 0.894 dB 和 0.858 dB，在 0.01 处为 0.896 dB 和 0.878 dB，两者仅相差 0.036 dB 和 0.018 dB，结果接近。近 1/2 组卷积码损失 0.845/0.864 dB，N640 损失 0.739/0.758 dB，N640 低约 0.106 dB。由于瞬时频偏按符号索引变化，不同发送长度决定完整码字所经历的频偏与相位轨迹，因此该比较同时包含编码结构和发送长度因素。

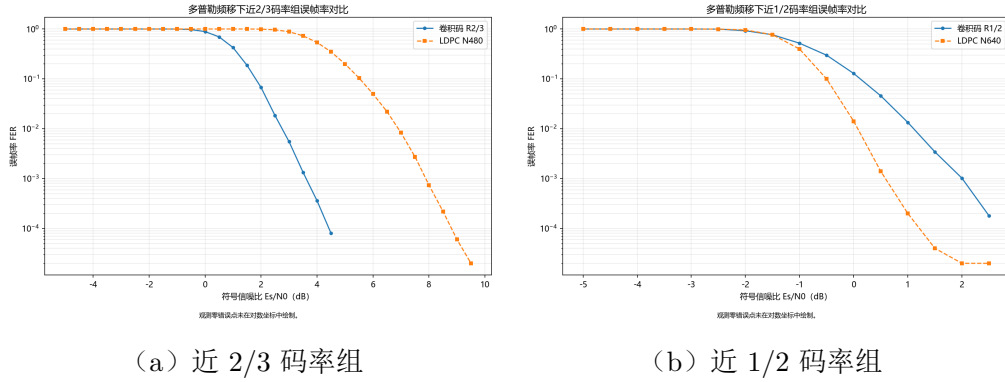


图 7.8: 线性时变频偏下的误帧性能

表??汇总六类信道的目标 FER 门限。N/A 表示在 $-5 \sim 10$ dB 正式范围内没有相邻非零 FER 实测点夹住目标值，对应目标不存在可报告的的门限估计。

7.5 连续遮挡与突发干扰条件下的可靠性

连续遮挡与突发干扰集中作用于一段连续发送符号，编码结构能否利用该区段之外的可靠观测恢复受影响信息成为关键。图??和图??分别给出 FER 与 BER。FER 衡量整帧成功率，BER 反映错误比特比例；两条曲线在局部连续非理想条件下可能呈现不同的下降速度。

7.5.1 已知 5% 短时遮挡

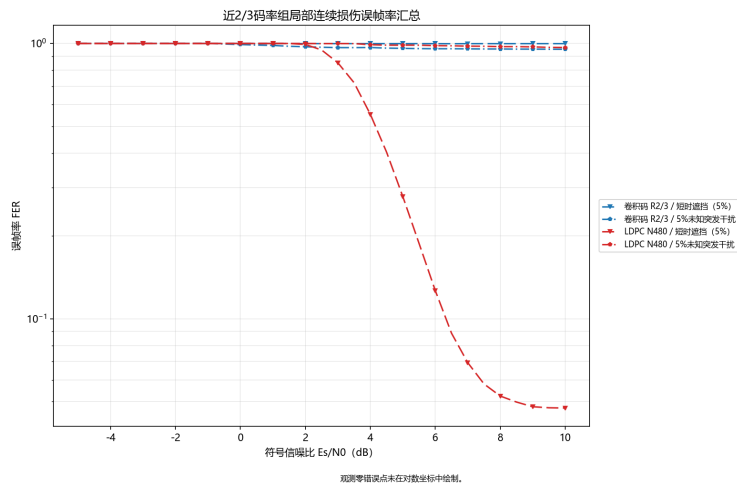
已知遮挡把连续 5% 发送符号变为擦除式观测，掩码已知且相应 LLR 置 0。两种无交织卷积码在当前正式信噪比范围内均未恢复到 $\text{FER} = 0.1$ ，因此其 $\text{FER} = 0.1/0.01$ 门限和相对 AWGN 损失均为 N/A。这里的“FER 保持较高水平”仅描述已有测点在 $-5 \sim 10$ dB 范围内的观测现象，不对应理论误差平台。

卷积码沿时间方向传播 trellis 状态，连续一段观测可靠度同时消失后，多条竞争路径在局部区间内缺少区分信息；在无交织条件下，连续遮挡不

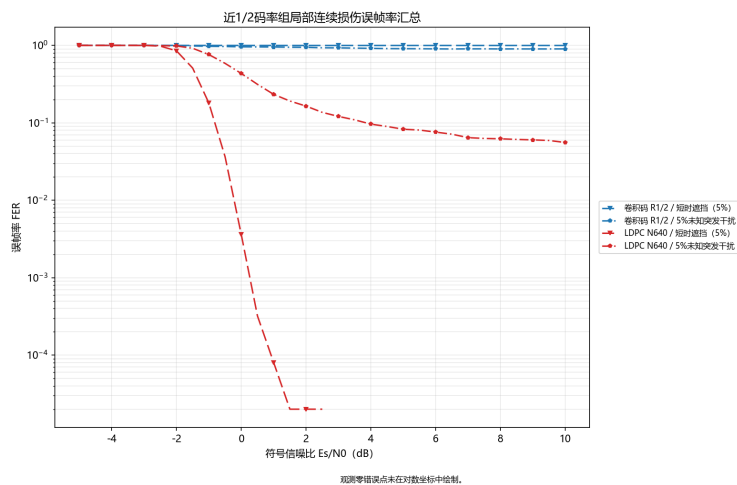
表 7.4: 六类信道目标 FER 门限估计

比较组	信道	FER= 0.1 门限/ E_s/N_0 /dB		FER= 0.01 门限/ E_s/N_0 /dB	
		卷积码	LDPC	卷积码	LDPC
近 2/3	AWGN	0.912	4.671	1.857	6.027
	固定多径	4.168	7.597	6.819	9.529
	固定频偏	1.445	5.089	2.459	6.454
	时变频偏	1.806	5.529	2.753	6.905
	已知 5% 遮挡	N/A	6.331	N/A	N/A
	未知 5% 突发	N/A	N/A	N/A	N/A
近 1/2	AWGN	-0.724	-1.241	0.244	-0.683
	固定多径	1.699	0.288	3.871	1.040
	固定频偏	-0.208	-0.848	0.815	-0.295
	时变频偏	0.120	-0.502	1.108	0.075
	已知 5% 遮挡	N/A	-0.816	N/A	-0.221
	未知 5% 突发	N/A	3.846	N/A	N/A

注：数值由夹住目标值的相邻非零 FER 实测点在 FER 对数域内插值得到；N/A 表示不具备插值覆盖条件。

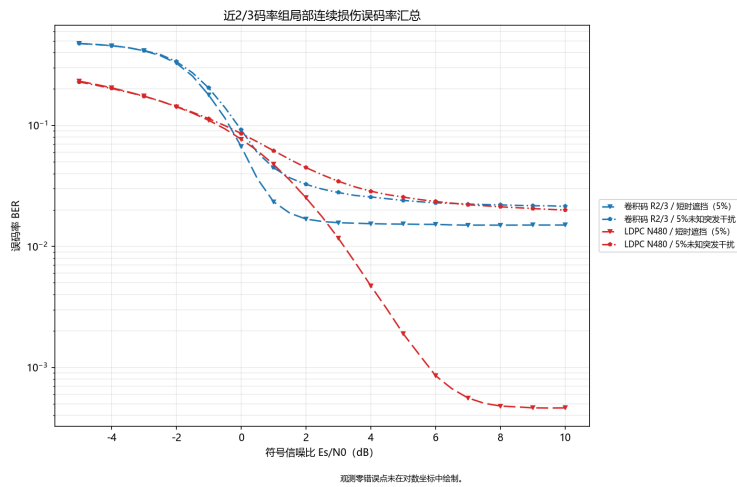


(a) 近 2/3 码率组

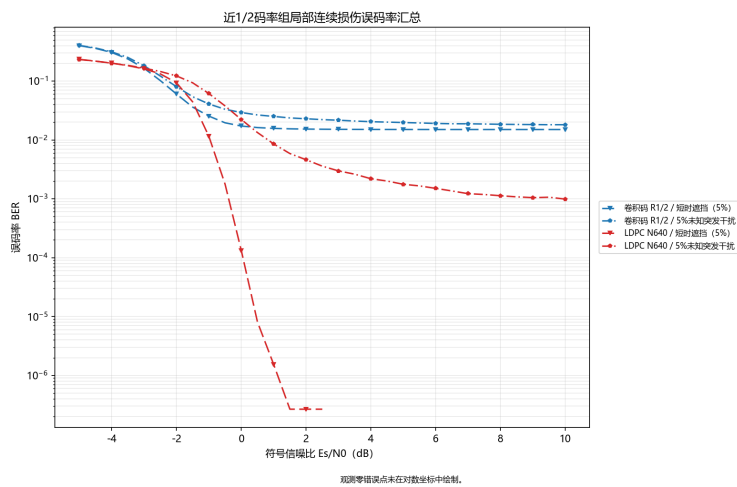


(b) 近 1/2 码率组

图 7.9: 连续遮挡与突发干扰条件下两组候选的 FER



(a) 近 2/3 码率组



(b) 近 1/2 码率组

图 7.10: 连续遮挡与突发干扰条件下两组候选的 BER

会被分散到不同译码时刻。LDPC 可利用遮挡区段之外的变量节点以及跨区段分布的校验约束进行迭代恢复。近 2/3 组 N480 在 FER= 0.1 处仍有 6.331 dB 门限, 相对 AWGN 损失为 1.659 dB, 但 FER= 0.01 未覆盖; 近 1/2 组 N640 在 0.1 和 0.01 处均可覆盖, 门限分别为 -0.816 dB 和 -0.221 dB, 相对损失分别为 0.425 dB 和 0.462 dB。该覆盖结论限于已知掩码、5% 遮挡比例和冻结配置。

7.5.2 未知 5% 突发干扰

未知突发干扰的影响不仅来自能量增大, 还来自可靠度模型失配。接收端不知道连续污染区段, 受污染样本仍按名义 AWGN 方差生成 LLR, 错误方向的高置信软信息可能进入 Viterbi 路径度量或 LDPC 消息传播。近 2/3 组两种方案在 FER= 0.1 和 0.01 处均无覆盖; 近 1/2 组只有 N640 达到 FER= 0.1, 门限为 3.846 dB, 相对自身 AWGN 损失 5.087 dB, 而 FER= 0.01 仍无覆盖。

BER 曲线表明, 提高 E_s/N_0 能够减弱背景 AWGN, 却不能消除固定 ISR 连续干扰带来的错误可靠信息, 因而高信噪比区的恢复受限。未覆盖目标记为 N/A, 不对曲线末端作门限外推。现有无交织方案在未知位置、ISR= 10 dB 的连续强干扰下可靠性受限; 突发位置检测、可靠度抑制和交织属于不同的接收机设计, 不在本章实测配置之内。

7.6 相对 AWGN 信道损失与鲁棒性

为分离方案固有 AWGN 基准与信道引入的附加代价, 定义目标 FER 为 p 时的相对 AWGN 信道损失

$$L_{\text{ch}}(p) = \gamma_{\text{ch}}(p) - \gamma_{\text{AWGN}}(p), \quad (7.3)$$

其中 $\gamma_{\text{ch}}(p)$ 和 $\gamma_{\text{AWGN}}(p)$ 分别为相应非 AWGN 信道条件与同一方案 AWGN 下达到 p 所需的 E_s/N_0 门限估计。两项均由相邻非零 FER 实测点在对数域内插值得到; 任一项没有覆盖时, L_{ch} 记为 N/A。此处鲁棒性定

义为方案从 AWGN 基线进入指定非理想信道后的性能保持能力，较小的 L_{ch} 表示在目标 FER 处所需的附加 E_s/N_0 较低。

表 7.5: 相对各方案自身 AWGN 基线的信道损失

比较组	信道	FER= 0.1 损失/dB		FER= 0.01 损失/dB	
		卷积码	LDPC	卷积码	LDPC
近 2/3	固定多径	3.256	2.926	4.962	3.502
	固定频偏	0.533	0.418	0.603	0.427
	时变频偏	0.894	0.858	0.896	0.878
	已知 5% 遮挡	N/A	1.659	N/A	N/A
	未知 5% 突发	N/A	N/A	N/A	N/A
近 1/2	固定多径	2.423	1.528	3.628	1.722
	固定频偏	0.516	0.393	0.571	0.387
	时变频偏	0.845	0.739	0.864	0.758
	已知 5% 遮挡	N/A	0.425	N/A	0.462
	未知 5% 突发	N/A	5.087	N/A	N/A

注： L_{ch} = 相应非 AWGN 信道条件下的目标 FER 门限 - 同一方案 AWGN 目标 FER 门限；
N/A 表示目标未被实测范围覆盖，未作外推。

表??显示，三类全帧非理想条件中固定多径的附加代价最大，固定频偏最小，线性时变频偏居中。两组 LDPC 在固定多径下的损失均低于对应卷积码；在 FER= 0.01 时，N480 和 N640 分别低 1.459 dB 和 1.905 dB。固定频偏下 LDPC 损失低 0.115~0.184 dB；线性时变频偏下近 2/3 组差异仅 0.018~0.036 dB，近 1/2 组约 0.106 dB。这些量级分别对应接近、较小差异和较低附加代价，不构成跨指标的统一排序。

已知遮挡与未知突发体现出不同的覆盖边界。已知遮挡中，N640 的相对损失不足 0.5 dB，而卷积码没有目标覆盖；未知突发中，即使 N640 在

FER=0.1 处可估计门限, 损失也升至 5.087 dB, 其余多数目标均为 N/A。此处 N/A 本身就是重要结果: 它表明在当前信噪比范围和接收假设下, 目标可靠性没有得到测量支持。

近 2/3 固定多径中, 卷积码的绝对 FER 门限低于 N480, 而 N480 相对自身 AWGN 的附加损失更小。绝对可靠性与相对 AWGN 基线的性能保持能力在该条件下给出不同侧面, 因而本章不采用单一标量总分。

7.7 软件译码时间与 LDPC 迭代特性

软件译码时间从 LLR 或软信息进入译码函数开始, 到 300 bit 电文及译码状态输出为止。它包括卷积码去打孔与 Viterbi 处理, 或 LDPC 分层 NMS 迭代, 但不包括编码、信道扰动构造、AWGN 叠加、均衡、实轴投影、LLR 生成和文件 I/O。接收算法时间则定义为信道处理时间与译码时间之和。表??列出的平均译码时间和 P95 译码时间均先在每个正式信噪比点内统计, 再对 31 个 E_s/N_0 工作点等权取算术平均, 并非将不同工作点的全部帧直接池化; 最大值属于有限样本中的最大观测软件译码时间, 不是严格最坏情况时延。

近 2/3 组中, N480 在六类信道下的平均译码时间均低于卷积码 R2/3: AWGN 下为 113.4 μs , 相对 307.7 μs ; 未知突发下迭代负担升高, 平均时间增至 209.0 μs , 但仍低于卷积码的 318.8 μs 。该结果表明“迭代译码必然比 Viterbi 慢”的预设并不成立, 实际软件时间还受校验图规模、提前停止和实现访存模式影响。

近 1/2 组中, N640 的平均译码时间高于卷积码 R1/2, AWGN 下为 364.7 μs 对 323.2 μs , 固定多径下为 496.9 μs 对 327.1 μs , 未知突发下达到 596.1 μs 且平均 P95 为 1407.9 μs 。N640 提高可靠性的同时增加了软件处理代价, 未知突发条件下的平均 P95 比 AWGN 增加 837.9 μs 。多径接收算法时间还包含 MMSE 均衡, 因此两种方案均高于相应译码时间。

LDPC 迭代统计覆盖所有帧, 不仅统计成功译码帧。表??中的平均迭代、P95 迭代和上限迭代帧比例均先在每个正式信噪比点内统计, 再对 31

表 7.6: 六类信道下的软件译码时间统计

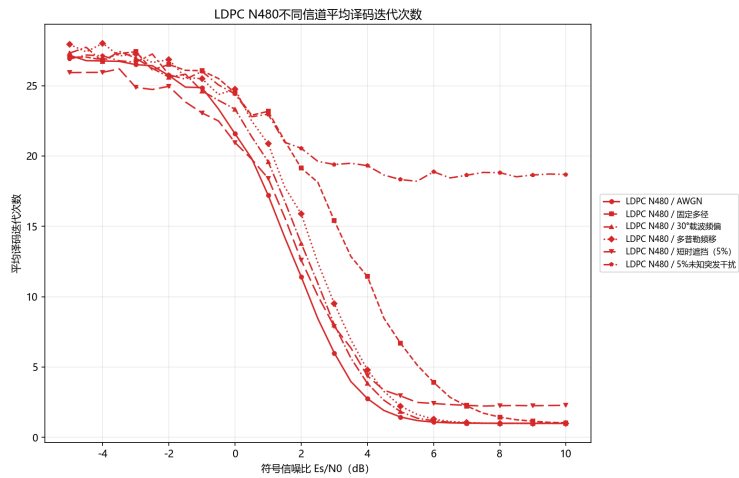
比较组	信道	平均译码/ μs		平均 P95 译码/ μs		最大观测译码/ μs	
		卷积码	LDPC	卷积码	LDPC	卷积码	LDPC
近 2/3	AWGN	307.7	113.4	419.1	232.3	35726.7	12396.6
	固定多径	317.1	148.8	441.1	317.2	5608.1	5199.8
	固定频偏	313.7	122.1	434.9	253.6	59356.8	14438.7
	时变频偏	341.5	149.0	483.1	299.0	59989.8	13634.0
	已知遮挡	313.3	124.0	434.3	274.0	1965.7	1692.7
	未知突发	318.8	209.0	442.2	422.0	1807.0	2760.5
近 1/2	AWGN	323.2	364.7	448.5	570.0	36699.3	42686.8
	固定多径	327.1	496.9	461.2	782.6	13418.2	43364.4
	固定频偏	317.5	417.1	441.9	633.7	38679.0	18152.7
	时变频偏	332.1	435.3	466.2	681.4	34330.9	39922.8
	已知遮挡	313.6	382.1	436.1	592.9	8571.6	3869.7
	未知突发	323.6	596.1	449.5	1407.9	9167.5	5648.2

表 7.6: 六类信道下的软件译码时间统计（续：平均接收算法时间）

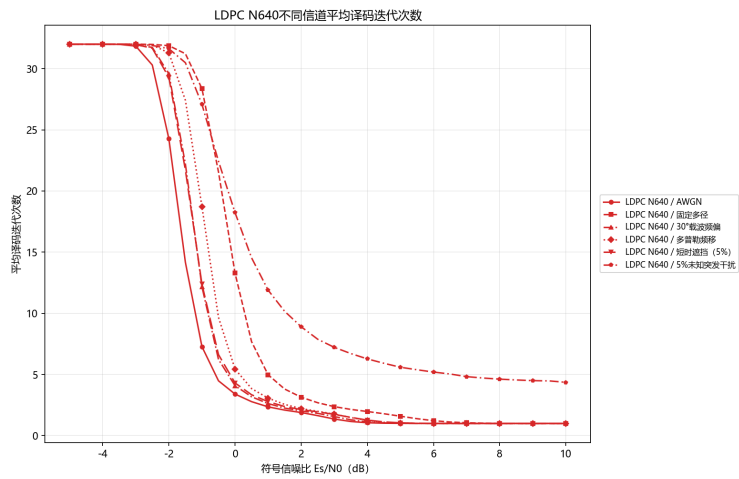
比较组	信道	卷积码/ μs	LDPC/ μs
近 2/3	AWGN	339.1	144.9
	固定多径	496.3	348.5
	固定频偏	349.2	157.7
	时变频偏	380.4	188.0
	已知遮挡	345.9	156.7
	未知突发	382.0	273.1
近 1/2	AWGN	366.4	408.0
	固定多径	676.2	904.6
	固定频偏	364.4	464.5
	时变频偏	381.3	485.0
	已知遮挡	356.0	424.9
	未知突发	407.2	681.4

个 E_s/N_0 工作点等权取算术平均；其中平均迭代描述总体计算负担，平均 P95 迭代表示各点尾部迭代值的等权均值，达到 32 次上限的平均帧比例反映迭代耗尽程度。图??和表??显示，未知突发使 N480 和 N640 的平均迭代升至 21.840 和 14.662 次，平均 P95 均为 32 次，上限迭代帧比例分别为 65.75% 和 38.42%，对应 AWGN 下的数值为 35.13% 和 21.95%。

达到 32 次迭代并不等同于必然帧错误，综合校验通过也不等同于存在 CRC 保护。本比较中的译码失败定义为最终综合校验未通过；若综合校验通过而 300 bit 电文仍错误，则记录为错误合法码字，不应描述成 CRC 未检错。这里的 LDPC 是 BG2 Direct QC-LDPC 短块构造，不代表完整 5G NR 物理层链路。



(a) LDPC N480



(b) LDPC N640

图 7.11: LDPC 在六类信道下的平均迭代次数

表 7.7: LDPC 在六类信道下的迭代行为

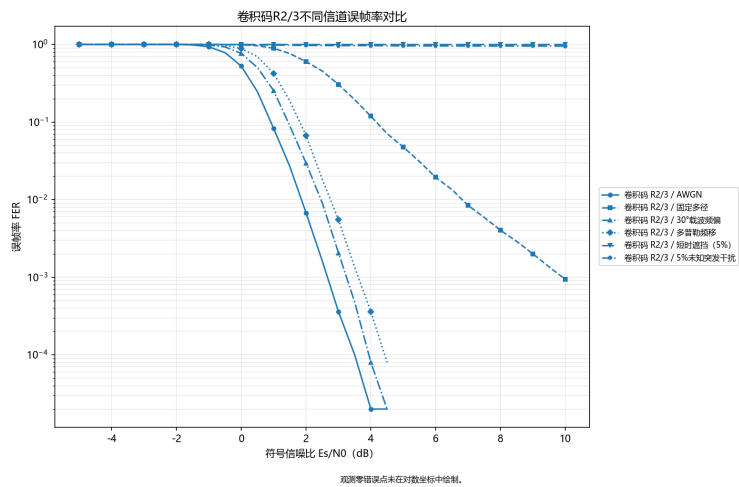
方案	信道	平均迭代	平均 P95 迭代	平均上限迭代帧比例/%
N480	AWGN	12.202	19.129	35.13
	固定多径	15.208	25.129	44.56
	固定频偏	12.930	20.129	37.46
	时变频偏	13.508	21.065	39.36
	已知遮挡	12.689	22.323	36.53
	未知突发	21.840	32.000	65.75
N640	AWGN	8.712	10.452	21.95
	固定多径	11.655	14.000	30.86
	固定频偏	9.481	11.032	24.50
	时变频偏	10.153	11.871	26.58
	已知遮挡	9.524	11.161	24.42
	未知突发	14.662	32.000	38.42

7.8 多信道综合比较与高速电文方案结论

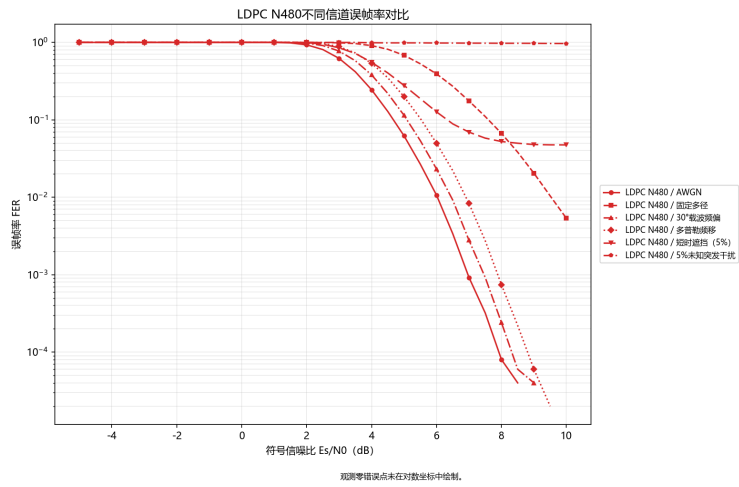
图??和图??从单一方案视角比较六类信道。近 2/3 组中，卷积码 R2/3 在 AWGN、固定多径、固定频偏和线性时变频偏下保持更低绝对 FER 门限，同时其平均软件译码时间约为 308~341 μ s；N480 的绝对门限较高，但平均译码时间更低，并在已知遮挡下保留 FER= 0.1 覆盖。近 1/2 组中，N640 在四类全帧条件和已知遮挡中均具有更低绝对门限，但平均译码和尾部时间高于卷积码 R1/2。

近 2/3 组中，卷积码 R2/3 在 AWGN、固定多径、固定频偏和线性时变频偏下具有较低绝对 FER 门限；N480 在已知遮挡下保留 FER= 0.1 覆盖，且六类信道的平均软件译码时间均低于卷积码 R2/3。近 1/2 组中，N640 在 AWGN、三类全帧非理想条件和已知遮挡下具有较低绝对门限，但多种条件下的平均、P95 译码时间和迭代负担高于卷积码 R1/2。

未知 5% 突发干扰对两组候选均造成较大可靠性退化。近 2/3 组两种方案均未覆盖 FER= 0.1/0.01，对应门限与相对 AWGN 损失均为 N/A；近 1/2 组仅 N640 覆盖 FER= 0.1，相对损失为 5.087 dB，而 N640 的 FER= 0.01 及卷积码 R1/2 的两个目标均未覆盖。在无交织、未知掩码和 ISR= 10 dB 的冻结配置下，连续强突发干扰构成两组方案的共同限制。

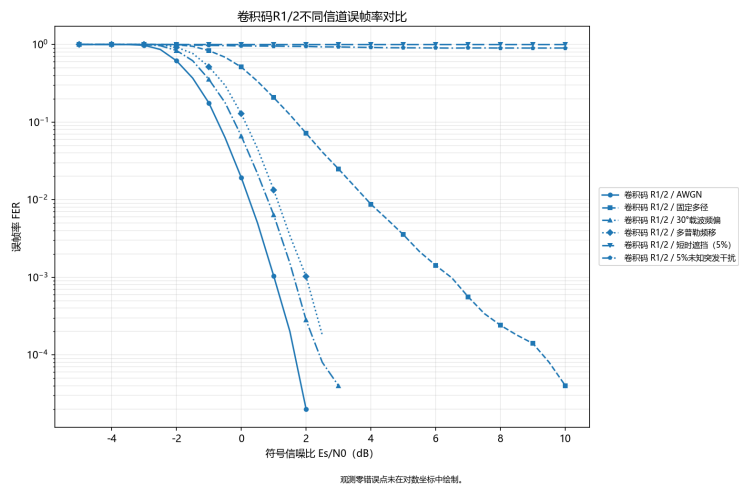


(a) 卷积码 R2/3

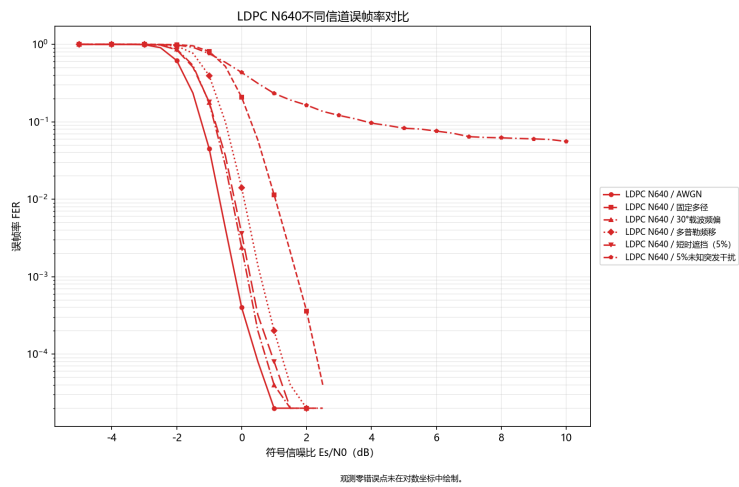


(b) LDPC N480

图 7.12: 近 2/3 码率组单方案跨六信道 FER



(a) 卷积码 R1/2



(b) LDPC N640

图 7.13: 近 1/2 码率组单方案跨六信道 FER

表 7.8: 六信道综合比较结论

比较组	信道	比较结论	实测依据与适用边界
近 2/3	AWGN	卷积码 R2/3 绝对门限较低	两个目标 FER 门限分别低 3.759 dB 和 4.170 dB
	固定多径	卷积码 R2/3 绝对门限较低	N480 相对损失较小且平均软件译码时间较低，绝对门限与基线相对保持能力不一致
	固定频偏	卷积码 R2/3 绝对门限较低	N480 附加损失仅低约 0.12~0.18 dB
	时变频偏	卷积码 R2/3 绝对门限较低	两者相对损失接近；结果包含不同发送长度造成的相位轨迹差异
	已知 5% 遮挡	LDPC N480 保留有限覆盖	卷积码无目标覆盖；N480 覆盖 FER= 0.1 且平均软件译码时间较低
	未知 5% 突发	未形成明确优选方案	两者均未覆盖 FER= 0.1；正式数据不支持基于目标 FER 门限形成单一方案结论
近 1/2	AWGN	LDPC N640 绝对门限较低	两个目标 FER 门限低 0.517 dB 和 0.926 dB，平均译码时间高约 41.5 μ s
	固定多径	LDPC N640 门限与相对损失均较低	接收算法时间高于卷积码 R1/2
	固定频偏	LDPC N640 绝对门限与附加损失均较低	平均软件译码时间高于卷积码 R1/2
	时变频偏	LDPC N640 绝对门限与相对损失均较低	结论限于冻结轨迹与发送长度
	已知 5% 遮挡	LDPC N640 覆盖两个目标	卷积码均未覆盖；结论限于已知掩码、5% 遮挡比例和冻结配置
	未知 5% 突发	LDPC N640 仅有有限覆盖	仅覆盖 FER= 0.1 且损失 5.087 dB；P95 译码时间和迭代负担上升



图 7.14: 高速电文多信道编码方案选型逻辑

综合结果形成两类清晰的场景分布：近 $2/3$ 组的卷积码 R2/3 在四类全帧条件下具有较低绝对门限，N480 在已知遮挡下保留目标覆盖且平均软件译码时间较低；近 $1/2$ 组的 N640 在五类已覆盖条件下具有较低绝对门限，同时承担较高的软件时间和迭代代价。未知突发条件下的大量 N/A 保留了实测证据边界，不产生超出冻结配置的方案结论。

第八章 译码算法对比与实现代价分析

8.1 对比目标与统一仿真条件

8.1.1 S6 任务目标

S6 在前序编码方案已经确定的基础上，固定业务输入、编码配置和加性高斯白噪声（Additive White Gaussian Noise, AWGN）条件，在各编码族内部更换译码实现，比较译码器对可靠性与实现代价的共同影响。可靠性采用比特错误率（Bit Error Rate, BER）和帧错误率（Frame Error Rate, FER）表征；实现代价分为算法操作规模、译码器存储开销和当前软件平台上的实测译码时间。平均译码时间与最大观测译码时间是本章的主要时间指标，P95、P99、迭代次数、提前停止率、首输出时延和决策时延用于补充描述尾部波动、收敛行为与流式输出特性。

表??给出三类比较的控制边界。卷积码硬、软判决是在编码器、码率和码长不变时更换分支度量输入，因而最接近单一译码因素对比；LDPC 的 BP 与归一化最小和（Normalized Min-Sum, NMS）共享逐帧输入及噪声，实现了严格配对；BCH 两条路线同时改变码型、发送长度、纠错能力、分块组织和译码器，因此只能解释为两套完整 BCH 实现方案的综合比较。

表 8.1: S6 译码算法对比的控制变量与变化变量

编码类型	冻结条件	变化条件	对比对象	主要指标	可比性说明
BCH	200 bit 业务输入、码型、分块、发送长度、BCH-S200 与 BCH-BPSK、AWGN、硬判决与统计规则	纠错能力及译码器	B200	BER、FER、操作、内存、时间	完整方案对比，不能作纯译码因果归因
卷积码	300 bit 输入、 $R = 300/612$ 、相同网格及 Viterbi 骨架	硬判决或浮点软判决；8 种组织与判决组合 整块或连续组织		BER、FER、ACS、回溯、内存、时间与输出时延	数据复用自既有正式试验，非 S6 strict pair-stop 重跑
LDPC	同一 300 bit 载荷、校验节点更新算法 N560 码字、LLR、图结构、迭代上限与停止规则		分层 BP 与分层 NMS	BER、FER、迭代、操作、内存、时间	逐信噪比共享载荷、码字和 LLR

8.1.2 统一仿真条件

三类结果均采用幅度为 ± 1 的二进制相移键控（Binary Phase Shift Keying, BPSK），以符号信噪比 E_s/N_0 作为主横轴。实值 AWGN 方差为

$$\sigma^2 = \frac{1}{2 \times 10^{(E_s/N_0)/10}}. \tag{8.1}$$

由码率换算的 E_b/N_0 只用于模块内部辅助记录，不能替代本章图中的 E_s/N_0 口径。各工作点从 -5 dB 到 10 dB，步长为 0.5 dB。每点至少运行 1000 帧，达到目标 200 个错误帧后可停止，最多运行 50000 帧。高信噪比处未观测到错误的原始零值仍保留在数据中，但不在对数坐标上伪造非零点，也不参与 FER 门限外推。

8.1.3 复杂度与时延定义

本章把实现代价拆分为三个互不替代的维度。算法操作规模记录译码过程中发生的结构化事件，适合在同一编码族和同一计数口径内比较；存储开销记录由类型和缓冲区规模推导的译码器内存模型，不是完整进程驻留集；软件译码时间是当前 C++ 实现、编译配置和处理器上的墙钟测量

表 8.2: S6 译码算法比较的公共仿真条件

项目	设置
调制与信道	BPSK；实值 AWGN
主信噪比口径	E_s/N_0 ，范围 $-5 \sim 10$ dB，步长 0.5 dB，共 31 点
停止条件	每点至少 1000 帧；目标错误帧数 200；每点最多 50000 帧
随机与噪声规则	按帧索引确定性生成信息与标准高斯母噪声；同一编码族的对比译码器共享规定输入。BCH 全局种子为 20260804；卷积码载荷与噪声种子分别为 2026072001 和 2026072904
可靠性判据	恢复载荷逐比特比较；任一载荷比特错误即记为错误帧
计时单位	微秒/帧；计时范围仅覆盖相应 C++ 译码调用
已记录运行环境	Windows 11、Intel Core i5-14400F、GNU C++ 15.2.0、C++17、Release、单线程、 <code>steady_clock</code>

值，不代表硬件绝对时延。BCH 的异或、有限域运算，卷积码的加比选 (Add-Compare-Select, ACS)，以及 LDPC 的双曲函数、比较和最小值操作具有不同语义与成本，不能简单相加成跨编码族的“统一总复杂度”。

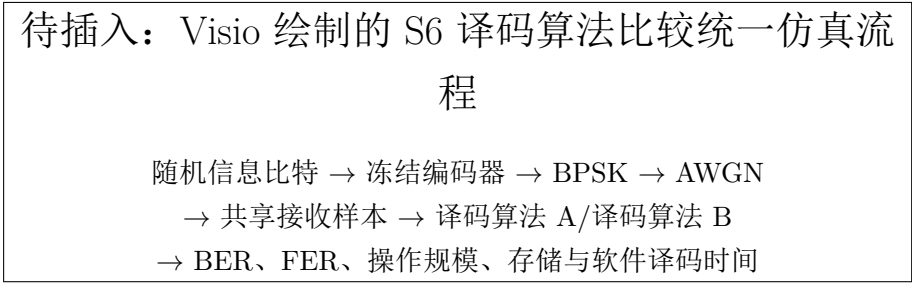


图 8.1: S6 译码算法比较统一仿真流程

表 8.3: 本章译码复杂度与时延指标定义

维度	指标与单位	统计含义	使用限制
算法操作规模	BCH 综合征、查表、BM、由译码源码中的结构化计数器累计，不同操作不能按一次等 Chien 和有限域事件；卷积 同族内反映计算结构与工作量 码 ACS 与回溯；LDPC 消息更新及节点运算，单位为事件/帧		价成本跨族相加
存储开销	译码器内存与峰值工作区，静态表、对象、消息、路径度量、回溯或工作缓冲的模型值 字节		不含完整进程 RSS，静态表与临时工作区须分开解释
平均译码时间	每点平均值及跨信噪比帧 加权平均值， $\mu\text{s}/\text{帧}$	计时器包围译码调用；汇总值按各点 帧数加权	依赖处理器、编译器、实现和运行状态
尾部与最大时间	P95、P99 和最大观测值， $\mu\text{s}/\text{帧}$	P95/P99 来自相应正式图数据；最大值为全部工作点的观测最大值	最大值反映观测尖峰，不等同于硬实时上界
流式输出时延	首输出和决策时延，符号	由在线调度和窗口结构决定	不是 CPU 译码时间，不能代替平均或最大墙钟时间

8.2 BCH 不同译码实现方案对比

8.2.1 比较对象与公平性边界

BCH-S200 把 200 bit 载荷补入 9 个零后分为 19 组，每组采用 BCH(15,11,1)，实际发送 285 bit，码率为 $200/285 = 0.7018$ ，译码器按综合征查询单错位置表。BCH-B200 采用缩短 BCH(255,207)，从母码信息区缩短 7 bit，实际发送 248 bit，码率为 $200/248 = 0.8065$ ，保证纠错能力 $t = 6$ ，译码流程由综合征求值、Berlekamp–Massey（BM）迭代和 Chien 搜索组成。

表??明确了因果边界：两种方案除了译码方法，还同时改变了码型、分组数、发送长度、码率和纠错能力。因此，以下 BER 与 FER 差异只能归属于本项目既定的两类完整 BCH 编译码实现，不能表述为“BM 本身必然优于查表”。

表 8.4: BCH 两类译码实现及比较边界

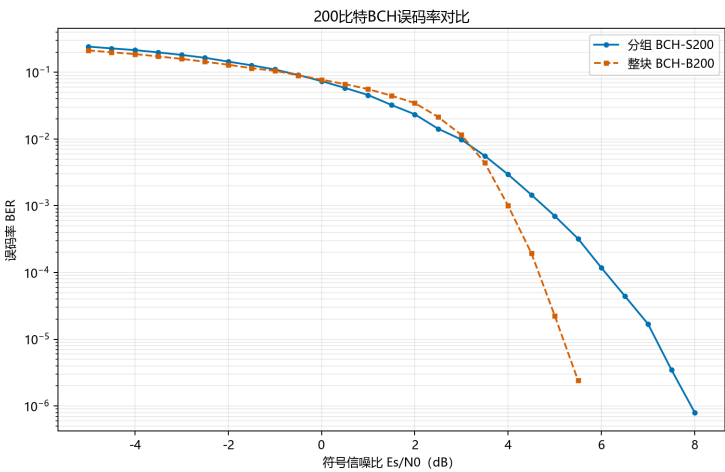
方案	载荷	BCH 码型	分组数	发送长度	码率	纠错能力	译码器	比较说明
BCH-S200	200	BCH(15,11,1)	19	285	0.7018	每组 1 bit	综合征查表	末组填充 9 bit, 短分组固定结构
BCH-B200	200	缩短 BCH(255,207)	短 1	248	0.8065	整块 6 bit	BM 与 Chien 搜索	缩短 7 bit, 整块代数译码

8.2.2 BER 与 FER

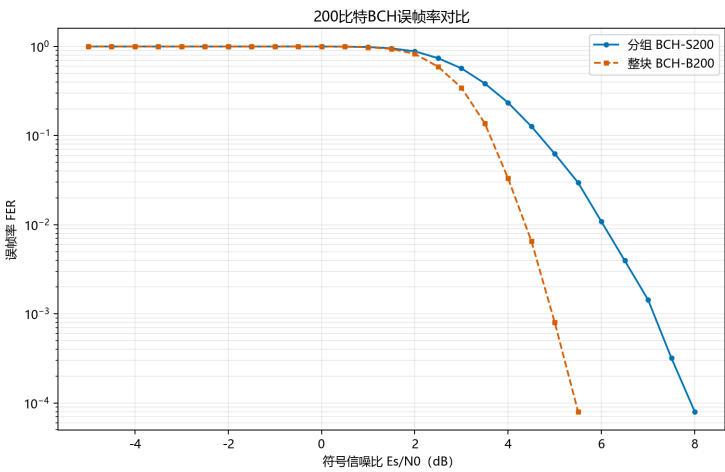
图??表明, 两种方案在低 E_s/N_0 区均处于高错误率区, 进入瀑布区后 BCH-B200 的 BER 和 FER 曲线更早下降。依据相邻真实非零 FER 点的 $\log_{10}(\text{FER})$ 线性插值, FER 为 0.1 时, BCH-S200 与 BCH-B200 所需 E_s/N_0 分别为 4.667 dB 和 3.612 dB, 完整整块方案节省 1.056 dB; FER 为 0.01 时, 两者分别为 6.040 dB 和 4.369 dB, 差值增至 1.671 dB。高信噪比处的零错误点仅表示有限样本中未观测到错误: S200 有 4 个零 BER/FER 点, B200 有 9 个, 图中没有对这些点进行对数伪值替换或外推。

8.2.3 计算结构与操作统计

BCH-S200 对 19 个短子块计算综合征; 综合征非零时执行位测试、查表、候选比特翻转和译后检查。图??显示, 其网格均值为每帧 26.92 次综合征计算、296.16 次综合征位测试和 7.92 次查表; 后三类纠错相关操作会随信道改善而减少。BCH-B200 每帧固定完成 3060 次综合征求值, 随后执行 BM 迭代和 Chien 位置测试, 网格均值分别为 8.64 次和 183.07 次, 同时伴随约 4117.52 次有限域加法、1143.42 次有限域乘法和 3.64 次有限域除法。该结构解释了整块方案更重的有限域计算负担, 但这些事件不是等价指令, 不能用计数总和直接换算处理器周期。

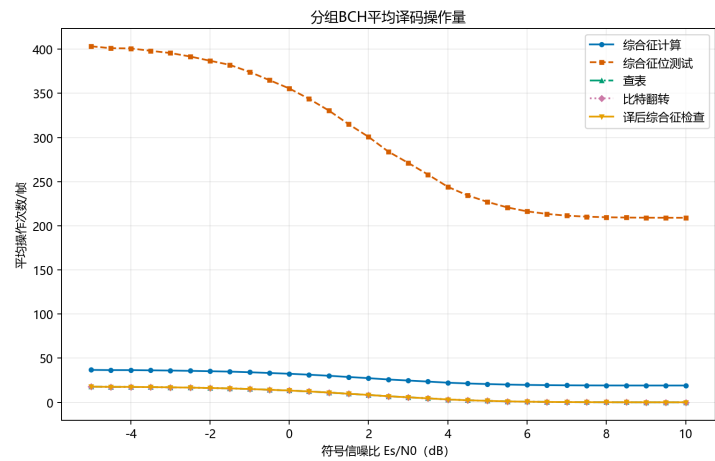


(a) 比特错误率

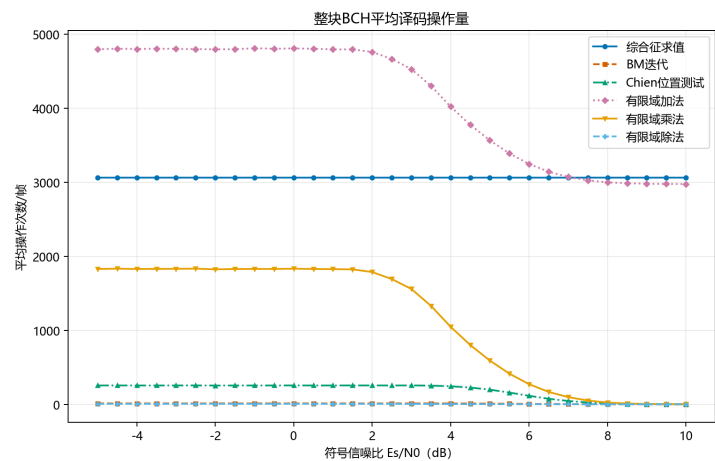


(b) 帧错误率

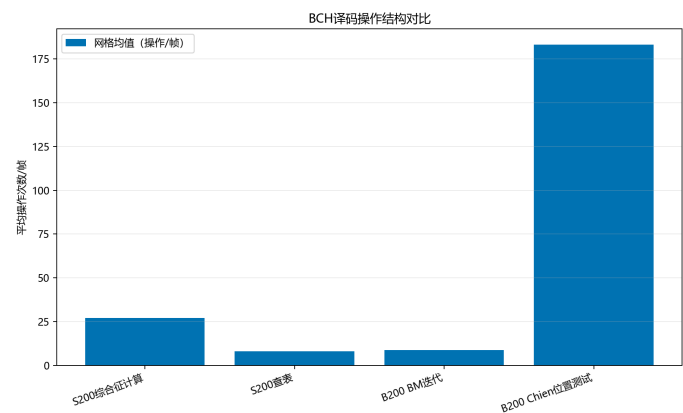
图 8.2: BCH 两类实现的 BER 与 FER



(a) 分组查表译码操作



(b) 整块代数译码操作



(c) 主要操作结构的网格均值

图 8.3: BCH 译码计算结构与操作统计

8.2.4 存储与软件译码时间

图??中的译码器内存由静态表、对象、输入输出缓冲和工作区按类型与数量建模。BCH-S200 总内存为 1580 byte, 峰值工作区 779 byte; BCH-B200 总内存为 2408 byte, 峰值工作区 810 byte。整块方案总内存增加 828 byte, 即 52.4%, 主要来自有限域表和代数译码状态; 峰值工作区仅增加 31 byte。该口径不包含完整进程驻留内存, 静态表与临时工作区也未混为同一概念。

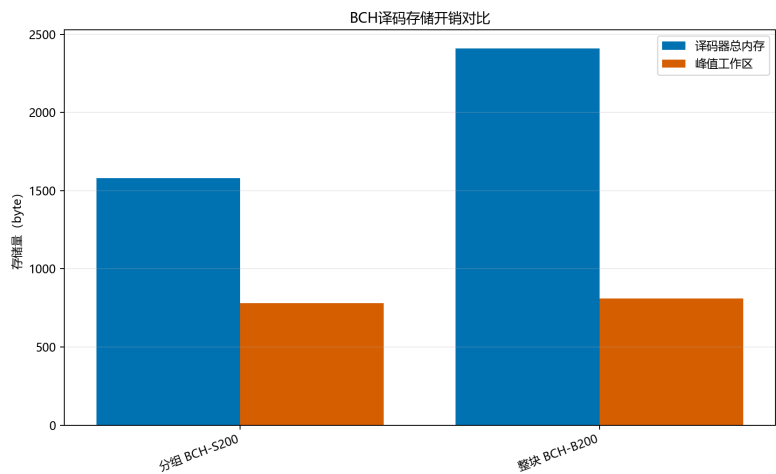


图 8.4: BCH 译码器存储开销

图??覆盖平均、P95、P99 和最大观测软件译码时间。按全部信噪比点的帧数加权, S200 与 B200 的平均译码时间分别为 10.217 μ s 和 17.415 μ s, 整块方案增加 70.5%; 最大观测值分别为 25240.3 μ s 和 27491.4 μ s, 增加 8.9%。逐点 P95 范围分别为 10.90–26.53 μ s 和 15.40–33.52 μ s。P95 和 P99 来自对应正式图数据, 而不是不含这两列的汇总表。最大观测值受操作系统调度等影响, 只能用于描述当前测量中的尖峰, 不能解释为硬实时上界。

8.2.5 BCH 工程结论

在本项目既定实现中, BCH-S200 的短分组、固定查表和较小工作区形成了更低的软件时间与更简单的实现结构, 适合资源受限且允许较高链

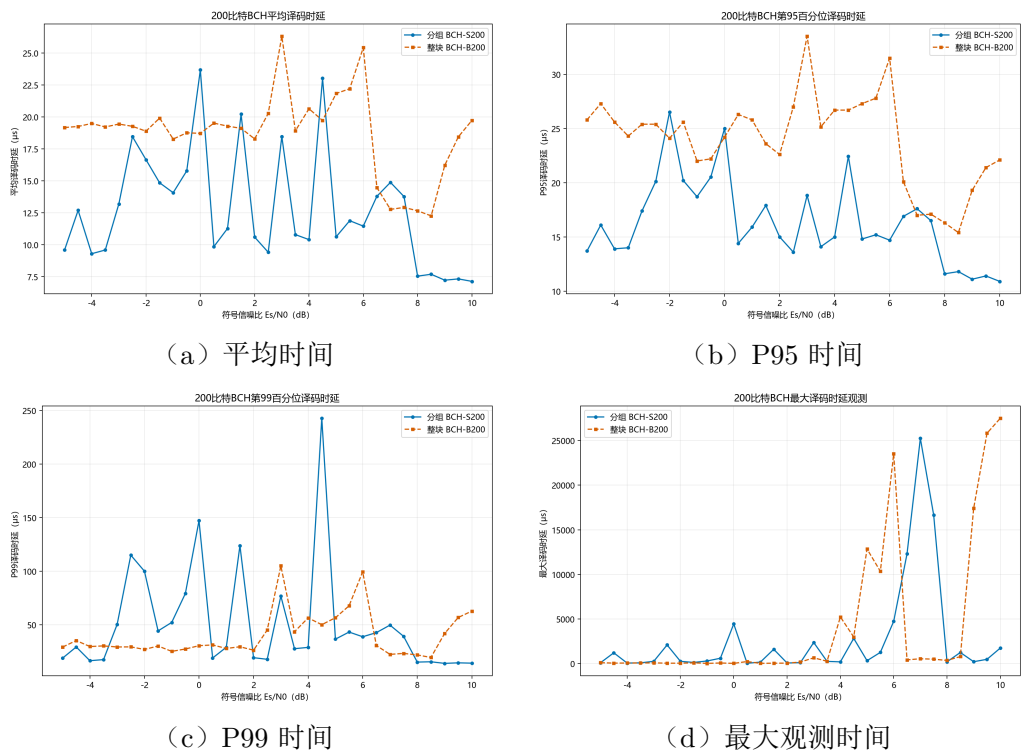


图 8.5: BCH 译码时间统计

表 8.5: BCH 可靠性、资源与软件时间汇总

方案	FER=0.1/dB	FER=0.01/dB	加权平均/ μs	最大观测/ μs	内存/byte	主要计算结构
BCH-S200	4.667	6.040	10.217	25240.3	1580	19 组综合征、查表、翻转和译后检查
BCH-B200	3.612	4.369	17.415	27491.4	2408	综合征、BM、Chien 及有限域运算

路信噪比的场景。BCH-B200 以 BM、Chien 搜索和有限域表开销换取整块 $t = 6$ 纠错能力, 在 FER 为 0.01 时比 S200 节省 1.671 dB, 同时平均软件译码时间增加约 70.5%、模型内存增加 52.4%。该权衡属于完整编码组织和译码结构共同变化的结果, 不应归因于单一译码算法。

8.3 卷积码硬判决与软判决 Viterbi 对比

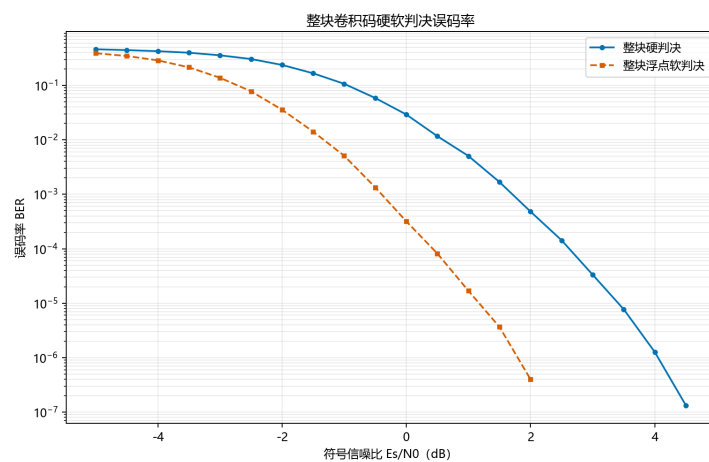
8.3.1 正式配置与判决定义

卷积码使用 300 bit 载荷和 6 个终止比特, $R = 1/2$ 不打孔, 实际发送 612 bit, 实际码率为 0.4902。整块方案一次处理 306 个网格输入步, $D = W = 306$ 、 $S = 300$; 连续方案按 50×6 、 100×3 和 150×2 三种业务组织输入, 均使用回溯深度 $D = 70$ 、窗口长度 $W = 128$ 和滑动步长 $S = 25$ 。硬判决把接收样值阈值化为 0/1 比特, 以汉明距离构造分支度量; 浮点软判决保留连续 BPSK 接收符号, 以接收符号与理想 ± 1 符号之间的欧氏距离平方构造分支度量, 并非 LLR 输入。

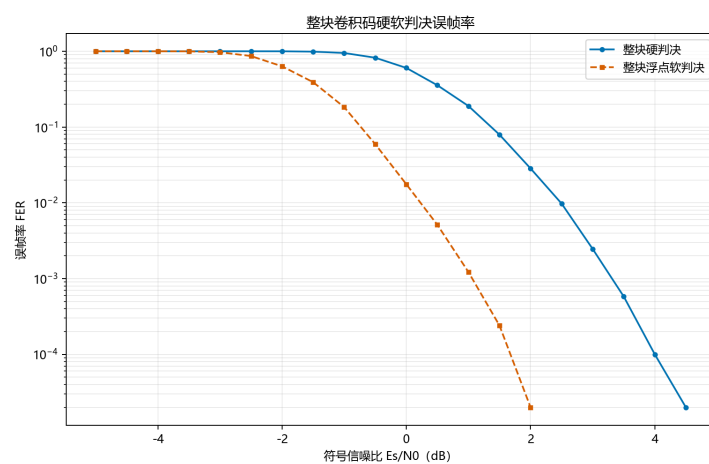
卷积码证据复用 S3 Stage14 的正式仿真数据并在 S6 中统一整理, S6 没有按相同停止时刻重新执行逐帧 strict pair-stop。因此, 本节依据相同配置、信噪比网格和正式统计曲线比较硬、软判决差异, 不进行逐帧差分因果推断。该限制不否定已有统计结果, 但限定了结论的精度和表达范围。

8.3.2 BER、FER 与门限差异

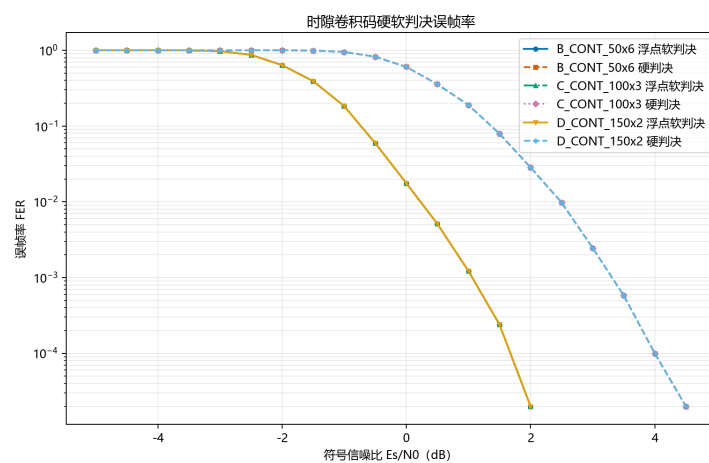
图??显示, 整块和三种连续组织下, 浮点软判决均比硬判决更早进入低错误率区。整块方案在 FER 为 0.1 时, 软、硬判决门限分别为 -0.731 dB 和 1.367 dB, 软判决节省 2.098 dB; 在 FER 为 0.01 时, 两者分别为 0.229 dB 和 2.489 dB, 差值为 2.260 dB。三种连续组织的 FER 曲线及门限与整块统计一致, 说明在当前终止、窗口和输出组织下, 可靠性差异主要由判决信息精度决定, 而组织方式主要改变执行与输出时序。



(a) 整块译码 BER



(b) 整块译码 FER



(c) 连续组织 FER

图 8.6: 卷积码硬判决与浮点软判决的 BER 和 FER

表 8.6: 卷积码硬判决与浮点软判决的可靠性门限

组织方式	判决	FER=0.1/dB	FER=0.01/dB	相对硬判决增益/dB	说明
整块及三种连续组织	硬判决	1.367	2.489	0	1 bit 判决, 汉明距离分支度量
整块及三种连续组织	浮点软判决	-0.731	0.229	2.098/2.260	连续符号, 欧氏距离平方分支度量

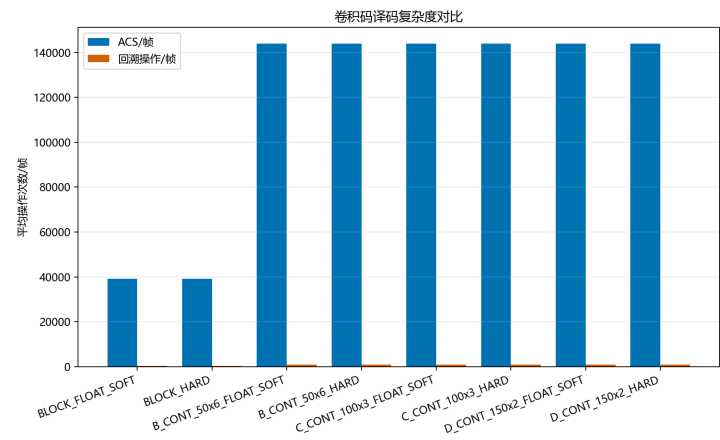
8.3.3 复杂度、存储与软件时间

硬、软 Viterbi 采用相同状态数、候选分支数、ACS 骨架和回溯骨架, 因此图??中同一组织的 ACS 与回溯计数相同。整块译码每帧 39168 次 ACS 和 306 次回溯操作; 连续组织因窗口重复处理, 每帧约 143998 次 ACS 和 866 次回溯操作。相同 ACS 计数不表示总计算代价相同: 硬判决分支度量以比特比较和汉明距离为主, 浮点软判决需要减法、平方和加法。模型内存由组织方式决定, 整块为 59776 byte, 连续窗口为 26112 byte, 判决精度在当前内存模型中没有改变这一数值。

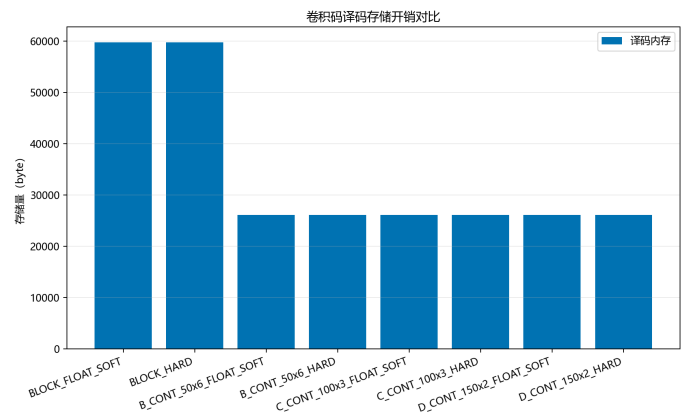
图??区分 CPU 软件译码时间、首输出时延和决策时延。整块硬、软判决的帧加权平均译码时间分别为 339.03 μ s 和 524.00 μ s, 浮点软判决增加 54.6%; 最大观测时间分别为 34231.5 μ s 和 65607.2 μ s。三种连续组织的硬判决加权平均时间为 1549.17–1565.58 μ s, 软判决为 1835.82–1849.69 μ s, 增加约 18%。连续组织的首输出时延为 298 或 398 个符号, 平均决策时延图所示值为 342、430 或 530 个符号; 二者由到达批次、窗口和回溯调度决定, 不能称为平均译码时间或最大译码时间。

表 8.7: 硬判决与浮点软判决 Viterbi 的计算结构和实现代价

判决	分支度量	共同骨架	整块加权平均/ μ s	整块最大/ μ s	工程特征
硬判决	0/1 比特与汉明距离	相同状态、候选分支、ACS 和回溯	339.03	34231.5	计算精度要求低, 可靠性损失较大
浮点软判决	连续符号与欧氏距离平方	相同状态、候选分支、ACS 和回溯	524.00	65607.2	保留幅值可靠性信息, 分支度量更重

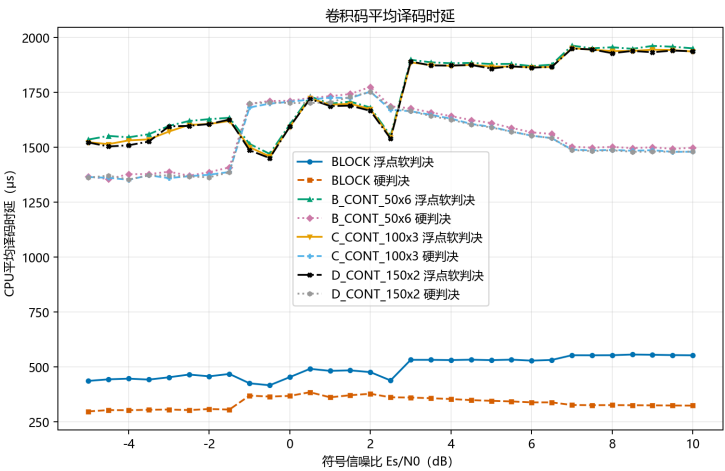


(a) ACS 与回溯操作

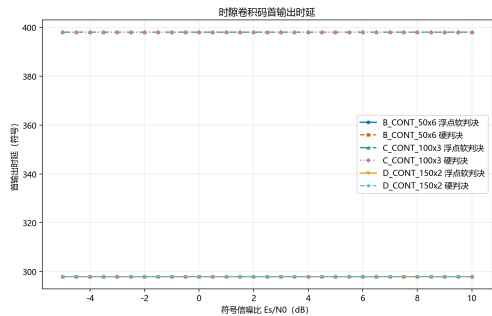


(b) 译码器内存

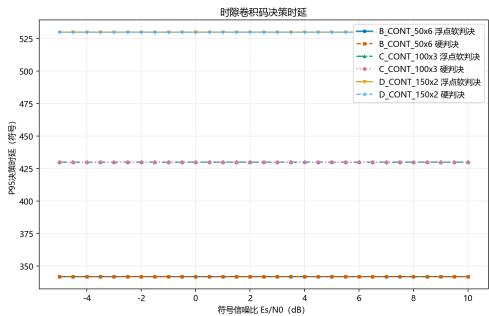
图 8.7: 卷积码 Viterbi 译码的操作规模与存储开销



(a) 平均 CPU 译码时间



(b) 首输出时延



(c) 决策时延

图 8.8: 卷积码 Viterbi 译码的软件时间与流式输出时延

8.3.4 卷积码工程结论

浮点软判决以更重的分支度量换取约 2.10–2.26 dB 的 FER 门限收益，适合链路可靠性优先、具备浮点或等效定点度量资源的接收机。硬判决降低输入精度和分支度量实现要求，适合资源或实时性更受约束且链路余量足够的场景。连续窗口方案减小模型内存并提供分批输出，但窗口重复计算使当前串行软件时间高于整块方案；首输出与决策时延应作为独立流式指标使用。

8.4 LDPC BP 与 NMS 译码对比

8.4.1 正式配置与配对条件

S6 采用 BG2 Direct QC-LDPC 的 N560 实例。老师给出的名义目标长度为 576 bit，Direct BG2 候选选择器最终映射到实际长度 560 bit，因此本章与图表均称为 N560。载荷为 300 bit，实际码率为 $300/560 = 0.5357$ ，提升因子 $Z_c = 56$ ，信息容量 448 bit，填充 148 bit，校验长度 112 bit，校验子矩阵秩为 112。BP 与 NMS 的最大迭代次数均为 32，均在完成一次完整迭代后计算综合征并提前停止，NMS 归一化系数为 $\alpha = 0.95$ 。

表 8.8: S6 LDPC 译码算法比较配置

项目	数值	项目	数值	说明
实际案例	N560	载荷/码长	300/560 bit	名义目标 576 映射到实际 560
实际码率	0.5357	Z_c	56	BG2 Direct QC-LDPC
填充/校验	148/112 bit	$\text{rank}(H_p)$	112	不使用速率匹配与交织
译码器	分层 BP、分层 NMS	最大迭代	32	完整迭代后综合征提前停止
NMS 系数	$\alpha = 0.95$	配对输入	载荷、码字、LLR	31 个信噪比点哈希一致

BP 和 NMS 共享同一载荷、码字、噪声、对数似然比 (Log-Likelihood

Ratio, LLR)、校验图、最大迭代次数和提前停止规则,只改变校验节点消息更新。逐信噪比的载荷、码字和 LLR 哈希均一致,使该对比能够把可靠性与时间差异较严格地归因于校验节点更新结构。

8.4.2 BER 与 FER

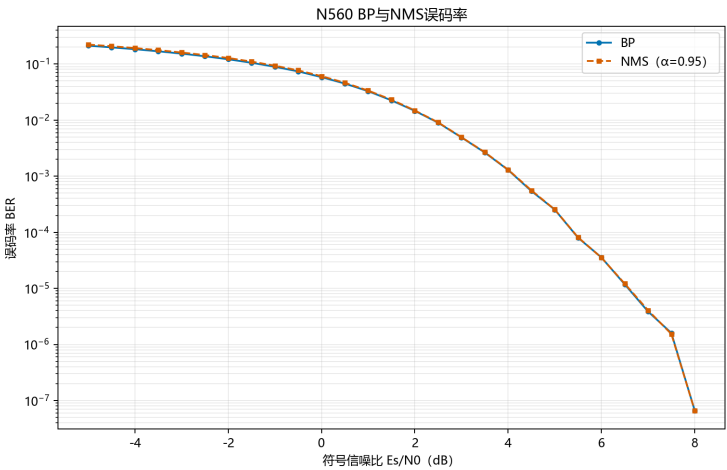
图??中 BP 与 NMS 曲线在瀑布区基本重合, NMS 表现出轻微性能损失。FER 为 0.1 时, BP 和 NMS 所需 E_s/N_0 分别为 4.541 dB 和 4.577 dB, NMS 损失 0.036 dB; FER 为 0.01 时, 两者分别为 5.866 dB 和 5.875 dB, 损失 0.010 dB。两种算法各有 4 个零 BER/FER 工作点, 零值没有用于门限外推。当前 $\alpha = 0.95$ 配置因此在两个目标 FER 处保留了接近 BP 的可靠性。

8.4.3 迭代与提前停止行为

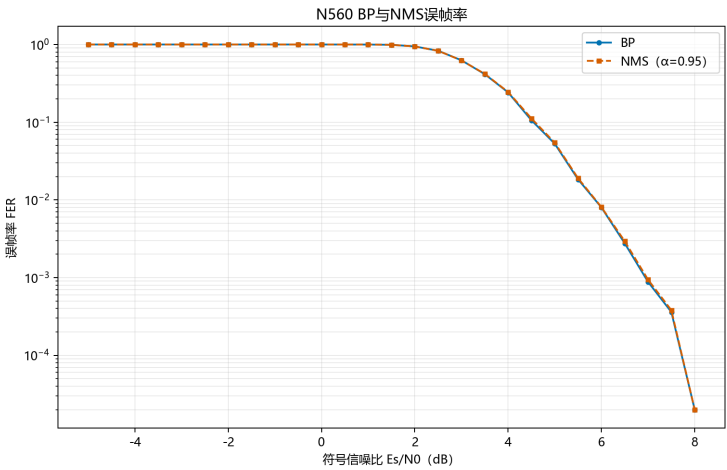
图??表明, 低信噪比区的大量帧达到 32 次上限, 进入瀑布区后平均迭代数随可靠性提高而下降, 提前停止率相应上升。31 点等权网格均值中, BP 和 NMS 平均迭代数分别为 16.44 和 13.11; 其逐点范围分别为 1–32 次和 1–28.47 次。NMS 在部分低信噪比点出现较高提前停止率, 其中可能包含错误码字满足校验的情况, 不能把“提前停止”单独解释为可靠译码成功。总软件代价同时取决于迭代次数和单次迭代计算结构, 不能只用平均迭代数判断复杂度。

8.4.4 复杂度、存储与软件译码时间

BP 校验节点更新包含双曲正切、反双曲正切及消息更新; NMS 以绝对值、符号、第一/第二最小值、比较和 α 缩放近似非线性更新。图??中的网格平均操作计数显示, BP 每帧约 25780.66 次双曲正切和 12890.33 次反双曲正切; NMS 每帧约 10280.70 次绝对值、20561.41 次比较和 6190.30 次最小值操作, 并包含符号与缩放。二者同为一次事件并不意味着处理器代价相同, 尤其不能把双曲函数和比较操作作为统一单位相加。

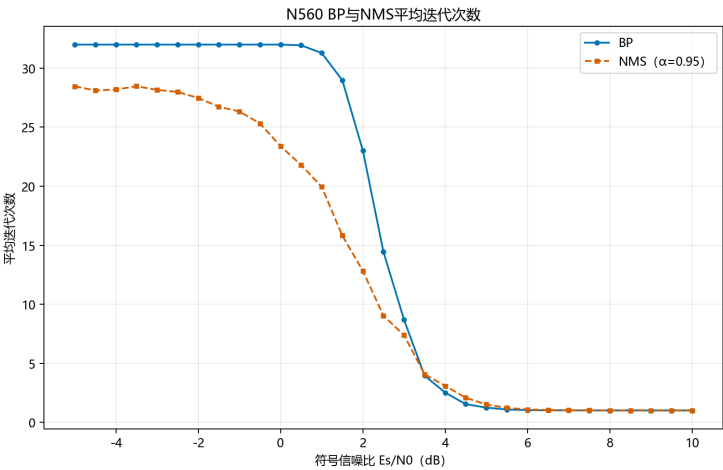


(a) 比特错误率

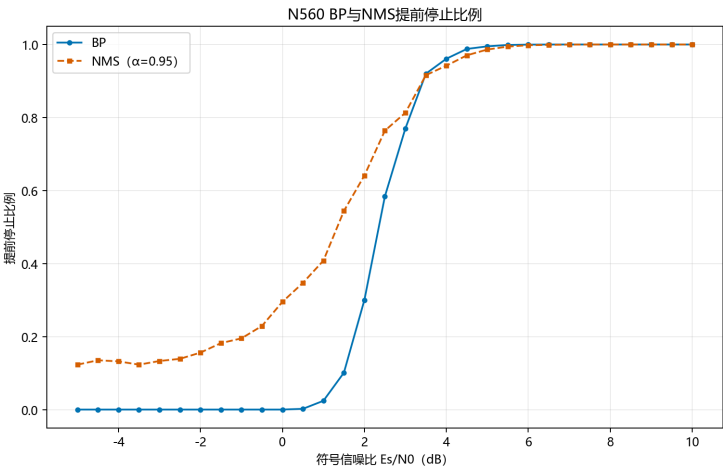


(b) 帧错误率

图 8.9: LDPC BP 与 NMS 译码的 BER 和 FER



(a) 平均迭代次数



(b) 提前停止率

图 8.10: LDPC BP 与 NMS 的迭代和提前停止行为

两种译码器的消息与后验缓冲布局相同，模型内存均为 11312 byte。按全部工作点帧数加权，BP 与 NMS 平均译码时间分别为 140.687 μ s 和 33.157 μ s，NMS 减少 76.4%，约为 BP 的 23.6%；最大观测时间分别为 34024.9 μ s 和 9468.0 μ s，NMS 减少 72.2%。逐点平均时间曲线还表明，低信噪比满迭代区域的 BP 时间可超过 2 ms，而 NMS 保持在 0.5 ms 以内。该差异同时来自更轻的单次校验节点更新和实际迭代行为。

表 8.9: LDPC BP 与 NMS 的可靠性、复杂度和时间指标

译码器	FER=0.1/dB	FER=0.01/dB	网格平均迭代	加权平均/ μ s	最大/ μ s	内存/byte	校验节点主要运算
BP	4.541	5.866	16.44	140.687	34024.9	11312	双曲正切、反双曲正切、消息更新
NMS	4.577	5.875	13.11	33.157	9468.0	11312	绝对值、比较、最小值、符号和缩放

8.4.5 LDPC 工程结论

BP 提供当前实现的可靠性基准，其非线性校验节点更新带来较高软件计算代价。NMS 以最小值近似和归一化系数简化校验节点更新，在 FER 为 0.1 和 0.01 时仅损失 0.036 dB 和 0.010 dB，同时将加权平均和最大观测软件译码时间分别降低 76.4% 和 72.2%，且不增加模型内存。对于本项目 N560、 $\alpha = 0.95$ 和 32 次迭代上限的配置，NMS 形成了可靠性损失很小而软件时间明显下降的工程替代；该判断不外推到其他码长、系数或硬件实现。

8.5 三类译码实现的工程特征综合

表??按编码族总结可靠性、计算、存储与时间特征，不建立跨族 BER 或操作计数排名。三类任务的载荷、码长、图结构和操作语义不同，BCH 的一次有限域乘法、Viterbi 的一次 ACS 与 LDPC 的一次比较不具有统一

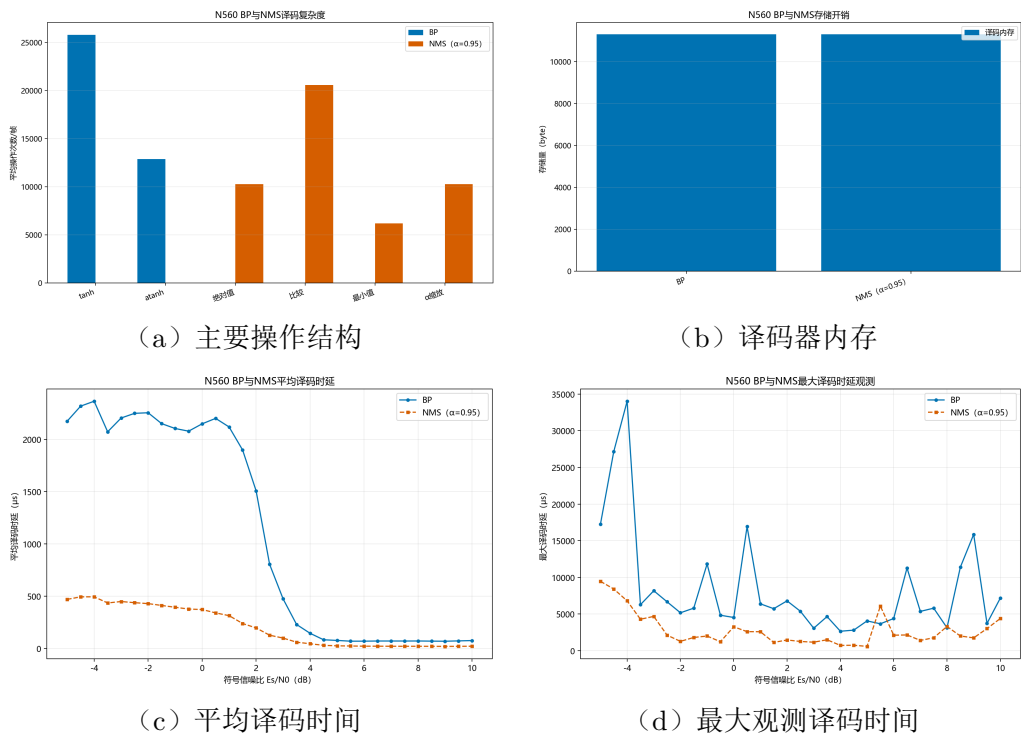


图 8.11: LDPC BP 与 NMS 的计算结构、存储和软件时间

成本。软件时间和模型内存虽然使用共同单位，也只反映当前软件环境和各自计时、建模范围，不能直接替代目标硬件评估。

表 8.10: 三类译码实现的工程特征汇总

编码族	译码器	可靠性特征	主要计算结构	存储特征	软件时间特征	适用倾向
BCH	分组查表	当前完整方案门限较高	短综合征、查表、翻转	固定表与小工作区	平均时间较低	结构固定、资源受限
BCH	BM 与 Chien	当前完整方案门限降低	综合征、BM、Chien、有限域运算	有限域表与代数状态增加	平均时间增加	整块纠错能力优先
卷积码	硬判决 Viterbi	损失幅值可靠性信息	汉明度量、ACS、回溯	与同组织软判决相同	分支度量较轻	资源或实时性优先
卷积码	浮点软判决 Viterbi	FER 门限改善约 2.1–2.3 dB	欧氏度量、ACS、回溯	与同组织硬判决相同	CPU 时间增加	可靠性优先
LDPC	分层 BP	可靠性基准	非线性校验节点和消息更新	消息缓冲 11312 byte	当前实现较慢	性能基准与高精度实现
LDPC	分层 NMS	门限损失不超过 0.04 dB	最小值、符号和归一化	消息缓冲 11312 byte	平均时间降低 76.4%	低复杂度软件实现

从同族选择看，BCH 需要在短分组查表的低实现代价与整块代数方案的纠错收益之间权衡；卷积码需要在软信息带来的约 2 dB 可靠性收益与分支度量开销之间权衡；LDPC 则体现出 NMS 以很小门限损失换取显著软件加速的特点。工程选型应把 BER、FER、操作结构、存储、平均时间和最大观测时间同时纳入，而不是只根据一条误码曲线作结论。

8.6 本章结论

BCH 比较表明，分组综合征查表适合短分组和固定结构，BCH-S200 以较小内存和较低软件时间完成译码；BM 与 Chien 搜索适合较长整块 BCH，BCH-B200 在 FER 为 0.01 时节省 1.671 dB，但平均译码时间和模型内存分别增加 70.5% 和 52.4%。该 BER/FER 差异属于两套完整方案的综合差异，不能纯归因于译码器。

卷积码比较表明，浮点软判决保留连续接收幅值的可靠性信息，在 FER 为 0.1 和 0.01 时相对硬判决分别获得 2.098 dB 和 2.260 dB 收益；代价是

整块加权平均 CPU 译码时间增加 54.6%，分支度量需要浮点减法、平方和加法。硬判决适合计算精度与实时资源更紧、同时链路余量允许可靠性损失的场景。连续方案的首输出和决策时延属于符号调度指标，不能与 CPU 墙钟时间混称。

LDPC 比较表明，BP 是当前 N560 配置的性能基准，NMS 通过最小值、符号和 $\alpha = 0.95$ 缩放替代非线性校验节点计算。NMS 在两个目标 FER 处的损失均小于 0.04 dB，加权平均和最大观测软件译码时间分别降低 76.4% 和 72.2%，模型内存保持不变。复杂度判断必须同时考虑单次迭代结构与实际迭代次数。

三类结果共同说明，译码器选择需要同时满足可靠性、计算复杂度、存储和实时性约束。误码曲线回答链路余量问题，操作计数揭示实现结构，平均时间描述常态软件负担，最大观测时间暴露尾部风险；只有在保持各指标定义与比较边界的前提下，才能形成可追溯的工程选择。

第九章 交织抗突发错误分析

TODO: 依据 S7 正式数据和突发模型实现编写。本节所需证据：S7 CSV、源码、参数；待补 Visio：交织位置和交织器图。

第十章 综合方案推荐

TODO: 只有完成结论-证据矩阵复核后才可写入推荐。本节所需证据: 结论证据矩阵; 待补 Visio: 最终方案选择图。

第十一章 结论与后续建议

TODO: 不得在证据台账仍为待复核时写入结论。本节所需证据：最终候选、差异登记、缺失清单；待补 Visio：无。

附录 A 完整参数表

本附录列出已经由 Common 文档、源码和任务审计材料确认的公共参数。BCH 参数已随第??章冻结；CC、LDPC 及后续任务仍在对应章节完成证据审计后补充，不用规划值代替正式配置。

A.1 Common 长度与码率字段

表 A.1: Common 长度、附加位和码率参数

字段	冻结值或类型	含义
payloadLength	200 或 300 bit	原始有效 payload 长度
codecInputLength	任务确定	编码器实际输入长度
encodedLength	正整数, 不超过任务约束	Common 码率分母
transmittedLength	正整数, 任务扩展输出	实际进入 BPSK 和信道的长度
filler/crc/tailLength	非负整数	不增加 payload 码率分子
打孔/缩短长度	非负整数	分别记录打孔和缩短规模
codeRate	$K_{\text{payload}}/N_{\text{encoded}}$	由 <code>computeCodeRate</code> 计算
maxCodeBlockLength	1000 bit	教师要求和 Common 配置的单块上限

A.2 Common 调制、噪声与停止参数

表 A.2: Common 基础仿真参数

参数	冻结值	说明
BPSK mapping	$0 \mapsto +1, 1 \mapsto -1$	实值离散符号模型
SNR input	Eb/N0 in dB	<code>computeAwgnSigma</code> 输入
AWGN sigma	$\sqrt{1/(2R10^{EbN0dB/10})}$	返回标准差
hard decision	$y \geq 0 \mapsto 0; y < 0 \mapsto 1$	零点判 0
LLR	$2y/\sigma^2$	正值倾向 bit 0
masterNoiseSeed	uint64	标准高斯主种子
noiseGroupId	uint64, 默认 0	公平对比组身份
noisePolicyVersion	1	当前噪声策略版本
NoiseKey	seed、group、frame、symbol、version	不包含 snrIndex
formal minFrames	5000	达到前不按目标错误数停止
formal targetFrameErrors	200	达到最小帧数后生效
formal maxFrames	50000	优先级最高的上限判断

A.3 Pool、Schema 和时延字段

基础结果字段包括实验身份、payload/encoded 长度、codeRate、Eb/N0、SNR 索引、处理帧数、payload 总比特、错误计数、成功帧、BER、FER、successRate、基础时延、stopReason、pool ID 和 configHash。transmittedLength、误纠、声明成功率、置信区间、零错误标志、迭代次数、复杂度和交织参数由任务 Schema 扩展。

Common 基础时延覆盖 encode、channel、decode、recovery 和 total 阶段的平均时间，并保存 decode 与 total 的最大观测时间。identity pipeline

表 A.3: Common pool 及结果身份参数

项目	冻结值	说明
Frame Pool schema	manifest v2	完整标识为 common03 frame pool manifest v2; packed bits, LSB-first
Frame generator	splitmix64_payload_v2	payloadPolicyVersion=1
Frame shard size	默认 1000 帧	单片 SHA-256 和 overall hash
Noise Pool schema	manifest v1	完整标识为 common04 noise pool manifest v1; float64 little-endian
Noise generator	splitmix64_box_muller_v1	保存未缩放标准高斯样本
Noise shard size	默认 1000 帧	版本化头、分片 SHA-256
Checkpoint schema	common04.checkpoint.v1	保存身份、进度、计数和累计时间
Result schema	common04.result_summary.v1	基础 SummaryRow
Metadata schema	common04.metadata.v1	createdTime 仅用于运行时元数据
Config hash	SHA-256 canonical config	排除时间和临时路径

的前两项包含 payload 或噪声生成/池读取，不能直接解释为排除 I/O 的纯编码和纯信道算法时间。

A.4 BCH 母码与长度适配参数

表 A.4: BCH 正式方案的完整长度参数

方案	K_p	母码 n	母码 k	t	子块	填充/缩短	输出/说明
K200_S15	200	15	11	1	19	填充 9	285, 逐块查表
K200_M255K207	200	255	207	6	1	缩短 7	248, $GF(2^8)$
K200_M511K421	200	511	421	10	1	缩短 221	290, $GF(2^9)$
K200_M511K385	200	511	385	14	1	缩短 185	326, $GF(2^9)$
K300_S15	300	15	11	1	28	填充 8	420, 逐块查表
K300_M255K207	300	255	207	6	2	每块缩短 57	396, 每块 150 bit
K300_M511K421	300	511	421	10	1	缩短 121	390, $GF(2^9)$
K300_M511K385	300	511	385	14	1	缩短 85	426, $GF(2^9)$

BCH(511,421) 与 BCH(511,385) 使用本原多项式表示 $0x211$ ；实际码率均按 payload 除以输出长度计算。300 bit 的 BCH(255,207) 方案是两个独立缩短块，不应误写为一个 396 bit 母码。

A.5 LDPC 正式参数

表 A.5: LDPC 正式配置摘要

输入	BG	实际码长	Z_c	$k_b/n_b/m_b$	填充位	码率	最大迭代	NMS α	信噪比
300 bit	2	480	48	8/10/2	84	0.6250	32	0.95	$E_s/N_0 = -5 \sim 10$ dB
300 bit	2	560 (目标 576)	56	8/10/2	148	0.5357	32	0.95	$E_s/N_0 = -5 \sim 10$ dB
300 bit	2	640	40	8/16/8	20	0.4688	32	0.80	$E_s/N_0 = -5 \sim 10$ dB

表中 α 为大规模性能仿真使用的归一化因子。归一化因子对 FER、迭代次数、软件译码时间和错误合法码字率的影响见第??章。

附录 B 补充结果图

本附录补充正文未采用、但能提供独立信息的 BER 曲线。所有曲线均来自正式 CSV，横轴采用 E_s/N_0 ；零观测错误不替换为人为非零值。

B.1 BCH 在 AWGN 下的 BER

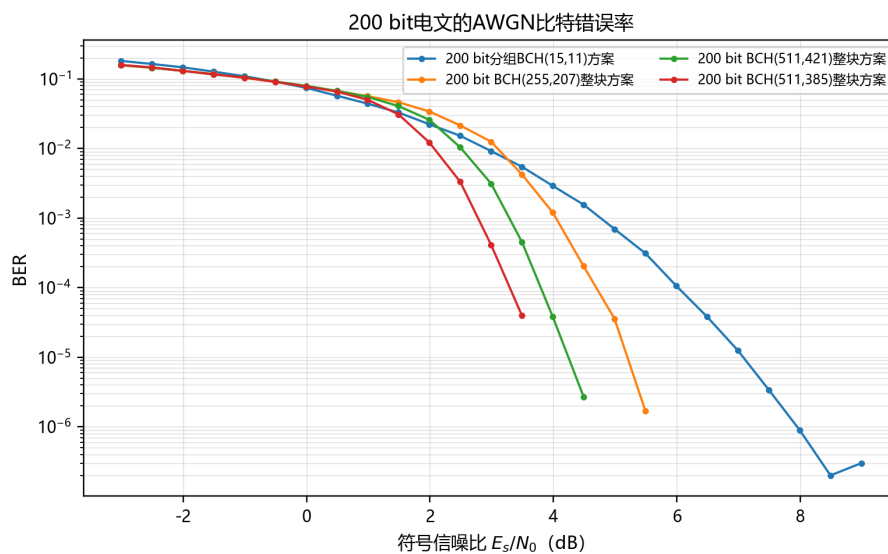


图 B.1: 200 bit 电文的 AWGN 比特错误率

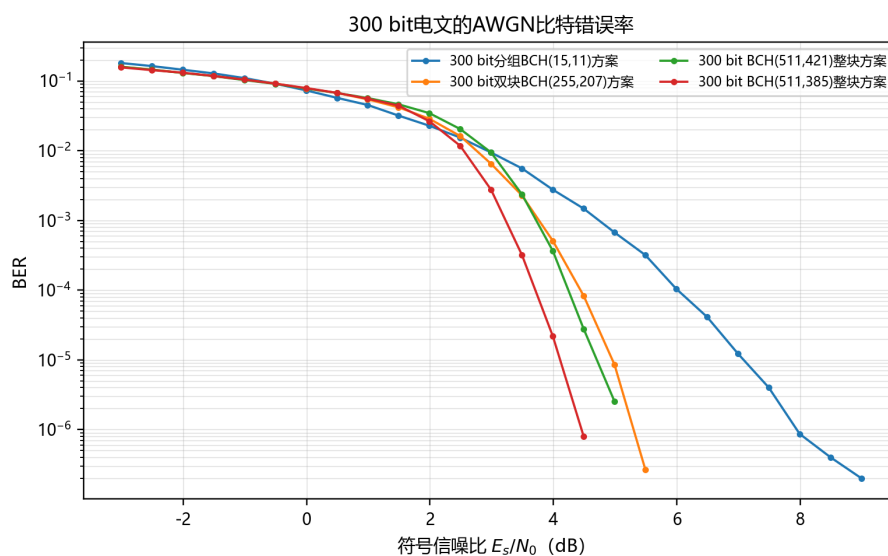


图 B.2: 300 bit 电文的 AWGN 比特错误率

B.2 卷积码时隙组织的 BER

第??章正文以 300 bit 电文是否完整交付为主，保留硬、软判决 FER 图；对应 BER 组合图置于本附录，用于观察错误比特密度而不重复扩张正文。

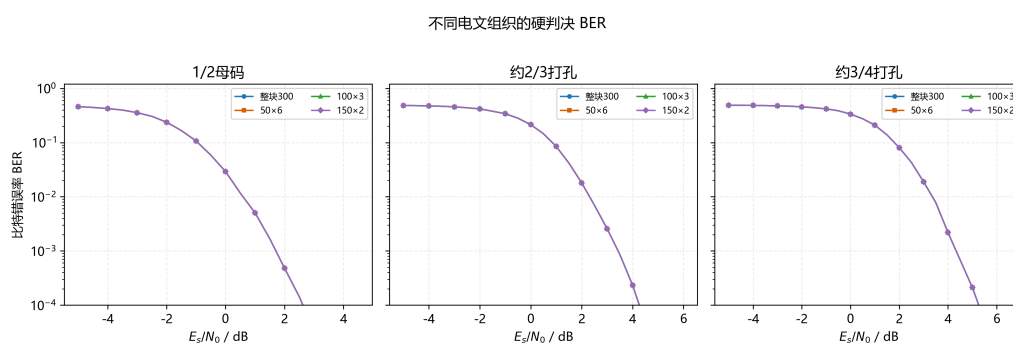


图 B.3: 不同电文组织的硬判决比特错误率

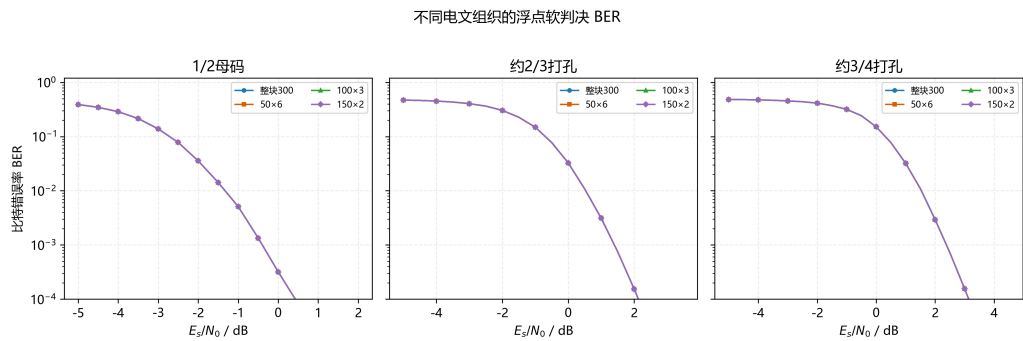


图 B.4: 不同电文组织的浮点软判决比特错误率

附录 C 复杂度统计口径

TODO: 依据 S6 环境记录、统计脚本与正式结果复核。

附录 D 仿真可复现性说明

D.1 随机输入与噪声身份

可复现性首先要求一帧 payload 和一条标准高斯噪声能够由稳定身份重新定位。Frame Pool 的 payload 由主种子、payload 长度、帧索引和 payload 策略版本确定；Noise Pool 的样本由主噪声种子、noiseGroupId、frameIndex、symbolIndex 和 noisePolicyVersion 确定。两套派生过程使用不同的域和算法标识，避免把 payload 随机字与噪声随机字混为同一键空间。

NoiseKey 不包含 SNR、码型、译码器名称、进程号、线程号或调用顺序。改变 E_b/N_0 时，Common 保持标准高斯 z 不变并通过 σ 缩放；改变 frameIndex 时生成另一帧噪声。同一公平对比组可据此共享噪声样本，而硬/软 Viterbi 或 BP/NMS 不会因为译码器名称不同重新过信道。

D.2 Frame Pool 与 Noise Pool

Frame Pool 只保存 payload bits。manifest 记录 schemaVersion、frame-PoolId、payloadLength、总帧数、分片大小、masterSeed、payloadPolicyVersion、生成算法、位存储方式、每帧字节数、各分片范围和 SHA-256。payload 按 LSB-first 打包，未使用的高位固定为 0。overall hash 对稳定的 manifest 身份字段和各分片 hash 计算，排除绝对路径、用户名、临时目录和创建时间。

Noise Pool 分片由固定头和 float64 little-endian 样本构成。头中保存

magic、版本、首帧、帧数、每帧符号数、主噪声种子、噪声组和策略版本；样本按 frame-major 顺序排列。读取器先验证 manifest，再验证完整分片 SHA-256 和头字段。noisePoolId 由 overall hash 派生，用于结果、checkpoint 和配置之间的身份绑定。

大型正式 Frame Pool 和 Noise Pool 不进入 Git。仓库只保存格式、生成算法、参数、hash 摘要和必要的小型 fixture；正式运行环境可在外部目录重新生成或分发大文件，但结果必须记录对应 pool ID 及 manifest 位置。

D.3 Manifest、SHA-256 与 configHash

Pool manifest 验证文件内容身份，任务 manifest 验证源码、配置和结果的审计范围，二者用途不同。每个二进制分片的 SHA-256 覆盖完整文件内容；overall hash 覆盖稳定 manifest 字段和分片 hash。文件名或分片范围不一致、重复、重叠、空洞、错误字节数以及 hash 不匹配都应拒绝读取。

Common-04 的 configHash 由规范化文本计算，当前包含 caseName、payloadLength、encodedLength、Eb/N0 列表、frame/noise pool ID、停止配置和译码输入模式。createdTime、临时目录和运行平台不进入 configHash。具体编码任务增加配置字段时，应建立自己的规范化顺序，并保存算法版本、码型参数、量化、信道和停止规则。

D.4 Checkpoint 与 resume

Checkpoint 保存下一帧索引而不是仅保存“完成百分比”。恢复前验证 schemaVersion、experimentId、configHash、framePoolId、noisePoolId、payloadLength、encodedLength、snrIndex、Eb/N0、noisePolicyVersion、globalSeed、shardIndex 和 shardCount；随后检查 nextFrameIndex 位于请求范围内，processedFrames 与已经跨过的帧数相等，totalPayloadBits 与 payload 长度一致，成功帧和错误帧之和等于已处理帧数。

Common checkpoint JSON 还保存各阶段时间累计、最大时间、reported

success、miscorrection、decoder failure 及译码状态计数。公共结构支持这些字段并不意味着每个 runner 都会填充非零真实值；任务必须用中断前后对比测试确认实际写入字段。Common-04 identity 测试验证了连续运行与 37 帧 checkpoint 后恢复到 100 帧的计数等价性，但具体码译码器仍需独立进行 resume 测试。

写入时先创建目标路径加 .tmp 后缀的临时文件，完成 flush 后再替换正式 checkpoint。Windows 使用 MoveFileExW，并设置 MOVEFILE_REPLACE_EXISTING 与 MOVEFILE_WRITE_THROUGH 两个标志。替换失败时删除临时文件并报错，恢复逻辑不得加载半写入文件。

D.5 Shard 执行与 merge

仿真分片用 shardIndex、frameStart 和 frameCount 描述。合并前按 frameStart 排序，要求 shardIndex 唯一、每片非空、帧范围连续且没有重叠或空洞。experimentId、configHash、pool ID、payload/encoded 长度、snrIndex 和 Eb/N0 必须完全一致。

每片 processedFrames 应等于 frameCount，totalPayloadBits 应等于 frameCount 乘 payloadLength，successfulFrames 加 frameErrors 应等于 frameCount。合并时计数与时间和相加，最大时间取最大值；BER、FER、successRate 和平均时间从合并后的总量重新计算。任务扩展的误纠、声明状态、迭代和复杂度计数也应遵循“先合并计数、后计算比例”的规则。

D.6 CSV 身份字段与结果追溯

基础 CSV 至少保留 schemaVersion、experimentId、stage、codeType、caseName、payloadLength、encodedLength、codeRate、ebN0_dB、snrIndex、processedFrames、原始错误计数、基础时延、stopReason、framePoolId、noisePoolId 和 configHash。正式绘图数据另保留 esN0Db、actualRate、snrInputType 和 plotXAxisType；Es/N0 只作为展示坐标，不覆盖原始 Eb/N0。

任务级 CSV 需增加 `transmittedLength`、附加位、译码器声明状态、误纠、迭代、复杂度、量化和交织参数。任何派生图表都应能回查到原始 CSV 行、配置 hash、pool ID、执行命令及 Git 功能提交。

D.7 Git 记录与大型资产原则

源码和小型配置由 Git 功能提交保存，Stage 审计文件记录被审计的功能提交 SHA，不记录审计提交自身 SHA。批次分支中的单个 Stage 只使用 manifest 声明的 functional range，不把整个 `main...HEAD` 误当作自身功能边界。报告引用实验结论时，记录对应代码版本、任务 Stage 和正式结果路径。

大型池文件、正式 checkpoint、formal 中间结果、build 目录、可执行文件和临时绘图文件默认不提交。可以提交小型 golden vector、fixture hash、冻结配置和结果摘要，但必须在 Stage 计划中明确用途。未进入 Git 的大型资产依靠 manifest、SHA-256、pool ID、configHash 和命令记录完成身份核验，而不是依靠文件修改时间。

D.8 BCH 第 4 章结果追溯

S1 最高权威目录及主数据为：

- `Task/BCH/simulation/stages/S1/stage07_awgn_dense_formal;`
- `published_results/stage07_awgn_dense_formal_results.csv。`

S2 按场景直接使用多径 common-SNR、CFO、遮挡和突发四个 formal 目录中的正式 CSV；未采用历史 integration 汇总替换这些结果。完整映射见 `Report/audit/round05_bch_write/01_authoritative_result_sources.md`。

第 4 章报告版图源位于 `Report/data/chapter04/bch`，图位于 `Report/figures/bch`，生成入口见 `Report/scripts/build_chapter04_bch_assets.py`。S1 和固定多径的原始 waveform-SNR 按 $E_s/N_0 = \text{sourceSnrDb} - 10 \log_{10} 2$ 换算；相位漂移和遮

挡 CSV 中的 `snrDb` 已由 $E_b/N_0 + 10 \log_{10} R$ 得到，直接作为 E_s/N_0 。派生 CSV 保留来源路径和原始坐标，BER、FER 与处理帧数不作改写。

本轮报告基于分支 `S8-PaperDocu`、起始提交 `c1a8238a` 生成；Round05-B 未重跑 formal、未修改 Task 目录，也未创建提交。

D.9 当前适用边界

Common-04 已验证 identity 基线、pool-backed smoke/prescan、checkpoint 恢复、C++/Python 噪声一致性及分片合并，但没有在 Common Stage 运行 BCH、卷积码或 LDPC 正式曲线。Wilson 区间、零错误标志和 P95 等字段在 Common-01 定义中冻结，尚未进入 Common-04 基础 Summary-Row。后续任务应在使用前确认实现，不得把文档中的预留定义当作已经生成的结果。

本附录说明公共机制和复现步骤；具体 BCH、CC、LDPC 以及 S1-S7 的命令、配置、数据 hash 和功能提交由相应章节及任务审计文件补充。